

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf\_49c1018a-e67\agent-acf279e2c2ff06aea.jsonl`
- SHA-256: `e11ba606e0a7b8c7c3325e28b8c399a30b5a64b9b6d5d255a4cb811561699386`
- Source modified: `2026-06-18T01:40:32+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_49c1018a-e67`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:37:03.856Z)

Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read READ-ONLY in this worktree (origin/master), DO NOT modify:

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
```

A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.

You are the COMPLETENESS CRITIC. Two hunt passes (by-area, then by-class) found nothing so far. Identify the code surface most likely UNDER-examined (e.g. a module nobody opened, an error path, a rarely-exercised branch, generated/serialized output) and HUNT IT directly. Read the code. Report only real, reachable defects with exact file+line and a concrete trigger; empty list is fine.

2. assistant (2026-06-18T01:37:06.583Z)

I'll hunt the under-examined code surface. Let me start by orienting on the repo structure and identifying what's least likely to have been examined.

3. user (2026-06-18T01:37:08.250Z)

4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)

```
---
CLAUDE.md
README.md
__pycache__
backend
config.json
docs
eval_baselines
mypy.ini
pyproject.toml
pytest.ini
requirements.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
```

tests
tools
training

4. user (2026-06-18T01:37:09.189Z)

backend/annotations/automated_provider.py
backend/annotations/base.py
backend/annotations/file_provider.py
backend/annotations/__init__.py
backend/config/loader.py
backend/config/models.py
backend/config/__init__.py
backend/eval/ablation.py
backend/eval/annotations.py
backend/eval/audio/e1_probe.py
backend/eval/audio/__init__.py
backend/eval/batch.py
backend/eval/calibrate_local.py
backend/eval/calibrate_wasb.py
backend/eval/calibration.py
backend/eval/classifier.py
backend/eval/distill_local.py
backend/eval/eval_partial_labels.py
backend/eval/eval_stats.py
backend/eval/experiment.py
backend/eval/export_wasb_segments.py
backend/eval/fusion_audio.py
backend/eval/fusion_compare.py
backend/eval/fusion_golden.py
backend/eval/gemini_refine.py
backend/eval/golden_fixtures.py
backend/eval/gt_loader.py
backend/eval/harness.py
backend/eval/metrics.py
backend/eval/per_user_eval.py
backend/eval/per_user_gate.py
backend/eval/per_user_regression.py
backend/eval/promotion.py
backend/eval/rally_seq_eval.py
backend/eval/rally_seq_proto.py
backend/eval/regression.py
backend/eval/score_calibration.py
backend/eval/segmentation_metrics.py
backend/eval/served_gate_a.py
backend/eval/serve_contrast.py
backend/eval/splits.py
backend/eval/tolerance_metrics.py
backend/eval/training.py
backend/eval/windowing.py
backend/eval/__init__.py
backend/features/rally_seq.py
backend/features/__init__.py
backend/infrastructure/database.py
backend/infrastructure/execution_policy.py
backend/main.py
backend/pipeline/audio/hit_detector.py
backend/pipeline/audio/__init__.py
backend/pipeline/detectors/audio_features.py
backend/pipeline/detectors/base.py
backend/pipeline/detectors/fusion_features.py
backend/pipeline/detectors/person_detector.py
backend/pipeline/detectors/rally_gate.py
backend/pipeline/detectors/tracknet_runner.py
backend/pipeline/detectors/wasb_infer.py

```
backend/pipeline/detectors/wasb_runner.py
backend/pipeline/detectors/__init__.py
backend/pipeline/personalization/serving.py
backend/pipeline/personalization/__init__.py
backend/pipeline/segmenters/base.py
backend/pipeline/segmenters/fusion_hybrid.py
backend/pipeline/segmenters/gemini.py
backend/pipeline/segmenters/motion.py
backend/pipeline/segmenters/tracknet_hybrid.py
backend/pipeline/segmenters/trajectory_hybrid.py
backend/pipeline/segmenters/wasb_hybrid.py
backend/pipeline/segmenters/yolo_hybrid.py
backend/pipeline/segmenters/__init__.py
backend/pipeline/stitcher/compiler.py
backend/pipeline/validators/base.py
backend/pipeline/validators/blur.py
backend/pipeline/validators/format.py
backend/pipeline/validators/pts_continuity.py
backend/pipeline/validators/relevance.py
backend/pipeline/validators/resolution_fps.py
backend/pipeline/validators/scene_cut.py
backend/pipeline/validators/__init__.py
backend/providers/base.py
backend/providers/gemini.py
backend/providers/__init__.py
backend/reporting/harvest.py
backend/reporting/run_signals.py
backend/reporting/__init__.py
backend/results.py
backend/sports/badminton.py
backend/sports/base.py
backend/sports/__init__.py
backend/tools/export.py
backend/tools/rally_slicer.py
backend/tools/__init__.py
backend/utils/geometry.py
backend/utils/hashing.py
backend/utils/logging_setup.py
backend/utils/paths.py
backend/utils/run_telemetry.py
backend/utils/video.py
backend/utils/__init__.py
```

5. assistant (2026-06-18T01:37:13.095Z)

Let me focus on under-examined surfaces: serialization/output paths, error handlers, and rarely-run modules. Let me look at a few candidates that a precision pass and a class pass might skip ? the compiler, harvest/reporting, export tools, and database migrations.

6. user (2026-06-18T01:37:14.053Z)

```
1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12    # `font='Sans` fails). Overridable via stitching.watermark.font_path. This module lives
```

```

at
13     # backend/pipeline/stitcher/, so parents[2] == backend/.
14     _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16     class VideoCompiler:
17         def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                 watermark: Optional[Any] = None):
19             self.output_dir = output_dir
20             self.end_padding = end_padding
21             self.begin_padding = begin_padding
22             # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23             # normalized to a dict (or None) so this module stays config-package-agnostic.
24             self.watermark = self._normalize_watermark(watermark)
25             os.makedirs(output_dir, exist_ok=True)
26
27         @staticmethod
28         def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29             if watermark is None:
30                 return None
31             if hasattr(watermark, "model_dump"):
32                 return watermark.model_dump()
33             if isinstance(watermark, dict):
34                 return dict(watermark)
35             return None
36
37         @staticmethod
38         def _ff_quote(path: str) -> str:
39             """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40             filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41             slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43             The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44             option parser treats ':' as the option separator, so a Windows drive-letter
45             path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46             the encode fails with "Invalid argument". This silently disabled the
47             on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48             return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50         @staticmethod
51         def _parse_time(val: Union[int, float, str]) -> float:
52             """Normalizes a timestamp value to float seconds.
53             Handles: float/int (pass-through), plain numeric strings ("12.4"),
54             and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55             This mirrors the parse_time logic used in gemini.py to ensure
56             the stitcher can always consume whatever format the indexer produced."""
57             if isinstance(val, (int, float)):
58                 return float(val)
59             if isinstance(val, str):
60                 val = val.strip()
61                 if ":" in val:
62                     # Handle MM:SS, HH:MM:SS, etc.
63                     parts = val.split(":")
64                     parts.reverse()
65                     total = 0.0
66                     for i, p in enumerate(parts):
67                         try:
68                             total += float(p) * (60 ** i)
69                         except ValueError:
70                             pass
71                     return total
72                 try:
73                     return float(val)
74                 except ValueError:

```

```

75         logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
76         return 0.0
77         logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
78         return 0.0
79
80     @staticmethod
81     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
82         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
83
84         A highlight reel must never replay the same footage. The same rally can land in
85         the segment list more than once ? e.g. two detector runs accumulated for one
86         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
87         or
88         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
89         rally
90         twice. Merging any range that overlaps or abuts the previous one collapses true
91         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
92         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
93         """
94         parsed = []
95         for raw_s, raw_e in segments:
96             s = VideoCompiler._parse_time(raw_s)
97             e = VideoCompiler._parse_time(raw_e)
98             if e > s:
99                 parsed.append((s, e))
100         parsed.sort()
101         merged: List[Tuple[float, float]] = []
102         for s, e in parsed:
103             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
104                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
105             else:
106                 merged.append((s, e))
107         return merged
108
109     def _get_video_duration(self, video_path: str) -> float:
110         """Probes the video file to get its total duration in seconds.
111         Returns 0.0 if it cannot be determined."""
112         try:
113             result = subprocess.run(
114                 [
115                     "ffprobe", "-v", "error",
116                     "-show_entries", "format=duration",
117                     "-of", "default=noprint_wrappers=1:nokey=1",
118                     video_path
119                 ],
120                 capture_output=True, text=True, check=True
121             )
122             return float(result.stdout.strip())
123         except Exception as e:
124             logger.warning(f"Could not determine video duration via ffprobe: {e}")
125             return 0.0
126
127     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
128         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
129         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
130         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
131         because the mark is already baked into every clip identically."""
132         wm = self.watermark
133         if not wm or not wm.get("enabled"):
134             return None
135
136         text = (wm.get("text") or "").strip()

```

```

135         logo_path = (wm.get("logo_path") or "").strip()
136         if logo_path and not os.path.exists(logo_path):
137             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")
138             logo_path = ""
139         if not text and not logo_path:
140             return None
141
142         margin = int(wm.get("margin", 24))
143         opacity = float(wm.get("opacity", 0.85))
144         position = str(wm.get("position", "bottom-right"))
145         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
146         dt_xy = {
147             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
148             "bottom-left": f"x={margin}:y=h-th-{margin}",
149             "top-right": f"x=w-tw-{margin}:y={margin}",
150             "top-left": f"x={margin}:y={margin}",
151         }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
152         ov_xy = {
153             "bottom-right": f"W-w-{margin}:H-h-{margin}",
154             "bottom-left": f"{margin}:H-h-{margin}",
155             "top-right": f"W-w-{margin}:{margin}",
156             "top-left": f"{margin}:{margin}",
157         }.get(position, f"W-w-{margin}:H-h-{margin}")
158
159         drawtext = ""
160         if text:
161             ratio = float(wm.get("font_size_ratio") or 0.035)
162             divisor = max(1, round(1.0 / ratio))
163             # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
164             tf_path = os.path.join(tmp_dir, "watermark.txt")
165             with open(tf_path, "w", encoding="utf-8") as fh:
166                 fh.write(text)
167             # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
168             # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
169             font_path = (wm.get("font_path") or "").strip()
170             if not font_path and _BUNDLED_FONT.exists():
171                 font_path = str(_BUNDLED_FONT)
172             if font_path:
173                 font_opt = f"fontfile='{self._ff_quote(font_path)}'"
174             else:
175                 font_opt = f"font='{wm.get('font', 'Sans')}'"
176             box = ""
177             if wm.get("box", True):
178                 box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
179             drawtext = (
180                 f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181                 f"fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
182             )
183
184         if logo_path and drawtext:
185             return
186         f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
187         if logo_path:
188             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
189         return drawtext
190
191     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
192         """
193         Extracts selected segments from a raw video and stitches them together
into a seamless highlights file, applying end_padding to prevent abrupt endings.

```

```

194         """
195         if not segments:
196             raise ValueError("No highlight segments provided to compile.")
197
198         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
199         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
200         original_count = len(segments)
201         segments = self._merge_overlapping(segments)
202         if len(segments) < original_count:
203             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
-> %d.",
204                         original_count - len(segments), original_count, len(segments))
205         if not segments:
206             raise ValueError("No valid (start < end) highlight segments after merge.")
207
208         if not os.path.exists(raw_video_path):
209             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
{raw_video_path}")
210
211         # Defense-in-depth: never let the output name escape output_dir (the API also
212         # validates this at the request boundary).
213         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
214
215         # Probe video duration so we can detect out-of-range segments
216         video_duration = self._get_video_duration(raw_video_path)
217         if video_duration > 0:
218             logger.info(f"Source video duration: {video_duration:.2f}s")
219         else:
220             logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
221
222         # We will create temporary clips and stitch them using FFmpeg concat demuxer
223         temp_clips = []
224         skipped_segments = []
225
226         # Create a temp directory within output_dir
227         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
228             logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
229             # Build the (optional) watermark filtergraph once; it is applied during
every
230             # per-clip re-encode so the mark is baked into each clip identically.
231             watermark_filter = self._watermark_filter(tmp_dir)
232             if watermark_filter:
233                 logger.info("[watermark] applying branding overlay to each clip.")
234             for idx, (raw_start, raw_end) in enumerate(segments):
235                 # Normalize timestamps to float seconds ? handles float, int,
236                 # plain numeric strings, and colon-separated formats like "MM:SS"
237                 start = self._parse_time(raw_start)
238                 end = self._parse_time(raw_end)
239                 segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
240
241                 # Validate the segment times
242                 if start < 0:
243                     logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
244                     start = 0.0
245
246                 if start >= end:
247                     logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
248                     skipped_segments.append((idx, start, end, "start >= end"))
249                     continue
250

```

```

251         # Check if segment extends beyond video duration
252         if video_duration > 0:
253             if start >= video_duration:
254                 logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
255                             f"This segment cannot be extracted ? the video ran
out. Skipping.")
256                 skipped_segments.append((idx, start, end, "start beyond video
end"))
257                 continue
258
259             if end > video_duration:
260                 original_end = end
261                 end = video_duration
262                 logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
263                             f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")
264
265                 clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
266
267                 # Precise sub-second trim using fast re-encoding to preserve stream
headers
268                 padded_start = max(0.0, start - self.begin_padding)
269                 padded_end = end + self.end_padding
270
271                 # Clamp padded_end to video duration if known
272                 if video_duration > 0 and padded_end > video_duration:
273                     padded_end = video_duration
274                     logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
275                             f"Clamping to {video_duration:.2f}s (end padding
partially applied).")
276
277                 trim_duration = padded_end - padded_start
278                 logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
279
280                 base_cmd = [
281                     "ffmpeg", "-y",
282                     "-ss", str(round(padded_start, 3)),
283                     "-to", str(round(padded_end, 3)),
284                     "-i", raw_video_path,
285                     "-c:v", "libx264",
286                     "-preset", "superfast",
287                     "-crf", "22",
288                     "-c:a", "aac",
289                     "-b:a", "128k",
290                 ]
291                 vf_args = ["-vf", watermark_filter] if watermark_filter else []
292
293                 # Try with the watermark first; if that encode fails (e.g. a bad
font/logo
294                 # path), retry once WITHOUT it so a branding misconfig can never nuke
the
295                 # whole reel. Clips stay codec-identical either way, so concat -c copy
holds.
296                 attempts = [vf_args, []] if vf_args else [[]]
297                 produced = False
298                 last_exc: Optional[subprocess.CalledProcessError] = None
299                 cmd = base_cmd + [clip_path]
300                 for attempt_vf in attempts:
301                     cmd = base_cmd + attempt_vf + [clip_path]
302                     try:
303                         subprocess.run(cmd, capture_output=True, check=True)

```



```

304         except subprocess.CalledProcessError as e:
305             last_exc = e
306             if attempt_vf and vf_args:
307                 _err = (e.stderr.decode(errors="replace").strip()[-300:]
308                     if getattr(e, "stderr", None) else "")
309                 logger.warning(
310                     f"{segment_label}: watermark encode failed; retrying
without it. "
311                     f"ffmpeg: {_err or '(no stderr captured)'}"
312                 )
313                 continue
314             break
315             if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
316                 produced = True
317                 if not attempt_vf and vf_args:
318                     logger.warning(f"{segment_label}: encoded WITHOUT the
watermark (the watermark filter failed).")
319                     break
320                 last_exc = None # ffmpeg succeeded but produced nothing
321                 break
322
323             if produced:
324                 temp_clips.append(clip_path)
325             elif last_exc is not None:
326                 stderr_msg = last_exc.stderr.decode(errors='replace').strip()
327                 logger.error(f"{segment_label}: FFmpeg trim failed.")
328                 print(f"  Command: {' '.join(cmd)}")
329                 print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
330                 skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
331                 continue
332             else:
333                 logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
334                 skipped_segments.append((idx, start, end, "empty output clip"))
335                 continue
336
337             # Report summary of skipped segments
338             if skipped_segments:
339                 logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
340                 for seg_idx, seg_start, seg_end, reason in skipped_segments:
341                     logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
342                 logger.info("")
343
344             if not temp_clips:
345                 raise RuntimeError(
346                     f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
347                     f"All {len(segments)} segments were skipped. "
348                     f"Check the errors above for details. "
349                     f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
350                 )
351
352             logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
353
354             # Create a list file for FFmpeg concat demuxer
355             list_file_path = os.path.join(tmp_dir, "clips.txt")
356             with open(list_file_path, "w") as f:
357                 for clip in temp_clips:
358                     # Escape single quotes in path
359                     safe_path = clip.replace("'", "'\\'")

```

```

360         f.write(f"file '{safe_path}'\n")
361
362     logger.info("Stitching clips together...")
363     # Stitch them together without re-encoding (since they were encoded with
identical settings)
364     concat_cmd = [
365         "ffmpeg", "-y",
366         "-f", "concat",
367         "-safe", "0",
368         "-i", list_file_path,
369         "-c", "copy",
370         output_path
371     ]
372
373     try:
374         subprocess.run(concat_cmd, capture_output=True, check=True)
375
376         # Final verification
377         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
378             raise RuntimeError(
379                 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
380                 f"but output file is missing or empty: {output_path}"
381             )
382
383         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
384         logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
385         return output_path
386     except subprocess.CalledProcessError as e:
387         stderr_msg = e.stderr.decode(errors='replace').strip()
388         raise RuntimeError(
389             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
390             f"  Command: {' '.join(concat_cmd)}\n"
391             f"  FFmpeg stderr: {stderr_msg[-500:]}"
392         ) from e
393

```

7. user (2026-06-18T01:37:14.580Z)

```

1     """Build an observability report for one indexer run.
2
3     Pure-ish harvest: given the validation results/context, the persisted segments,
4     and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5     *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6     so this module only records facts faithfully ? it does not decide warnings.
7
8     Imported only after `backend.reporting` confirms obsreport is installed, so it may
9     import obsreport freely.
10    """
11    from __future__ import annotations
12
13    import os
14    from pathlib import Path
15    from typing import Any, Dict, List, Optional
16
17    try:
18        import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime
19    except ImportError:
20        obsreport = None # type: ignore[assignment]
21
22    _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
23    _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
24
25
26    def load_spec() -> "obsreport.ReportSpec":

```

```

27         return obsreport.load_spec(_SPEC_PATH)
28
29
30     # --- media metadata -----
31     def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
32         """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
33         distinguishes a real probe from the absence of one (the fallback footgun)."""
34         meta: Dict[str, Any] = {"fps_source": "unknown"}
35         if video_path and os.path.exists(video_path):
36             try:
37                 meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
38             except OSError:
39                 pass
40
41         ffmata = (val_context or {}).get("ffprobe_meta") or {}
42         streams = ffmata.get("streams") or []
43         fmt = ffmata.get("format") or {}
44         if streams:
45             s = streams[0]
46             w, h = s.get("width"), s.get("height")
47             meta["width"], meta["height"] = w, h
48             if w and h:
49                 meta["resolution"] = f"{w}x{h}"
50             fps_str = s.get("r_frame_rate", "0/1")
51             try:
52                 num, den = (int(x) for x in fps_str.split("/"))
53                 if den:
54                     meta["fps"] = round(num / den, 2)
55                     meta["fps_source"] = "probed"
56             except (ValueError, ZeroDivisionError):
57                 pass
58             if s.get("codec_name"):
59                 meta["container"] = fmt.get("format_name") or s.get("codec_name")
60         if fmt.get("duration"):
61             try:
62                 meta["duration_s"] = round(float(fmt["duration"]), 1)
63             except (ValueError, TypeError):
64                 pass
65         if fmt.get("format_name"):
66             meta["container"] = fmt.get("format_name")
67
68         # Audio readiness signal (for Input Quality UX extension per review item 2).
69         # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
70         # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
71         # We only record presence here (cheap, from ffprobe already in context); the actual
72         # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
73         # tool for now.
74         audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
75         meta["audio_present"] = len(audio_streams) > 0
76         return meta
77
78
79     # --- stage builders -----
80     def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
81         with rec.stage("validation") as st:
82             total = len(val_results)
83             passed = sum(1 for r in val_results if getattr(r, "passed", False))
84             st.add_metric("checks_passed", passed)
85             st.add_metric("checks_total", total)
86             failures = [r for r in val_results if not getattr(r, "passed", True)]
87             for r in failures:
88                 st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,
'message', '')}")

```

```

89         st.status = "error" if failures else ("ok" if total else "not_run")
90
91
92     def _segmentation_stage(
93         rec: "obsreport.RunRecorder",
94         play_segments: List[Dict[str, Any]],
95         candidates: List[Dict[str, Any]],
96         segmenter_requested: Optional[str],
97         segmenter_actual: Optional[str],
98         config_default: Optional[str],
99         was_reingest: bool,
100        ran: bool,
101    ) -> None:
102        with rec.stage("segmentation") as st:
103            if not ran:
104                st.status = "not_run"
105            seg_count = len(play_segments)
106            total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
107            detector_tag = None
108            if play_segments:
109                detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
110            st.add_metric("candidate_windows", len(candidates))
111            st.add_metric("segment_count", seg_count, audience="both")
112            st.add_metric("total_play_s", total_play, unit="s")
113            # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
114            # pick it, and it differs from config's default (the genuine surprise case).
115            # An explicit --segmenter override or a halted run is not a divergence.
116            matches_default = True
117            if ran and segmenter_actual and config_default:
118                matches_default = (segmenter_actual == config_default)
119            st.details.update({
120                "segmenter_requested": segmenter_requested,
121                "segmenter_actual": segmenter_actual,
122                "config_default": config_default,
123                "segmenter_explicitly_requested": segmenter_requested is not None,
124                "matches_config_default": matches_default,
125                "was_reingest": was_reingest,
126                "detector": detector_tag,
127            })
128            # Surface metadata trustworthiness at the data level (not just in a warning):
129            # the motion baseline fabricates per-segment metadata; everything else measures
130            it.
131            if ran and segmenter_actual:
132                st.details["metadata_source"] = (
133                    "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
134                    else "measured (detector/AI)"
135                )
136
137    def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
138                play_segments: List[Dict[str, Any]], ran: bool,
139                run_signals: Optional[Dict[str, Any]] = None) -> None:
140        """Only meaningful for AI-based segmenters. Records whether the provider key was
141        present ? the signal the silent_provider_noop rule keys on ? plus any in-process
142        run signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
143        ai_based = segmenter_actual in _AI_SEGMENTERS
144        provider = (config.get("ai_provider") or "gemini")
145        key_env = f"{provider.upper()}_API_KEY"
146        with rec.stage("ai_handoff") as st:
147            if not ran or not ai_based:
148                st.status = "not_run" if not ran else "skipped"
149            st.details.update({
150                "ai_based": ai_based,
151                "provider": provider if ai_based else None,

```

```

152         "model": config.get("ai_model") if ai_based else None,
153         "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
154         # Chunks the provider refused as over its upload cap (no analysis ran for
155         # those time ranges). Detail name couples to the oversize_clip_skipped rule.
156         "oversize_clips_skipped": int((run_signals or
157 {}).get("oversize_clips_skipped", 0)) if ai_based else None,
158     })
159     if ai_based:
160         st.add_metric("confirmed_segments", len(play_segments))
161
162 def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
163               video_meta: Dict[str, Any]) -> None:
164     total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
165     coverage = None
166     dur = video_meta.get("duration_s")
167     if dur:
168         coverage = round(100.0 * total / dur, 1)
169     notes = []
170     if not play_segments:
171         notes.append("No highlights were produced for this video.")
172     rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
173                       coverage_pct=coverage, notes=notes)
174
175 # --- top-level -----
176 def record_run(
177     *,
178     video_id: str,
179     video_path: Optional[str],
180     config: Dict[str, Any],
181     val_results: List[Any],
182     val_context: Optional[Dict[str, Any]],
183     play_segments: List[Dict[str, Any]],
184     candidates: List[Dict[str, Any]],
185     segmenter_requested: Optional[str] = None,
186     segmenter_actual: Optional[str] = None,
187     was_reingest: bool = False,
188     segmentation_ran: bool = True,
189     tool_version: Optional[str] = None,
190     spec: Optional["obsreport.ReportSpec"] = None,
191     run_signals: Optional[Dict[str, Any]] = None,
192     **recorder_kwargs: Any,
193 ) -> "obsreport.RunRecorder":
194     """Assemble a finalized recorder for an indexer run (does NOT write)."""
195     rec = obsreport.RunRecorder(
196         "badminton-highlight-indexer",
197         tool_version=tool_version,
198         video_id=video_id,
199         video_id_kind="file_md5",
200         video_path=video_path,
201         spec=spec if spec is not None else load_spec(),
202         **recorder_kwargs,
203     )
204     vm = video_meta_from_context(val_context, video_path)
205     # If we never probed an fps but segmentation still ran, the segmenter must have
206     # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
207     # AND show the assumed numeric value so the report reads "30 (fallback)" not "?"
208     (fallback)".
209     if vm.get("fps_source") == "unknown" and segmentation_ran:
210         vm["fps_source"] = "fallback"
211         if vm.get("fps") is None:
212             vm["fps"] = 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0
213 / 1280)
214     rec.set_video_meta(**vm)

```

```

214
215     _validation_stage(rec, val_results)
216     config_default = (config.get("indexing") or {}).get("default_segmenter")
217     _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
218                         segmenter_actual, config_default, was_reingest,
segmentation_ran)
219     _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran,
run_signals)
220     _highlights(rec, play_segments, vm)
221
222     rec.finalize()
223     return rec
224

```

8. assistant (2026-06-18T01:37:18.089Z)

The compiler looks solid. Let me look at under-examined modules: the database migration ladder, the export tools, and serialization paths. Let me also check files that look rarely-touched.

9. user (2026-06-18T01:37:19.011Z)

```

1     """Minimal Value Layer exports (first slice of review item 3).
2
3     Provides timeline exports (EDL, JSON, CSV) and basic stats over the
4     play_segments returned by the indexer (or query_play_segments). These are
5     the "time-saved editing is table stakes" deliverables and also give a
6     machine-readable foundation for later human correction / golden feeding
7     and clip galleries.
8
9     Kept deliberately small and pure (no DB, no side effects) so they are
10    easy to call from CLI, notebooks, or a future API endpoint.
11    """
12
13    from __future__ import annotations
14
15    import json
16    from typing import Any, Dict, Iterable, List, Optional
17
18
19    def _to_list(segments: Iterable[Dict[str, Any]]) -> List[Dict[str, Any]]:
20        return list(segments) if not isinstance(segments, list) else segments
21
22
23    def _segment_metadata(s: Dict[str, Any]) -> Dict[str, Any]:
24        """A play segment's metadata as a dict, whether stored inline (segmenter output)
25        or as a JSON string (DB round-trip via query_play_segments)."""
26        md = s.get("metadata") or {}
27        if isinstance(md, str):
28            try:
29                md = json.loads(md)
30            except Exception:
31                md = {}
32        return md if isinstance(md, dict) else {}
33
34
35    def _fmt_hms(seconds: float) -> str:
36        """HH:MM:SS, matching the segmenters' _fmt_ts so the summary block lines up with
37        the per-rally [rally] log lines and the existing timestamp style."""
38        total = max(0, int(float(seconds)))
39        h, rem = divmod(total, 3600)
40        m, s = divmod(rem, 60)
41        return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44    def format_rally_summary(

```

```

45     segments: Iterable[Dict[str, Any]], title: str = "Rally summary"
46 ) -> str:
47     """Render a compact, scannable end-of-run block for manual QA against the rally CSV.
48
49     One header line (total rallies + total highlight duration in HH:MM:SS) followed by a
50     per-rally table ``# start end dur(s) shot_count`` in time order. Pure (no DB, no
51 I/O):
52
53     the same play_segment dicts that feed ``segments_to_csv`` / the rally CSV go in and
54     text comes out, so the log and CSV line up row-for-row for eyeball verification.
55
56     A segment with no ``shot_count`` in its metadata renders the count as ``-``.
57     """
58     segs = _to_list(segments)
59     total_dur = sum(
60         float(s.get("duration", float(s.get("end_time", 0)) - float(s.get("start_time",
61 0))))
62         for s in segs
63     )
64     header = (
65         f"=== {title}: {len(segs)} rallies, "
66         f"{_fmt_hms(total_dur)} ({total_dur:.1f}s) total play ==="
67     )
68     if not segs:
69         return header + "\n (no rallies detected)"
70
71     lines = [header, f"{'#':>3} {'start':<8} {'end':<8} {'dur(s)':>7} {'shots':>6}"]
72     for i, s in enumerate(segs, start=1):
73         start = float(s.get("start_time", 0.0))
74         end = float(s.get("end_time", start))
75         dur = float(s.get("duration", end - start))
76         shot = _segment_metadata(s).get("shot_count")
77         shot_str = "-" if shot is None else str(shot)
78         lines.append(
79             f"{i:>3} {_fmt_hms(start):<8} {_fmt_hms(end):<8} {dur:>7.1f}
80 {shot_str:>6}"
81         )
82     return "\n".join(lines)
83
84 def basic_stats(segments: Iterable[Dict[str, Any]]) -> Dict[str, Any]:
85     """Return simple aggregate stats for a list of play segments.
86
87     Keys: count, total_duration_s, avg_duration_s, min_duration_s, max_duration_s.
88     All durations are in seconds (float).
89     """
90     segs = _to_list(segments)
91     if not segs:
92         return {
93             "count": 0,
94             "total_duration_s": 0.0,
95             "avg_duration_s": 0.0,
96             "min_duration_s": 0.0,
97             "max_duration_s": 0.0,
98         }
99     durs = [float(s.get("duration", (s.get("end_time", 0) - s.get("start_time", 0)))]
100 for s in segs]
101     total = sum(durs)
102     return {
103         "count": len(segs),
104         "total_duration_s": round(total, 2),
105         "avg_duration_s": round(total / len(durs), 2),
106         "min_duration_s": round(min(durs), 2),
107         "max_duration_s": round(max(durs), 2),
108     }

```

```

106
107 def segments_to_json(
108     segments: Iterable[Dict[str, Any]],
109     video_id: str,
110     video_path: Optional[str] = None,
111     meta: Optional[Dict[str, Any]] = None,
112 ) -> Dict[str, Any]:
113     """Return a JSON-serializable timeline dict.
114
115     Shape (stable):
116     {
117         "video_id": "...",
118         "video_path": "...",
119         "meta": {...},
120         "segments": [ {"id":, "start_time":, "end_time":, "duration":, "metadata": {...}},
121     ],
122         "stats": basic_stats(...)
123     }
124     """
125     segs = _to_list(segments)
126     return {
127         "video_id": video_id,
128         "video_path": video_path,
129         "meta": meta or {},
130         "segments": segs,
131         "stats": basic_stats(segs),
132     }
133
134 def segments_to_csv(segments: Iterable[Dict[str, Any]], include_header: bool = True) ->
str:
135     """Return a CSV string (start,end,duration,shot_count,detector,...) .
136
137     Compatible with many editors and the existing golden/slicer schemas
138     (extra columns are harmless).
139     """
140     segs = _to_list(segments)
141     lines: List[str] = []
142     if include_header:
143
144 lines.append("start_time,end_time,duration,shot_count,shot_count_confidence,detector,id")
145         for s in segs:
146             md = s.get("metadata") or {}
147             shot = md.get("shot_count", "")
148             conf = md.get("shot_count_confidence", "")
149             det = md.get("detector", "")
150             lines.append(
151                 f"{s.get('start_time',0)},{s.get('end_time',0)},{s.get('duration',0)},"
152                 f"{shot},{conf},{det},{s.get('id','')}"
153             )
154         return "\n".join(lines) + "\n"
155
156 def segments_to_edl(segments: Iterable[Dict[str, Any]], title: str = "Highlights") ->
str:
157     """Return a simple EDL (CMX 3600-ish) for the segments.
158
159     Many NLEs will ingest this for a rough cut. Uses record timecode 00:00:00:00
160     as origin; clips are placed sequentially with the original source in/out.
161     """
162     segs = _to_list(segments)
163     lines: List[str] = [f"TITLE: {title}", "FCM: NON-DROP FRAME"]
164     for i, s in enumerate(segs, 1):
165         start = float(s.get("start_time", 0.0))
166         end = float(s.get("end_time", start))

```



```

167         # EDL is HH:MM:SS:FF (assume 30fps for the timecode display; real edit rate
168         # is irrelevant for a rough cut ? the numbers are just labels here).
169         def _tc(t: float) -> str:
170             h = int(t // 3600)
171             m = int((t % 3600) // 60)
172             s_ = int(t % 60)
173             f = int((t - int(t)) * 30) # 30 fps display
174             return f"{h:02d}:{m:02d}:{s_:02d}:{f:02d}"
175         rec_in = _tc((i - 1) * 0.0) # place back-to-back in the EDL timeline
176         rec_out = _tc((i - 1) * 0.0 + (end - start))
177         src_in = _tc(start)
178         src_out = _tc(end)
179         lines.append(f"{i:03d}  AX  V      C          {rec_in} {rec_out} {src_in}
{src_out}")
180         lines.append(f"* FROM CLIP NAME: {title} {i}")
181         lines.append("")
182         return "\n".join(lines)
183

```

10. user (2026-06-18T01:37:19.487Z)

```

1     """Slice individual rally clips out of a (long) annotated video.
2
3     Use cases:
4     - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5       boundaries against the labels.
6     - **Per-video calibration/training** ? grab a handful of labeled rallies as the
7       supervision the segmenter is tuned/adapted on for that specific video.
8
9     Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
10    end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
11        <out_dir>/<video_stem>__rally_<n>.mp4
12
13    Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
14    shows the same lead-in/out the model is trained and evaluated against. The window
15    math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in
16    `slice_rallies`.
17    """
18    from __future__ import annotations
19
20    import argparse
21    import csv
22    import os
23    import subprocess
24    from dataclasses import dataclass
25    from typing import Dict, List, Optional, Tuple
26
27    from backend.eval.annotations import parse_timestamp
28
29    # Match VideoCompiler defaults so sliced clips frame a rally the same way.
30    BEGIN_PADDING = 0.5
31    END_PADDING = 1.0
32
33
34    @dataclass
35    class ClipWindow:
36        rally: str          # kept as a string to allow inserted ids like "7.5"
37        start: float        # padded, clamped
38        end: float          # padded, clamped
39
40
41    def _read_time(row: Dict[str, str], decimal_key: str, mmss_key: str) -> Optional[float]:
42        """Prefer the decimal-seconds column; fall back to mm:ss. None if unusable."""
43        for key in (decimal_key, mmss_key):
44            val = (row.get(key) or "").strip()

```

```

45         if val:
46             try:
47                 return parse_timestamp(val)
48             except ValueError:
49                 continue
50     return None
51
52
53 def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float, float]]:
54     """Map rally_number -> (start, end) seconds, skipping rows with no usable times.
55
56     Accepts the golden schema (start_time/end_time decimal, optional start_mmss/
57     end_mmss). Rally numbers are kept as strings so inserted ids ("7.5") survive.
58     """
59     labels: Dict[str, Tuple[float, float]] = {}
60     with open(csv_path, newline="", encoding="utf-8") as f:
61         reader = csv.DictReader(f)
62         reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
63 [])]
64         for row in reader:
65             row = {(k or "").strip().lower(): v for k, v in row.items()}
66             rally = (row.get("rally_number") or "").strip()
67             if not rally:
68                 continue
69             start = _read_time(row, "start_time", "start_mmss")
70             end = _read_time(row, "end_time", "end_mmss")
71             if start is None or end is None or end <= start:
72                 continue # missing/degenerate (e.g. a not-yet-filled MISSED row)
73             labels[rally] = (start, end)
74     return labels
75
76 def compute_clip_windows(labels: Dict[str, Tuple[float, float]],
77 rally_numbers: List[str],
78 begin_pad: float = BEGIN_PADDING,
79 end_pad: float = END_PADDING,
80 video_duration: Optional[float] = None) -> List[ClipWindow]:
81     """Apply padding and clamp to the video, for the requested rallies.
82
83     rally_numbers may be ["all"] to take every labeled rally (in time order).
84     Unknown rally ids are skipped (caller can diff to warn).
85     """
86     if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
87         wanted = sorted(labels, key=lambda r: labels[r][0])
88     else:
89         wanted = [r for r in rally_numbers if r in labels]
90
91     windows: List[ClipWindow] = []
92     for rally in wanted:
93         start, end = labels[rally]
94         ps = max(0.0, start - begin_pad)
95         pe = end + end_pad
96         if video_duration is not None and video_duration > 0:
97             if ps >= video_duration:
98                 continue # entirely past the end
99             pe = min(pe, video_duration)
100         if pe <= ps:
101             continue
102         windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
103     return windows
104
105
106 def _probe_duration(video_path: str) -> Optional[float]:
107     try:
108         out = subprocess.run(

```

```

109         ["ffprobe", "-v", "error", "-show_entries", "format=duration",
110          "-of", "default=noprint_wrappers=1:nokey=1", video_path],
111         capture_output=True, text=True, check=True,
112     ).stdout.strip()
113     return float(out)
114 except (subprocess.SubprocessError, OSError, ValueError):
115     return None
116
117
118 def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
119                  out_dir: str, begin_pad: float = BEGIN_PADDING,
120                  end_pad: float = END_PADDING) -> List[str]:
121     """Write one padded clip per requested rally; returns the clip paths written."""
122     if not os.path.exists(video_path):
123         raise FileNotFoundError(f"video not found: {video_path}")
124     labels = load_rally_labels(labels_csv)
125     if not labels:
126         raise ValueError(f"no usable rally rows in {labels_csv}")
127     os.makedirs(out_dir, exist_ok=True)
128
129     duration = _probe_duration(video_path)
130     windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)
131     if not windows:
132         raise ValueError("no rallies matched the request (check --rallies vs the CSV)")
133
134     stem = os.path.splitext(os.path.basename(video_path))[0]
135     written: List[str] = []
136     for w in windows:
137         safe_rally = w.rally.replace(".", "_")
138         out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
139         cmd = [
140             "ffmpeg", "-y",
141             "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
142             "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
143             "-c:a", "aac", "-b:a", "128k",
144             out_path,
145         ]
146         try:
147             subprocess.run(cmd, capture_output=True, check=True)
148         except subprocess.CalledProcessError as e:
149             err = e.stderr.decode(errors="replace").strip()[-300:]
150             print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")
151             continue
152         if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
153             written.append(out_path)
154             print(f"[slicer] rally {w.rally} -> {out_path}")
155     [{w.start:.3f}-{w.end:.3f}s]}
156     return written
157
158 def main() -> None:
159     p = argparse.ArgumentParser(description="Slice rally clips from an annotated
160     video.")
161     p.add_argument("--video", required=True, help="Source video file")
162     p.add_argument("--labels", required=True, help="Labels CSV (golden rally format)")
163     p.add_argument("--rallies", required=True,
164                     help="Comma-separated rally numbers (e.g. 7,7.5,16) or 'all'")
165     p.add_argument("--out-dir", default="output/rally_clips", help="Output directory")
166     p.add_argument("--begin-pad", type=float, default=BEGIN_PADDING)
167     p.add_argument("--end-pad", type=float, default=END_PADDING)
168     args = p.parse_args()
169
170     rally_numbers = [r.strip() for r in args.rallies.split(",") if r.strip()]
171     written = slice_rallies(args.video, args.labels, rally_numbers,
172                             args.out_dir, args.begin_pad, args.end_pad)

```

```

172         print(f"\nWrote {len(written)} clip(s) to {args.out_dir}")
173
174
175     if __name__ == "__main__":
176         main()
177

```

11. assistant (2026-06-18T01:37:22.079Z)

These look clean. Let me look at the database migration ladder and the results module, which are core but rarely re-examined for edge cases.

12. user (2026-06-18T01:37:23.415Z)

```

1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43             return conn
44
45         def _init_db(self):
46             """Initializes tables for videos, play_segments, and events."""
47             with self._connect() as conn:
48                 cursor = conn.cursor()
49
50                 # Videos Table
51                 cursor.execute("""

```

```

51         CREATE TABLE IF NOT EXISTS videos (
52             id TEXT PRIMARY KEY,
53             filepath TEXT NOT NULL,
54             processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
55             status TEXT NOT NULL,
56             validation_results TEXT
57         )
58     """)
59
60     # Play Segments Table (Extensible metadata JSON)
61     cursor.execute("""
62         CREATE TABLE IF NOT EXISTS play_segments (
63             id INTEGER PRIMARY KEY AUTOINCREMENT,
64             video_id TEXT NOT NULL,
65             start_time REAL NOT NULL,
66             end_time REAL NOT NULL,
67             duration REAL NOT NULL,
68             server TEXT,
69             receiver TEXT,
70             metadata TEXT,
71             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
72         )
73     """)
74
75     # Events Table
76     cursor.execute("""
77         CREATE TABLE IF NOT EXISTS events (
78             id INTEGER PRIMARY KEY AUTOINCREMENT,
79             segment_id INTEGER NOT NULL,
80             timestamp REAL NOT NULL,
81             type TEXT NOT NULL,
82             player TEXT,
83             details TEXT,
84             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
85             CASCADE
86         )
87     """)
88
89     # Versioned migrations live here. PRAGMA user_version is the schema
90     # contract ? bump it for every change and add an ordered `if version < N`
91     # block below. Tests assert the expected version, so accidental drift fails
92     CI.
93     version = cursor.execute("PRAGMA user_version").fetchone()[0]
94
95     if version < 1:
96         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
97         # Common fields are columns; detector-specific metrics go in `stats`
98         JSON,
99         # so a new segmenter reuses this table by setting its own
100         `detector`/`stats`.
101         cursor.execute("""
102             CREATE TABLE IF NOT EXISTS candidate_segments (
103                 id INTEGER PRIMARY KEY AUTOINCREMENT,
104                 video_id TEXT NOT NULL,
105                 detector TEXT NOT NULL,
106                 chunk_index INTEGER,
107                 start_time REAL NOT NULL,
108                 end_time REAL NOT NULL,
109                 duration REAL NOT NULL,
110                 confidence REAL,
111                 stats TEXT,
112                 detection_params TEXT,
113                 confirmed_segment_id INTEGER,
114                 human_label TEXT,
115                 notes TEXT,

```

```

112         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
114         FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
115     )
116     """)
117     cursor.execute("""
118         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
119         ON candidate_segments(video_id, detector)
120     """)
121     cursor.execute("PRAGMA user_version = 1")
122
123     if version < 2:
124         # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
125         # /api/process request. `status` is the EXECUTION state of the job
126         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
127         # that failed validation) lives inside `result` JSON as
result["status"].
128         # `result` is the full sync-era response dict (incl. report paths), so
the
129         # observability report is the job's artifact, per the plan.
130         cursor.execute("""
131             CREATE TABLE IF NOT EXISTS jobs (
132                 id TEXT PRIMARY KEY,
133                 video_path TEXT NOT NULL,
134                 video_id TEXT,
135                 segmenter TEXT,
136                 player_pool TEXT,
137                 max_frames INTEGER,
138                 status TEXT NOT NULL DEFAULT 'queued',
139                 progress TEXT,
140                 error TEXT,
141                 result TEXT,
142                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
143                 started_at TIMESTAMP,
144                 finished_at TIMESTAMP
145             )
146             """)
147         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
148         cursor.execute("PRAGMA user_version = 2")
149
150     if version < 3:
151         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
152         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
153         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154         # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155         # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156         cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
157         cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
158         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
159         cursor.execute("PRAGMA user_version = 3")
160
161     if version < 4:
162         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
163         # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
164         # path can scope rows to an owner. **NULL = local install = byte-

```

```

identical to
165 # today** ? every existing row stays NULL and every existing query
(which never
166 # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
167 # Additive ONLY: no served-behavior change rides on this slice.
168 cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
169 cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
170 # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
171 # of index churn while making the eventual per-user lookups cheap.
172 cursor.execute(
173     "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
174     "WHERE user_id IS NOT NULL")
175 cursor.execute(
176     "CREATE INDEX IF NOT EXISTS idx_candidates_user "
177     "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
178 cursor.execute("PRAGMA user_version = 4")
179
180 if version < 5:
181     # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
182     # version): the resolved per-user `fusion_config` overlay + an INLINE
183     # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
184     # and provenance that earned it. `is_active` pins the one row the
185     # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
186     # only persists/loads it ? NOTHING reads it on the served path yet.
187     #
188     # user_id            ? owner of the model (NULL allowed = the local
189     # version            ? monotonic per-user model version
190     # baseline_version   ? the shared baseline this was fit against
191     # feature_schema_hash ? fingerprint of the feature set; MINOR-vs-
192     # fusion_config      ? JSON per-user FusionConfig overlay (cue_flags
193     # classifier_json    ? JSON LogisticRegression.to_dict() (None =
194     # promotion_evidence ? JSON per_user_gate PromotionDecision (why it
195     # n_videos_at_fit    ? held-out-user corpus size when fit (min_n
196     # is_active          ? the one resolved row for this user (partial-
197     unique below)
198     cursor.execute("""
199         CREATE TABLE IF NOT EXISTS user_models (
200             id INTEGER PRIMARY KEY AUTOINCREMENT,
201             user_id TEXT,
202             version INTEGER NOT NULL DEFAULT 1,
203             baseline_version TEXT,
204             feature_schema_hash TEXT,
205             fusion_config TEXT,
206             classifier_json TEXT,
207             promotion_evidence TEXT,
208             n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
209             is_active INTEGER NOT NULL DEFAULT 0,
210             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
211         )
212         """)
213     # One model version per user (NULLs compare distinct in SQLite, so the
214     local

```

```

213             # install can still carry multiple versioned rows; the active-row guard
below
214             # is what enforces a single served model).
215             cursor.execute("""
216                 CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
217                 ON user_models(user_id, version)
218             """)
219             # At most ONE active model per user: a partial-unique index over the
active rows.
220             # (SQLite treats NULL user_id rows as distinct, so the local install's
single
221             # active row is still guarded ? there is one local user.)
222             cursor.execute("""
223                 CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
224                 ON user_models(user_id) WHERE is_active = 1
225             """)
226             cursor.execute("PRAGMA user_version = 5")
227             # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
228
229             conn.commit()
230
231             def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
232                 """Register or update a video row WITHOUT touching its child segments.
233
234                 Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
235                 with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
236                 That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
237                 before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
238                 row in place; clearing segments is now an explicit, deliberate operation
239                 (clear_video_data / prune_segments).
240                 """
241                 try:
242                     with self._connect() as conn:
243                         conn.cursor().execute(
244                             "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
245                             "ON CONFLICT(id) DO UPDATE SET "
246                             "filepath=excluded.filepath, status=excluded.status, "
247                             "validation_results=excluded.validation_results",
248                             (video_id, filepath, status, json.dumps(validation_results))
249                         )
250                         conn.commit()
251                     return True
252                 except Exception as e:
253                     logger.error(f"Failed to add video: {e}")
254                     return False
255
256             def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
257                 """Return (max play_segment id, max candidate_segment id) for a video (0 if
none).
258
259                 Captured before a (re)segmentation so the run's new rows (id > watermark) can be
260                 told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
261                 """
262                 with self._connect() as conn:
263                     cur = conn.cursor()
264                     p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
265                                     (video_id,)).fetchone()[0]
266                     c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE

```



```

video_id = "?",
267             (video_id,)).fetchone()[0]
268         return int(p), int(c)
269
270     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
271                       play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
272                       cand_after: Optional[int] = None) -> None:
273         """Delete play/candidate segments by id range (events cascade from
play_segments).
274
275         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
276         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
rows
277         and keep the OLD ones when the run failed/produced nothing (`*_after`).
278         """
279         with self._connect() as conn:
280             cur = conn.cursor()
281             if play_upto is not None:
282                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
283             if play_after is not None:
284                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
285             if cand_upto is not None:
286                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
287             if cand_after is not None:
288                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
289             conn.commit()
290
291     def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
292         try:
293             with self._connect() as conn:
294                 cursor = conn.cursor()
295                 cursor.execute(
296                     "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
297                     (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
298                 )
299                 conn.commit()
300                 return cursor.lastrowid
301         except Exception as e:
302             logger.error(f"Failed to add play segment: {e}")
303             return None
304
305     def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
306         try:
307             with self._connect() as conn:
308                 conn.cursor().execute(
309                     "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
310                     (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
311                 )
312                 conn.commit()
313                 return True
314         except Exception as e:
315             logger.error(f"Failed to add event: {e}")
316             return False

```

```

317
318     def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
319                               chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
320                               stats: Optional[Dict[str, Any]] = None,
321                               detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
322         """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
323         detector-specific dicts stored as JSON. Returns the new candidate id, or
324         None."""
325         try:
326             with self._connect() as conn:
327                 cursor = conn.cursor()
328                 cursor.execute(
329                     """INSERT INTO candidate_segments
330                     (video_id, detector, chunk_index, start_time, end_time, duration,
331                     confidence, stats, detection_params)
332                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
333                     (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
334                     confidence, json.dumps(stats or {}), json.dumps(detection_params or
{})))
335                 conn.commit()
336                 return cursor.lastrowid
337         except Exception as e:
338             logger.error(f"Failed to add candidate segment: {e}")
339             return None
340
341     def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
342         """Record that a candidate detection was confirmed as a given play_segment."""
343         try:
344             with self._connect() as conn:
345                 conn.cursor().execute(
346                     "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
347                     (segment_id, candidate_id)
348                 )
349                 conn.commit()
350             return True
351         except Exception as e:
352             logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
353             return False
354
355     def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
only_unlabeled: bool = False) -> List[Dict[str, Any]]:
356         """Fetch candidate detections for the annotation / fine-tuning workflow.
357         `stats` and `detection_params` are deserialized from JSON."""
358         query = "SELECT * FROM candidate_segments WHERE video_id = ?"
359         params: List[Any] = [video_id]
360
361         if detector:
362             query += " AND detector = ?"
363             params.append(detector)
364         if only_unlabeled:
365             query += " AND human_label IS NULL"
366
367         query += " ORDER BY start_time"
368
369         candidates = []
370         with self._connect() as conn:
371             rows = conn.cursor().execute(query, params).fetchall()
372             for row in rows:

```

```

374         d = dict(row)
375         d["stats"] = json.loads(d["stats"] or "{}")
376         d["detection_params"] = json.loads(d["detection_params"] or "{}")
377         candidates.append(d)
378     return candidates
379
380     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
381         with self._connect() as conn:
382             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
383 (video_id,)).fetchone()
384             if row:
385                 d = dict(row)
386                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
387                 return d
388             return None
389
390     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
391 List[Dict[str, Any]]:
392         """
393         Dynamically queries play segments based on filters:
394         - duration_min: float
395         - server: string
396         - receiver: string
397         - event_type: string (e.g. smash, foul)
398         """
399         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
400         params = [video_id]
401
402         if filters.get("duration_min"):
403             query += " AND r.duration >= ?"
404             params.append(filters["duration_min"])
405
406         if filters.get("server"):
407             query += " AND r.server = ?"
408             params.append(filters["server"])
409
410         if filters.get("receiver"):
411             query += " AND r.receiver = ?"
412             params.append(filters["receiver"])
413
414         if filters.get("event_type"):
415             # Check if there is an event of a specific type inside this segment
416             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
417 e.type = ?)"
418             params.append(filters["event_type"])
419
420         segments_list = []
421         with self._connect() as conn:
422             cursor = conn.cursor()
423             rows = cursor.execute(query, params).fetchall()
424
425             for row in rows:
426                 r_dict = dict(row)
427                 r_id = r_dict["id"]
428                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
429
430                 # Fetch associated events
431                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
432 (r_id,)).fetchall()
433                 r_dict["events"] = []
434                 for er in event_rows:
435                     e_dict = dict(er)
436                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
437                     r_dict["events"].append(e_dict)

```

```

435         segments_list.append(r_dict)
436
437     return segments_list
438
439     def clear_video_data(self, video_id: str):
440         with self._connect() as conn:
441             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
442             conn.commit()
443
444     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
445     # Writers follow the repo convention (swallow + logger.error, return bool); the
446     # worker logs loudly on a failed transition so a job can't silently wedge.
447
448     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
449                   player_pool: Optional[List[str]] = None,
450                   max_frames: Optional[int] = None,
451                   policy: Optional[Dict[str, Any]] = None) -> bool:
452         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
453         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
454         try:
455             with self._connect() as conn:
456                 conn.cursor().execute(
457                     "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
458                     "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
459                     (job_id, video_path, segmenter,
460                      json.dumps(player_pool) if player_pool is not None else None,
461                      max_frames, json.dumps(policy) if policy is not None else None)
462                 )
463                 conn.commit()
464             return True
465         except Exception as e:
466             logger.error(f"Failed to create job {job_id}: {e}")
467             return False
468
469     def mark_job_running(self, job_id: str) -> bool:
470         try:
471             with self._connect() as conn:
472                 conn.cursor().execute(
473                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
474                     "progress='starting' WHERE id = ?", (job_id,))
475                 conn.commit()
476             return True
477         except Exception as e:
478             logger.error(f"Failed to mark job {job_id} running: {e}")
479             return False
480
481     def set_job_progress(self, job_id: str, progress: str) -> bool:
482         try:
483             with self._connect() as conn:
484                 conn.cursor().execute(
485                     "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
486                 conn.commit()
487             return True
488         except Exception as e:
489             logger.error(f"Failed to set progress for job {job_id}: {e}")
490             return False
491
492     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
493         try:
494             with self._connect() as conn:
495                 conn.cursor().execute(
496                     "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
497                 conn.commit()

```

```

498         return True
499     except Exception as e:
500         logger.error(f"Failed to set video_id for job {job_id}: {e}")
501         return False
502
503     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
504         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
505         validation) is inside `result` ? job status 'done' means 'the worker
506         finished'."""
507         try:
508             with self._connect() as conn:
509                 conn.cursor().execute(
510                     "UPDATE jobs SET status='done', progress='finished', result=?, "
511                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
512                     (json.dumps(result), job_id))
513                 conn.commit()
514             return True
515         except Exception as e:
516             logger.error(f"Failed to mark job {job_id} done: {e}")
517             return False
518
519     def mark_job_failed(self, job_id: str, error: str) -> bool:
520         """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
521         try:
522             with self._connect() as conn:
523                 conn.cursor().execute(
524                     "UPDATE jobs SET status='failed', error=?, "
525                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
526                 conn.commit()
527             return True
528         except Exception as e:
529             logger.error(f"Failed to mark job {job_id} failed: {e}")
530             return False
531
532     def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
533         """Fetch one job; `result`/`player_pool` JSON deserialized."""
534         with self._connect() as conn:
535             row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
536             (job_id,)).fetchone()
537             if not row:
538                 return None
539             d = dict(row)
540             d["result"] = json.loads(d["result"]) if d["result"] else None
541             d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
542             None
543             d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
544             return d
545
546     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
547     -----
548     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
549         """Attach/replace a job's JSON ExecutionPolicy."""
550         try:
551             with self._connect() as conn:
552                 conn.cursor().execute(
553                     "UPDATE jobs SET policy=? WHERE id = ?",
554                     (json.dumps(policy) if policy is not None else None, job_id))
555                 conn.commit()
556             return True
557         except Exception as e:
558             logger.error(f"Failed to set policy for job {job_id}: {e}")
559             return False
560
561     def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
562     None) -> bool:

```

```

558         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
559         worker. `claim_due_job` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
560         computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
561         try:
562             with self._connect() as conn:
563                 conn.cursor().execute(
564                     "UPDATE jobs SET status='waiting', "
565                     "next_poll_at=datetime('now', ?), "
566                     "progress=COALESCE(?, progress) WHERE id = ?",
567                     (f"{max(0, int(delay_sec))} seconds", progress, job_id))
568                 conn.commit()
569             return True
570         except Exception as e:
571             logger.error(f"Failed to set job {job_id} waiting: {e}")
572             return False
573
574     def seconds_until_next_due(self) -> Optional[float]:
575         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
576         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
577         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
578         try:
579             with self._connect() as conn:
580                 row = conn.cursor().execute(
581                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
582                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
583                     "FROM jobs WHERE status='waiting']").fetchone()
584                 if row is None or row["secs"] is None:
585                     return None
586                 return max(0.0, float(row["secs"]))
587         except Exception as e:
588             logger.error(f"Failed to compute next-due delay: {e}")
589             return None
590
591     def claim_due_job(self) -> Optional[Dict[str, Any]]:
592         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
593         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
594         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
595         first. Returns the claimed job dict, or None if nothing is runnable."""
596         try:
597             with self._connect() as conn:
598                 cur = conn.cursor()
599                 row = cur.execute(
600                     "SELECT id, status FROM jobs WHERE status='queued' "
601                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
602                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
603                 if not row:
604                     return None
605                 cur.execute(
606                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
607                     "WHERE id=? AND status=?", (row["id"], row["status"]))
608                 conn.commit()
609                 if cur.rowcount != 1:
610                     return None
# lost the race to a concurrent

```

```

worker
611         return self.get_job(row["id"])
612     except Exception as e:
613         logger.error(f"Failed to claim a due job: {e}")
614         return None
615
616     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
617         """Crash recovery: any job still queued/running from a previous process is dead
618         (the executor is in-process). Mark failed so clients see a terminal state, not a
619         hang. Called once at startup. Returns the number of jobs swept."""
620         try:
621             with self._connect() as conn:
622                 cur = conn.cursor()
623                 cur.execute(
624                     "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
625                     "WHERE status IN ('queued', 'running')", (reason,))
626                 conn.commit()
627                 return cur.rowcount
628         except Exception as e:
629             logger.error(f"Failed to sweep stale jobs: {e}")
630             return 0
631
632     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
633     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
634     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
635     # serving.py) resolves them, and wiring that bridge into the production decision
636     # seam
637     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
638     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
639     # JSON-serialised dicts; the rest are scalar columns.
640
641     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
642                          baseline_version: Optional[str] = None,
643                          feature_schema_hash: Optional[str] = None,
644                          fusion_config: Optional[Dict[str, Any]] = None,
645                          classifier_json: Optional[Dict[str, Any]] = None,
646                          promotion_evidence: Optional[Dict[str, Any]] = None,
647                          n_videos_at_fit: int = 0,
648                          is_active: bool = False) -> Optional[int]:
649         """Insert or replace ONE per-user model row keyed by `(user_id, version)`".
650
651         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
652         the
653         rowid). When `is_active` is True this row becomes the user's single active
654         model ?
655         any previously-active row for the SAME user is first cleared, so the
656         `idx_user_models_one_active` partial-unique invariant always holds (one active
657         model
658         per user). Returns the row id, or None on failure.
659         """
660         try:
661             with self._connect() as conn:
662                 cur = conn.cursor()
663                 if is_active:
664                     # Demote the current active row first (NULL user_id = local: IS NULL
665                     match).
666                     if user_id is None:
667                         cur.execute(
668                             "UPDATE user_models SET is_active = 0 "
669                             "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
670                             (version,))
671                     else:
672                         cur.execute(
673                             "UPDATE user_models SET is_active = 0 "

```

```

669             "WHERE user_id = ? AND is_active = 1 AND version != ?",
670             (user_id, version))
671         cur.execute(
672             """INSERT INTO user_models
673             (user_id, version, baseline_version, feature_schema_hash,
674             fusion_config, classifier_json, promotion_evidence,
675             n_videos_at_fit, is_active)
676             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
677             ON CONFLICT(user_id, version) DO UPDATE SET
678             baseline_version=excluded.baseline_version,
679             feature_schema_hash=excluded.feature_schema_hash,
680             fusion_config=excluded.fusion_config,
681             classifier_json=excluded.classifier_json,
682             promotion_evidence=excluded.promotion_evidence,
683             n_videos_at_fit=excluded.n_videos_at_fit,
684             is_active=excluded.is_active""",
685             (user_id, version, baseline_version, feature_schema_hash,
686             json.dumps(fusion_config) if fusion_config is not None else None,
687             json.dumps(classifier_json) if classifier_json is not None else
688             None,
689             json.dumps(promotion_evidence) if promotion_evidence is not None
690             else None,
691             n_videos_at_fit, 1 if is_active else 0))
692         conn.commit()
693         rid = cur.execute(
694             "SELECT id FROM user_models WHERE "
695             + ("user_id IS NULL" if user_id is None else "user_id = ?")
696             + " AND version = ?",
697             ((version,) if user_id is None else (user_id, version))).fetchone()
698         return int(rid[0]) if rid else None
699     except Exception as e:
700         logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
701         {e}")
702     return None
703
704 def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
705     """Return the single active per-user model row for ``user_id`` (NULL = local
706     install),
707     with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
708     deserialised,
709     or None if the user has no active model. Does not touch the served pipeline."""
710     with self._connect() as conn:
711         if user_id is None:
712             row = conn.cursor().execute(
713                 "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
714                 1").fetchone()
715         else:
716             row = conn.cursor().execute(
717                 "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
718                 (user_id,)).fetchone()
719         if not row:
720             return None
721         d = dict(row)
722         for col in ("fusion_config", "classifier_json", "promotion_evidence"):
723             d[col] = json.loads(d[col]) if d[col] else None
724         return d

```

13. assistant (2026-06-18T01:37:28.913Z)

The migration ladder has a notable concern: migrations use `ALTER TABLE ADD COLUMN` (v3, v4) wrapped in `if version < N` blocks, but these run inside `\_init\_db` which always executes `CREATE TABLE IF NOT EXISTS` first. That's fine. The `idx\_user\_models\_one\_active` and upsert logic look careful. Let me look at the execution_policy and serving modules, which are newer and self-scheduling logic is error-prone.

14. user (2026-06-18T01:37:30.018Z)

```
1      """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3      Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4      - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5      for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6      so it is abandoned, not joined ? it never blocks the process.
7      - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8      ``max_wait_sec`` deadline).
9
10     **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11     waiting (a fake clock advances instantly).
12
13     **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14     transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15     ``execution.policy`` logger to watch the poll trail.
16     """
17     from __future__ import annotations
18
19     import logging
20     import threading
21     import time as _time
22     from dataclasses import asdict, dataclass
23     from enum import Enum
24     from typing import Callable, Optional, TypeVar
25
26     T = TypeVar("T")
27
28     #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
29     LOG = logging.getLogger("execution.policy")
30
31
32     class WaitMode(str, Enum):
33         """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
34         WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
35         ABORT = "abort" # one attempt; on hang/fail, stop immediately
36         BLOCK = "block" # bounded in-process blocking wait (no
reschedule)
37
38
39     class PolicyTimeout(TimeoutError):
40         """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
41
42
43     @dataclass(frozen=True)
44     class ExecutionPolicy:
45         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46         mode: WaitMode = WaitMode.WAIT_AND_RESUME
47         poll_every_n_sec: float = 10.0
48         op_timeout_sec: float = 120.0 # HARD per-attempt deadline ? the hang-killer
that was missing
49         max_wait_sec: float = 1800.0 # total wall budget for a poll loop
```

```

50         max_retries: int = 5                # retries (=> max_retries+1 attempts) for
failures/hangs
51         backoff_base_sec: float = 3.0        # exponential: backoff_base * 2**attempt
52         on_timeout: str = "reschedule"       # reschedule | partial | fail (consumed by
P2/P3)
53         resumable: bool = True
54
55         def to_dict(self) -> dict:
56             d = asdict(self)
57             d["mode"] = self.mode.value
58             return d
59
60         @classmethod
61         def from_dict(cls, d: dict) -> "ExecutionPolicy":
62             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
63             kw = {k: v for k, v in d.items() if k in known}
64             if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65                 kw["mode"] = WaitMode(kw["mode"])
66             return cls(**kw)
67
68
69         #: The sane default (also what a job gets if it declares no policy).
70         DEFAULT_POLICY = ExecutionPolicy()
71
72
73         def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
74             """Deterministic exponential backoff (no jitter ? reproducible tests)."""
75             return policy.backoff_base_sec * (2 ** attempt)
76
77
78         def run_bounded(fn: Callable[[], T], policy: ExecutionPolicy = DEFAULT_POLICY, *,
79                         label: str = "op", logger: Optional[logging.Logger] = None,
80                         sleep: Callable[[float], None] = _time.sleep) -> T:
81             """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
backoff.
82
83             Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
hung, or re-raises the
84             last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
retries).
85             """
86             log = logger or LOG
87             attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
88             last_exc: Optional[BaseException] = None
89             for attempt in range(attempts):
90                 box: dict = {}
91                 done = threading.Event()
92
93                 # box/done bound as defaults (not closed over) ? the thread is started + joined
within this
94                 # same iteration, but explicit binding keeps each attempt isolated and silences
B023.
95                 def _runner(_box: dict = box, _done: threading.Event = done) -> None:
96                     try:
97                         _box["val"] = fn()
98                     except BaseException as e:                # relay ANY error (incl. timeouts) to the
caller thread
99                         _box["exc"] = e
100                     finally:
101                         _done.set()
102
103                 threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
104                 if done.wait(timeout=policy.op_timeout_sec):
105                     if "exc" in box:
106                         last_exc = box["exc"]

```

```

107         log.info("%s: attempt %d/%d failed: %s", label, attempt + 1, attempts,
last_exc)
108     else:
109         if attempt:
110             log.info("%s: succeeded on attempt %d/%d", label, attempt + 1,
attempts)
111             return box["val"]                # type: ignore[return-value]
112     else:
113         last_exc = PolicyTimeout(f"{label} hung >
op_timeout_sec={policy.op_timeout_sec}s")
114         log.warning("%s: attempt %d/%d HUNG (>%.0fs) ? abandoning", label, attempt +
1, attempts,
115                     policy.op_timeout_sec)
116         if attempt < attempts - 1:
117             wait = _backoff_sec(policy, attempt)
118             log.debug("%s: backing off %.1fs before retry", label, wait)
119             sleep(wait)
120     assert last_exc is not None
121     log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
122     raise last_exc
123
124
125     def poll_until(check: Callable[[], Optional[T]], policy: ExecutionPolicy =
DEFAULT_POLICY, *,
126                  label: str = "op", logger: Optional[logging.Logger] = None,
127                  sleep: Callable[[float], None] = _time.sleep,
128                  monotonic: Callable[[], float] = _time.monotonic) -> T:
129         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
value, or raise
130         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
still in progress.
131
132         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
INFO/WARNING.
133         """
134         log = logger or LOG
135         start = monotonic()
136         polls = 0
137         log.info("%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
138                 label, policy.poll_every_n_sec, policy.max_wait_sec)
139         while True:
140             result = check()
141             if result is not None:
142                 log.info("%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() -
start)
143                 return result
144             elapsed = monotonic() - start
145             if elapsed >= policy.max_wait_sec:
146                 log.warning("%s: still not ready after %.0fs (max_wait) ? giving up", label,
elapsed)
147                 raise PolicyTimeout(f"{label} not ready within
max_wait_sec={policy.max_wait_sec}s")
148             polls += 1
149             log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
150             sleep(policy.poll_every_n_sec)
151

```

15. user (2026-06-18T01:37:30.526Z)

```

1     """Per-user serving bridge ? resolve a user's fusion config + classifier from the v5
store.
2
3     PERSONALIZATION_PLAN.md §2 (Phase 1, the "#1 wall"): a personalized model is a per-user
4     ``Variant`` (cue_flags + ``min_crossings`` / ``min_players`` + an optional
``LogisticRegression``

```

```

5      + threshold). It does NOT require a segmenter rewrite ? it is a per-user
  ``FusionConfig`` overlay
6      resolved UPSTREAM of detection, plus a classifier the decision seam re-scores persisted
features
7      with. This module is that resolution step, and **only** that step.
8
9      What this module does (additive, pure-ish ? one DB read, no writes):
10     - ``resolve_user_fusion_config`` ? overlay the active ``user_models`` row's
  ``fusion_config``
11     onto the served fusion config dict, returning the config the served path WOULD use.
Falls
12     back to the served baseline (a copy, unchanged) when there is no active model, or
the model
13     is below the promotion-confidence floor (``n_videos_at_fit < min_n``), or it carries
no
14     fusion overlay. The min-N floor here is the SAME ``per_user_gate`` promotion floor:
below it
15     a per-user A/B is not trusted enough to override a baseline default (PLAN §1.3).
16     - ``load_user_classifier`` ? rehydrate the stored ``classifier_json`` into a
17     ``LogisticRegression`` via ``from_dict`` (None when the user has no active model or
it is a
18     config-only model). ``LogisticRegression.load`` is never called in
  ``backend/pipeline``
19     today ? this is the load half of the serving bridge.
20
21     ? NOT WIRED INTO THE SERVED PATH. ``fusion_hybrid._select_for_handoff`` still reads only
the
22     static ``idx_cfg["fusion"]`` ? it does NOT call anything here. Wiring this resolver into
that
23     seam (so a pinned, promoted per-user model actually changes the served handoff) is the
NEXT,
24     **owner-gated** PR, because it is the first change to served behavior. Until then this
is pure
25     groundwork: import it, unit-test it, but nothing in production reads it. With NULL
  ``user_id``
26     (the local install) and no active model, ``resolve_user_fusion_config`` returns the
served
27     config unchanged ? byte-identical to today.
28     """
29     from __future__ import annotations
30
31     import copy
32     from typing import Any, Dict, Optional
33
34     from backend.eval.classifier import LogisticRegression
35
36     __all__ = [
37         "DEFAULT_MIN_N",
38         "resolve_user_fusion_config",
39         "load_user_classifier",
40     ]
41
42     #: Promotion-confidence floor (PERSONALIZATION_PLAN.md §1.3, owner-confirmed ``min_n =
5``).
43     #: A per-user model fit on fewer than this many of the user's videos is NOT trusted
enough to
44     #: override a baseline fusion default ? we fall back to the served baseline. This
mirrors
45     #: ``backend.eval.per_user_gate.should_promote_for_user``'s ``min_n`` default.
46     DEFAULT_MIN_N = 5
47
48
49     def resolve_user_fusion_config(
50         db: Any,
51         user_id: Optional[str],

```

```

52     served_cfg: Dict[str, Any],
53     *,
54     min_n: int = DEFAULT_MIN_N,
55 ) -> Dict[str, Any]:
56     """Resolve the fusion config the served path should use for ``user_id``.
57
58     Overlays the active ``user_models`` row's ``fusion_config`` (a partial per-user
59     ``FusionConfig`` dict ? e.g. raised ``min_crossings`` / ``min_players`` / flipped
60     ``cue_flags``) onto a COPY of ``served_cfg`` (the baseline served fusion config
dict, i.e.
61     ``idx_cfg["fusion"]``). The original ``served_cfg`` is never mutated.
62
63     Falls back to the served baseline (an unmodified deep copy) when ANY of:
64     - the user has no active model;
65     - the active model was fit below the promotion floor (``n_videos_at_fit < min_n``)
? too
66         little of the user's own footage to trust a per-user override (PLAN §1.3);
67     - the active model carries no ``fusion_config`` overlay (a classifier-only model).
68
69     So a brand-new / NULL (local) user with no active model gets the served baseline
unchanged
70     ? never denied service, never altered. The overlay is a shallow key-level merge:
per-user
71     keys win, every key the user did not set keeps the baseline value.
72     """
73     base = copy.deepcopy(served_cfg)
74     model = db.get_active_user_model(user_id)
75     if not model:
76         return base
77     if int(model.get("n_videos_at_fit") or 0) < min_n:
78         # Below the trust floor ? keep the baseline (the conservative default).
79         return base
80     overlay = model.get("fusion_config")
81     if not overlay:
82         return base # config-only fallback: a classifier-only model has nothing to
overlay
83     base.update(overlay)
84     return base
85
86
87     def load_user_classifier(db: Any, user_id: Optional[str]) ->
Optional[LogisticRegression]:
88         """Rehydrate the active per-user classifier for ``user_id``, or None.
89
90         Returns a ``LogisticRegression`` rebuilt from the stored ``classifier_json`` (via
91         ``from_dict``) when the user has an active model that carries one; None when there
is no
92         active model or it is a config-only (no-classifier) model. The caller (the FUTURE
served
93         wiring) would feed this model's ``predict_proba`` over the persisted
``window_stats`` ?
94         the same math ``experiment.predict`` runs offline. Nothing calls this on the served
path
95         yet; it is the load half of the serving bridge.
96         """
97         model = db.get_active_user_model(user_id)
98         if not model:
99             return None
100         blob = model.get("classifier_json")
101         if not blob:
102             return None
103         return LogisticRegression.from_dict(blob)
104

```

These look careful. Let me examine the classifier (serialization round-trip via to_dict/from_dict) and the LogisticRegression math, since that's a load-bearing serialized output path that's easy to get subtly wrong.

17. user (2026-06-18T01:37:34.429Z)

```
1      """A tiny, dependency-free logistic-regression classifier (numpy only).
2
3      The offline learned segmenter (ADR-004) must run locally with no external API and
4      no heavy ML stack ? the project already depends on numpy, and sklearn is not
5      installed. This implements just enough: standardized features, balanced-class
6      gradient descent, sigmoid probabilities, and JSON (de)serialization so a trained
7      model is a small, human-readable, version-controllable file.
8
9      It deliberately stays simple: the rally-window features are few and roughly
10     linearly separable (high player motion + sustained activity => real rally), so a
11     well-regularized linear model is a sensible, inspectable baseline. Swap in a
12     richer model later behind the same fit/predict_proba/save/load interface.
13     """
14
15     import json
16     from typing import List, Optional
17
18     import numpy as np
19
20
21     class LogisticRegression:
22         def __init__(self, lr: float = 0.1, epochs: int = 1000, l2: float = 1e-3):
23             self.lr = lr
24             self.epochs = epochs
25             self.l2 = l2
26             self.w: Optional[np.ndarray] = None
27             self.b: float = 0.0
28             self.mean: Optional[np.ndarray] = None
29             self.std: Optional[np.ndarray] = None
30             self.feature_names: Optional[List[str]] = None
31
32         @staticmethod
33         def _sigmoid(z: np.ndarray) -> np.ndarray:
34             # Clip for numerical stability (avoid overflow in exp).
35             return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))
36
37         def _standardize(self, X: np.ndarray) -> np.ndarray:
38             return (X - self.mean) / self.std
39
40         def fit(self, X, y, feature_names: Optional[List[str]] = None,
41               class_weight: str = "balanced") -> "LogisticRegression":
42             """Fit with optional balanced class weighting (rally windows are imbalanced
43             ? many spurious candidates per real rally)."""
44             X = np.asarray(X, dtype=float)
45             y = np.asarray(y, dtype=float)
46             if X.ndim != 2 or X.shape[0] == 0:
47                 raise ValueError("X must be a non-empty 2D array")
48             self.feature_names = feature_names
49
50             self.mean = X.mean(axis=0)
51             self.std = X.std(axis=0)
52             self.std[self.std == 0] = 1.0 # guard constant features
53             Xs = self._standardize(X)
54             n, d = Xs.shape
55
56             if class_weight == "balanced":
57                 n_pos = max(1, int(y.sum()))
58                 n_neg = max(1, int((1 - y).sum()))
59                 w_pos, w_neg = n / (2.0 * n_pos), n / (2.0 * n_neg)
```

```

60         else:
61             w_pos = w_neg = 1.0
62             sw = np.where(y == 1, w_pos, w_neg)
63
64             self.w = np.zeros(d)
65             self.b = 0.0
66             for _ in range(self.epochs):
67                 p = self._sigmoid(Xs @ self.w + self.b)
68                 err = (p - y) * sw
69                 grad_w = Xs.T @ err / n + self.l2 * self.w
70                 grad_b = float(err.mean())
71                 self.w -= self.lr * grad_w
72                 self.b -= self.lr * grad_b
73             return self
74
75     def predict_proba(self, X) -> np.ndarray:
76         if self.w is None:
77             raise RuntimeError("model is not fitted")
78         Xs = self._standardize(np.asarray(X, dtype=float))
79         return self._sigmoid(Xs @ self.w + self.b)
80
81     def predict(self, X, threshold: float = 0.5) -> np.ndarray:
82         return (self.predict_proba(X) >= threshold).astype(int)
83
84     # --- persistence (small JSON, no pickle/joblib) -----
85
86     def to_dict(self) -> dict:
87         return {
88             "model": "logreg",
89             "lr": self.lr, "epochs": self.epochs, "l2": self.l2,
90             "w": self.w.tolist() if self.w is not None else None,
91             "b": self.b,
92             "mean": self.mean.tolist() if self.mean is not None else None,
93             "std": self.std.tolist() if self.std is not None else None,
94             "feature_names": self.feature_names,
95         }
96
97     @classmethod
98     def from_dict(cls, d: dict) -> "LogisticRegression":
99         m = cls(lr=d.get("lr", 0.1), epochs=d.get("epochs", 1000), l2=d.get("l2", 1e-3))
100         m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101         m.b = float(d.get("b", 0.0))
102         m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103         m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104         m.feature_names = d.get("feature_names")
105         return m
106
107     def save(self, path: str) -> None:
108         with open(path, "w") as f:
109             json.dump(self.to_dict(), f, indent=2)
110
111     @classmethod
112     def load(cls, path: str) -> "LogisticRegression":
113         with open(path) as f:
114             return cls.from_dict(json.load(f))
115

```

18. assistant (2026-06-18T01:37:37.456Z)

Clean. Let me now look at error/serialization paths in the eval modules that aren't excluded. Let me check `gt\_loader.py`, `golden\_fixtures.py`, `tolerance\_metrics.py`, and `promotion.py` ? these handle external file parsing and are error-prone.

19. user (2026-06-18T01:37:38.997Z)

```

1      """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3      The repo historically carried two incompatible loaders:
4      - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5        ``curate export`` format), raises on bad rows;
6      - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7        (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9      A file in one convention fed to the other loader yielded zero/garbage intervals with no
10     error. This module accepts BOTH conventions (header-mapped, else positional), so every
11     quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12     flows through one reader. Both legacy entry points now delegate here.
13
14     Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15     - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16       columns ? rally_number, ending_reason, sport, ? ? are ignored);
17     - no header: columns 0,1 positionally.
18     Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19     (``annotations.parse_timestamp``).
20     """
21     import csv
22     import logging
23     from typing import List, Optional, Tuple
24
25     from backend.eval.annotations import parse_timestamp
26
27     logger = logging.getLogger(__name__)
28
29     Interval = Tuple[float, float]
30
31     _START_NAMES = ("start", "start_time")
32     _END_NAMES = ("end", "end_time")
33
34     def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35         """If `row` is a header naming start/end columns, return their indices, else
36         None."""
37         cells = [(c or "").strip().lower() for c in row]
38         start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)
39         end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
40         if start_idx is not None and end_idx is not None:
41             return start_idx, end_idx
42         return None
43
44     def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45         """Load GT rally intervals from either golden-CSV convention.
46
47         strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48         (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49         strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50         skipped ? a silently shrinking GT set flattens precision.
51
52         Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53         (they break one-to-one matching and usually indicate a labeling error).
54         """
55         intervals: List[Interval] = []
56         skipped = 0
57         cols: Optional[Tuple[int, int]] = None # (start_idx, end_idx); None until detected
58         saw_header = False
59
60         with open(csv_path, newline="", encoding="utf-8-sig") as f:
61             for lineno, row in enumerate(csv.reader(f), start=1):
62                 if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):

```



```

63         continue
64     if cols is None and not saw_header:
65         header = _header_columns(row)
66         if header is not None:
67             cols = header
68             saw_header = True
69             continue # consume the header row
70         cols = (0, 1) # headerless: positional start,end
71     assert cols is not None # every branch above assigns it
72     si, ei = cols
73     try:
74         if len(row) <= max(si, ei):
75             raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76         start = parse_timestamp(row[si])
77         end = parse_timestamp(row[ei])
78         if start < 0:
79             raise ValueError(f"negative start ({start})")
80         if start >= end:
81             raise ValueError(f"start ({start}) >= end ({end})")
82     except ValueError as e:
83         if strict:
84             raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85         skipped += 1
86         continue
87     intervals.append((start, end))
88
89     if skipped:
90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92     intervals.sort(key=lambda iv: iv[0])
93     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94         if s2 < e1:
95             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                             "labeling error? Overlaps break one-to-one matching.",
97                             csv_path, s1, e1, s2, e2)
98     return intervals
99

```

20. user (2026-06-18T01:37:39.624Z)

```

1     """Golden Regression Fixtures (M1) ? video-free, CONTROL-only characterization harness.
2
3     `docs/GOLDEN_REGRESSION_FIXTURES.md` ($4, $6, $7). This module is the **single shared
4     replay code path** used by both the freeze tool and the characterization gate, and it
5     deliberately stays on the **CONTROL path** (D5 in that doc): the learned
6     variants train a numpy/OpenBLAS logistic regression whose near-threshold decisions flip
7     across CPUs, so they can never anchor a cross-machine byte/parse snapshot. Here we
replay
8     only `experiment.CONTROL` (`is_learned() == False`), which keeps all candidate
intervals,
9     merges overlaps, and **avoids numpy EXECUTION on the replay path** ? i.e. no numpy array
/
10    LogisticRegression math runs while replaying. (numpy is still *imported* transitively
via
11    `experiment` ? the learned-variant classifier module and MUST be installed; the CONTROL
12    path simply never calls into it.)
13
14    The replay reuses the already-shipped eval stack VERBATIM::
15
16    parse_trajectory_csv ? distill_local.build_candidates(proposal=pinned)
17    ? VideoCandidates(name, intervals, features={5 shuttle cols},
gts=load_gt_intervals)
18    ? experiment.predict(CONTROL, vc) ? metrics.score(preds, gts, iou)

```

```

19
20     Tier-0 / privacy invariants (D3, §4c, §6 of the doc), enforced here by construction:
21
22     * Fixtures are **synthetic** ? deterministic formulae, NO randomness ? so they carry
zero
23     provenance/licensing risk and are safe to commit publicly (owner Q7 "synthetic-only").
24     * Every fixture is tagged ``kind="synthetic"`, ``minors_free=true``,
``license="synthetic"`.
25     * ``expected.json`` carries **only the 5 shuttle feature columns** ? there is, by
design, no
26     person / pose / gait / face / audio field anywhere in the schema (positive-assertion
test).
27     * The windowing params ``(vt, mg, mwd)`` are **pinned inside each fixture's provenance**
and
28     passed explicitly to ``build_candidates`` as a raw 3-tuple ? we do NOT import or rely
on the
29     named ``windowing.SERVED`` constant / ``WindowingPreset``, so a future SERVED retune
cannot
30     silently churn fixtures.
31     * ``max_win`` (W1 bounded-windowing) is **RECORDED** in provenance/manifest but NOT
applied by
32     the replay: ``distill_local.build_candidates`` (shared code, intentionally not
modified here)
33     takes only the 3-tuple ``(vt, mg, mwd)`` and
``TrackNetRunner.trajectory_to_action_windows``
34     has no max-window arg, so these fixtures characterize the **W1-OFF** path by design.
The pinned
35     ``max_win`` is a forward-looking annotation, NOT a knob the gate exercises ? do not
assume it
36     tracks a future SERVED W1 flip (it would need new wiring + a re-freeze to take
effect).
37
38     §6 honest-limits caveat (carried in the CLI / test messages, not only the docs): a green
suite proves only that the deterministic post-trajectory transforms are unchanged for
the
40     frozen synthetic cases ? NOT that the detector/decoder/AI handoff/DB are correct, and
NOT
41     that rally detection is good. Synthetic-tier numbers measure plumbing correctness, never
field quality (never cite them in QUALITY_ITERATIONS.md / the paper).
42
43     Offline + deterministic. The replay imports `tracknet_runner` (stdlib-only for the
parse/window math ? no TF) and reaches no numpy/cv2 EXECUTION on the CONTROL path, but
it is
46     NOT a zero-third-party-import module: importing `experiment` transitively imports numpy
(via
47     the learned-variant classifier), so numpy must be installed even though the CONTROL
replay
48     never calls into it.
49     """
50     from __future__ import annotations
51
52     import argparse
53     import json
54     import math
55     import os
56     import re
57     import shutil
58     import sys
59     import tempfile
60     from dataclasses import dataclass
61     from pathlib import Path
62     from typing import Any, Dict, List, Optional, Sequence, Tuple
63
64     # --- shipped eval stack, reused verbatim (CONTROL / pure-stdlib path only)
-----

```

```

65     from backend.eval import distill_local
66     from backend.eval.experiment import CONTROL, VideoCandidates, predict
67     from backend.eval.gt_loader import load_gt_intervals
68     from backend.eval.metrics import score
69     from backend.eval.regression import gt_hash
70     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
71
72     Interval = Tuple[float, float]
73
74     #: Bump when the fixture/expected schema changes shape. Loaders refuse a fixture whose
75     #: ``schema_version`` exceeds this (forward-incompatible), per the schema-drift guard.
76     CURRENT_SCHEMA = 1
77
78     #: Repo-root-relative fixtures dir, resolved from THIS file (never cwd) so the harness
works
79     #: from any working directory / a fresh clone. ``parents[2]`` = backend/eval ? backend ?
root.
80     FIXTURES_DIR: Path = Path(__file__).resolve().parents[2] / "eval_baselines" / "fixtures"
81
82     #: The pinned SERVED windowing the synthetic fixtures freeze at: (velocity_thresh,
merge_gap,
83     #: min_window_duration). Mirrors the historical SERVED values (0.02, 2.0, 2.0) but is a
LITERAL
84     #: here on purpose ? the spec forbids depending on the named ``windowing.SERVED``
constant so a
85     #: future retune of SERVED can never silently re-baseline a committed fixture. Each
fixture also
86     #: re-states these in its provenance; the replay reads them from there.
87     PINNED_VT = 0.02
88     PINNED_MERGE_GAP = 2.0
89     PINNED_MIN_WINDOW = 2.0
90     #: W1 (bounded-windowing) max-window seconds. 0 = OFF, matching the SERVED default.
NOTE: this is
91     #: only RECORDED (provenance/manifest) ? it is NOT applied by the replay.
``_pinned_proposal``
92     #: returns just ``(vt, mg, mwd)`` and ``distill_local.build_candidates`` (shared code,
deliberately
93     #: NOT modified by this harness) takes no max-window arg, so the fixtures characterize
the W1-OFF
94     #: path. Do NOT assume this value tracks a future SERVED W1 flip ? that would need new
wiring.
95     PINNED_MAX_WINDOW = 0.0
96
97     #: The 5 fps/resolution-invariant shuttle feature columns produced by distill_local. Re-
stated
98     #: here (not imported as a mutable alias) so a fixture's expected.json is pinned to
exactly these.
99     SHUTTLE_FEATURES: Tuple[str, ...] = (
100         "duration", "peak_velocity", "mean_velocity", "active_ratio", "net_crossings",
101     )
102     assert tuple(distill_local.FEATURE_NAMES) == SHUTTLE_FEATURES, (
103         "distill_local.FEATURE_NAMES drifted from the pinned shuttle feature set"
104     )
105
106     #: Forbidden key substrings ? biometric / identity / non-shuttle streams that MUST NOT
appear
107     #: anywhere in a Tier-0 fixture (D3 / S4c). The privacy assertion scans for these.
108     FORBIDDEN_KEYS: Tuple[str, ...] = ("person", "pose", "gait", "face", "audio", "player")
109
110     _FLOAT_ROUND_DP = 6 # all written floats rounded to this many decimals (determinism)
111     _FLOAT_TOL = 1e-6 # abs tolerance when parse-comparing recomputed vs committed floats
112
113     #: A 32-hex-char MD5 (the ``video_id`` shape, ``compute_video_id``) ? the leak-scan
refuses to
114     #: emit a fixture whose written text contains one (it would be an attribution back-

```

```

channel, $4c/$5).
115     _MD5_RE = re.compile(r"\b[0-9a-fA-F]{32}\b")
116
117     #: Strict slug charset for a real-fixture ``--label`` ? lowercase alnum, ``_``/``-`` (no
spaces,
118     #: no punctuation, no uppercase). A free-text label ("Lin Dan vs Lee, India Open 2024")
is an
119     #: attribution back-channel that ``_scan_forbidden`` does NOT catch (it isn't a
FORBIDDEN_KEY / MD5
120     #: / source-path token), so it is rejected up-front by construction (red-team: free-text
id leak).
121     _LABEL_SLUG_RE = re.compile(r"^[a-z0-9][a-z0-9_\-]*$")
122
123
124     # =====
125     # Synthetic trajectory generation (deterministic ? NO random)
126     # =====
127
128
129     @dataclass(frozen=True)
130     class RallySpec:
131         """One in-flight rally segment: the shuttle oscillates across the net midline.
132
133         ``start_frame`` ? first frame index; ``n_frames`` ? length; ``period`` ? oscillation
134         period in frames (governs net-crossing count); ``amplitude`` ? px excursion from the
135         net midline along the play axis. Deterministic sine motion ? fixed
velocity/crossings.
136         """
137
138         start_frame: int
139         n_frames: int
140         period: float = 30.0
141         amplitude: float = 400.0
142
143
144     @dataclass(frozen=True)
145     class GapSpec:
146         """Dead-time between rallies ? what separates one window from the next.
147
148         ``kind="still"`` ? shuttle visible but resting (velocity ? 0 ? no active frames);
149         ``kind="occluded"`` ? shuttle not visible (Visibility=0 ? trajectory broken). Both
150         end an active run so the next rally forms a distinct window.
151         """
152
153         start_frame: int
154         n_frames: int
155         kind: str = "still" # "still" | "occluded"
156
157
158     @dataclass(frozen=True)
159     class FixtureSpec:
160         """A full synthetic scenario: a frame-indexed sequence of rallies and gaps + its GT.
161
162         ``gts`` are the hand-authored ground-truth rally intervals (seconds) ? the synthetic
163         "labels.csv". For a synthetic fixture they line up with the rallies by construction,
164         which is exactly what makes the quality-floor a *plumbing-correctness* check, not a
165         field-quality one ($6).
166         """
167
168         slug: str
169         description: str
170         fps: float
171         frame_width: float
172         frame_height: float
173         segments: Tuple[Any, ...] # ordered (RallySpec | GapSpec)

```

```

174     gts: Tuple[Interval, ...]
175     net_midline: float = 640.0
176
177
178     def _rally_points(spec: RallySpec, net_midline: float, frame_height: float
179                       ) -> List[Tuple[int, int, float, float]]:
180         """Deterministic in-flight shuttle: sine oscillation across the net midline (play
axis = x)."""
181         rows: List[Tuple[int, int, float, float]] = []
182         y_mid = frame_height / 2.0
183         for i in range(spec.n_frames):
184             x = net_midline + spec.amplitude * math.sin(2.0 * math.pi * i / spec.period)
185             # A small orthogonal wobble keeps y-spread < x-spread so the play axis is
unambiguously x.
186             y = y_mid + 30.0 * math.sin(2.0 * math.pi * i / (spec.period / 2.0))
187             rows.append((spec.start_frame + i, 1, x, y))
188         return rows
189
190
191     def _gap_points(spec: GapSpec, net_midline: float, frame_height: float
192                   ) -> List[Tuple[int, int, float, float]]:
193         """Dead-time rows: resting-visible (still) or absent (occluded)."""
194         rows: List[Tuple[int, int, float, float]] = []
195         y_mid = frame_height / 2.0
196         for i in range(spec.n_frames):
197             if spec.kind == "occluded":
198                 rows.append((spec.start_frame + i, 0, 0.0, 0.0))
199             elif spec.kind == "still":
200                 rows.append((spec.start_frame + i, 1, net_midline, y_mid))
201             else: # pragma: no cover - guarded by builder
202                 raise ValueError(f"unknown gap kind {spec.kind!r} (use 'still'/'occluded')")
203         return rows
204
205
206     def make_synthetic_trajectory(spec: FixtureSpec) -> List[Tuple[int, int, float, float]]:
207         """Build the deterministic ``Frame, Visibility, X, Y`` rows for a synthetic
scenario."""
208         rows: List[Tuple[int, int, float, float]] = []
209         for seg in spec.segments:
210             if isinstance(seg, RallySpec):
211                 rows.extend(_rally_points(seg, spec.net_midline, spec.frame_height))
212             elif isinstance(seg, GapSpec):
213                 rows.extend(_gap_points(seg, spec.net_midline, spec.frame_height))
214             else: # pragma: no cover - guarded by dataclass shape
215                 raise TypeError(f"unknown segment type {type(seg).__name__}")
216         rows.sort(key=lambda r: r[0])
217         return rows
218
219
220     def write_trajectory_csv(rows: Sequence[Tuple[int, int, float, float]], path: Path) ->
None:
221         """Write trajectory rows as the TrackNet CSV (``Frame,Visibility,X,Y``), LF + 6dp
floats."""
222         lines = ["Frame,Visibility,X,Y"]
223         for frame, vis, x, y in rows:
224             lines.append(f"{int(frame)},{int(vis)},{_fmt(x)},{_fmt(y)}")
225         path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
226
227
228     def write_labels_csv(gts: Sequence[Interval], path: Path) -> None:
229         """Write GT intervals as a ``start,end`` labels CSV (the strict gt_loader format),
LF."""
230         lines = ["start,end"]
231         for s, e in gts:
232             lines.append(f"{_fmt(s)},{_fmt(e)}")

```

```

233     path.write_text("\n".join(lines) + "\n", encoding="utf-8", newline="\n")
234
235
236     # =====
237     # Built-in synthetic scenarios (the 3 committed slugs)
238     # =====
239
240
241     def _fr(seconds: float, fps: float) -> int:
242         return int(round(seconds * fps))
243
244
245     def builtin_specs() -> List[FixtureSpec]:
246         """The committed synthetic scenarios. Deterministic; chosen so each exercises a
distinct
247         windowing behaviour (one window / dead-time split / occlusion split)."""
248         fps = 30.0
249         fw, fh = 1280.0, 720.0
250         net = 640.0
251
252         # 1) clean_single_rally: one ~4s in-flight rally ? exactly 1 candidate window,
crossings>0.
253         single = FixtureSpec(
254             slug="clean_single_rally",
255             description="One continuous ~4s rally; shuttle oscillates across the net ? 1
window.",
256             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
257             segments=(RallySpec(start_frame=0, n_frames=120),),
258             # Window is [0.033, 3.967]; GT is the rally span the synthetic motion fills.
259             gts=((0.0, 4.0),),
260         )
261
262         # 2) multi_rally_with_deadtime: rally, 3s STILL dead-time, rally ? 2 windows split
by the
263         # low-velocity gap (> merge_gap=2.0s) ? proves dead-time separates windows.
264         r0 = RallySpec(start_frame=0, n_frames=120)
265         gap_still = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="still")
266         r1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
267         multi = FixtureSpec(
268             slug="multi_rally_with_deadtime",
269             description="Two ~4s rallies separated by 3s of still (resting-shuttle) dead-
time ? 2 windows.",
270             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
271             segments=(r0, gap_still, r1),
272             gts=((0.0, 4.0), (7.0, 11.0)),
273         )
274
275         # 3) occlusion_break: rally, 3s OCCLUDED (Visibility=0), rally ? 2 windows split by
the
276         # visibility break ? proves occlusion (not just low velocity) separates windows.
277         o0 = RallySpec(start_frame=0, n_frames=120)
278         gap_occ = GapSpec(start_frame=120, n_frames=_fr(3.0, fps), kind="occluded")
279         o1 = RallySpec(start_frame=120 + _fr(3.0, fps), n_frames=120)
280         occ = FixtureSpec(
281             slug="occlusion_break",
282             description="Two ~4s rallies separated by 3s of shuttle occlusion (Visibility=0)
? 2 windows.",
283             fps=fps, frame_width=fw, frame_height=fh, net_midline=net,
284             segments=(o0, gap_occ, o1),
285             gts=((0.0, 4.0), (7.0, 11.0)),
286         )
287
288         return [single, multi, occ]
289
290

```

```

291     def _spec_by_slug() -> Dict[str, FixtureSpec]:
292         return {s.slug: s for s in builtin_specs()}
293
294
295     # =====
296     # The shared replay (CONTROL / pure-stdlib path)
297     # =====
298
299
300     def _fmt(x: float) -> str:
301         """Render a float with up to 6dp, no scientific notation, ``-0.0`` normalised to
302         ``0.0``."""
303         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
304         if r == 0.0:
305             r = 0.0
306         return f"{r:.6f}"
307
308     def _round(x: float) -> float:
309         r = round(float(x) + 0.0, _FLOAT_ROUND_DP)
310         return 0.0 if r == 0.0 else r
311
312
313     def _pinned_proposal(prov: Optional[Dict[str, Any]]) -> Tuple[float, float, float]:
314         """The windowing proposal tuple ``(vt, mg, mwd)``, read from the fixture provenance
315         when present
316         (the pinned params travel with the fixture), else the module-level pinned defaults.
317         NEVER
318         ``windowing.SERVED``. NOTE: the provenance ``max_win`` (W1) is intentionally NOT
319         read here ?
320         ``build_candidates`` takes only this 3-tuple, so the replay characterizes the W1-OFF
321         path; the
322         pinned ``max_win`` is a recorded annotation, not a knob the gate applies (see
323         PINNED_MAX_WINDOW)."""
324         if prov and "windowing" in prov:
325             w = prov["windowing"]
326             return (float(w["vt"]), float(w["mg"]), float(w["mwd"]))
327         return (PINNED_VT, PINNED_MERGE_GAP, PINNED_MIN_WINDOW)
328
329
330     def replay_fixture(slug_or_dir: Any) -> Dict[str, Any]:
331         """Replay one fixture through the shipped CONTROL path; return its recomputed
332         snapshot.
333         Resolves ``slug_or_dir`` (a slug string or a fixture directory ``Path``) to its
334         committed
335         ``trajectory.csv`` + ``labels.csv`` + ``provenance.json``, then:
336
337         parse_trajectory_csv ? build_candidates(proposal=pinned) ? VideoCandidates(5
338         shuttle
339         cols only) ? predict(CONTROL, vc) ? score(preds, gts, iou).
340
341         Returns ``{slug, schema_version, iou, candidates:[{start,end,shuttle:{5 names}}],
342         control_segments:[[s,e]], score:{...}, gt_hash}``. CONTROL keeps every candidate
343         then
344         merges overlaps; NO numpy is touched.
345         """
346         fdir = _resolve_dir(slug_or_dir)
347         slug = fdir.name
348         traj_csv = fdir / "trajectory.csv"
349         labels_csv = fdir / "labels.csv"
350         prov = _read_json(fdir / "provenance.json") if (fdir / "provenance.json").is_file()
351         else None
352
353         fps, fw = _fps_fw(prov, slug)

```

```

345     iou = float(prov["iou"]) if (prov and "iou" in prov) else 0.5
346     proposal = _pinned_proposal(prov)
347
348     intervals, feature_rows = distill_local.build_candidates(
349         str(traj_csv), fps=fps, frame_width=fw, proposal=proposal)
350
351     # Build VideoCandidates DIRECTLY with ONLY the 5 shuttle feature columns. We never
call
352     # fusion_compare.load_videos / ablation.load_videos ? those read c['person']
unconditionally
353     # and would KeyError on these person-free fixtures (HARD CONSTRAINT 1).
354     features: Dict[str, List[float]] = {
355         name: [row[i] for row in feature_rows] for i, name in
enumerate(SHUTTLE_FEATURES)
356     }
357     gts = load_gt_intervals(str(labels_csv), strict=True)
358     vc = VideoCandidates(name=slug, intervals=intervals, features=features, gts=gts)
359
360     preds = predict(CONTROL, vc) # CONTROL: keep all ? merge_overlaps; pure-stdlib, no
numpy
361     sc = score(preds, gts, iou)
362
363     candidates: List[Dict[str, Any]] = []
364     for j, (s, e) in enumerate(intervals):
365         shuttle = {name: _round(feature_rows[j][i]) for i, name in
enumerate(SHUTTLE_FEATURES)}
366         # net_crossings is conceptually an integer count ? keep it exact (int), per
constraint 2.
367         shuttle["net_crossings"] =
int(round(feature_rows[j][SHUTTLE_FEATURES.index("net_crossings")]))
368         candidates.append({"start": _round(s), "end": _round(e), "shuttle": shuttle})
369
370     return {
371         "slug": slug,
372         "schema_version": CURRENT_SCHEMA,
373         "iou": _round(iou),
374         "candidates": candidates,
375         "control_segments": [[_round(s), _round(e)] for s, e in preds],
376         "score": {
377             "iou_threshold": _round(sc.iou_threshold),
378             "num_pred": int(sc.num_pred),
379             "num_gt": int(sc.num_gt),
380             "true_positives": int(sc.true_positives),
381             "false_positives": int(sc.false_positives),
382             "false_negatives": int(sc.false_negatives),
383             "precision": _round(sc.precision),
384             "recall": _round(sc.recall),
385             "f1": _round(sc.f1),
386             "mean_iou": _round(sc.mean_iou),
387             "mean_start_error": _round(sc.mean_start_error),
388             "mean_end_error": _round(sc.mean_end_error),
389         },
390         "gt_hash": gt_hash(gts),
391     }
392
393
394     # =====
395     # Freeze / refresh (expected.json + provenance.json are correct-by-construction)
396     # =====
397
398
399     def _provenance(spec_or_dir: Any, slug: str, fps: float, fw: float, iou: float = 0.5
400         ) -> Dict[str, Any]:
401         """The committed provenance block ? pins the windowing + privacy/licensing tags
($4c, $5)."""

```



```

402     desc = ""
403     if isinstance(spec_or_dir, FixtureSpec):
404         desc = spec_or_dir.description
405     return {
406         "slug": slug,
407         "schema_version": CURRENT_SCHEMA,
408         "kind": "synthetic",
409         "license": "synthetic",
410         "minors_free": True,
411         "description": desc,
412         "fps": _round(fps),
413         "frame_width": _round(fw),
414         "iou": _round(iou),
415         "windowing": {
416             "vt": PINNED_VT,
417             "mg": PINNED_MERGE_GAP,
418             "mwd": PINNED_MIN_WINDOW,
419             "max_win": PINNED_MAX_WINDOW,
420         },
421         "paths": {
422             "trajectory": "trajectory.csv",
423             "labels": "labels.csv",
424             "expected": "expected.json",
425         },
426         "note": (
427             "Synthetic, deterministic, owned (no real footage). Tier-0 / CONTROL-only. "
428             "Plumbing-correctness fixture ? NOT a measure of rally-detection quality
429         ),
430     }
431
432
433     def freeze_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR,
434                       iou: float = 0.5) -> Dict[str, Any]:
435         """Compute ``expected.json`` + ``provenance.json`` for one fixture from its
436 COMMITTED
437         ``trajectory.csv`` + ``labels.csv`` via the replay (so expected is correct-by-
438 construction).
439
440         Idempotent: rewrites the same bytes given the same committed inputs. Requires the
441 trajectory
442 + labels CSVs to already exist (use ``--freeze-synthetic`` to generate them first).
443
444         """
445         fdir = fixtures_dir / slug
446         traj_csv, labels_csv = fdir / "trajectory.csv", fdir / "labels.csv"
447         if not traj_csv.is_file() or not labels_csv.is_file():
448             raise FileNotFoundError(
449                 f"{slug}: trajectory.csv/labels.csv missing ? run --freeze-synthetic to
450 generate them")
451
452         # Provenance first (it pins the windowing the replay reads), then replay ? expected.
453         spec = _spec_by_slug().get(slug)
454         fps, fw = (spec.fps, spec.frame_width) if spec is not None else
455 _fps_fw_from_prov(fdir, slug)
456         prov = _provenance(spec if spec is not None else fdir, slug, fps, fw, iou)
457         _write_json(fdir / "provenance.json", prov)
458
459         expected = replay_fixture(fdir)
460         _write_json(fdir / "expected.json", expected)
461         return expected
462
463
464     def freeze_synthetic(fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
465         """Generate trajectory.csv + labels.csv for every built-in scenario, freeze
466 expected.json +

```

```

460     provenance.json, and (re)write the manifest. Returns the slugs frozen."""
461     specs = builtin_specs()
462     fixtures_dir.mkdir(parents=True, exist_ok=True)
463     slugs: List[str] = []
464     for spec in specs:
465         fdir = fixtures_dir / spec.slug
466         fdir.mkdir(parents=True, exist_ok=True)
467         rows = make_synthetic_trajectory(spec)
468         write_trajectory_csv(rows, fdir / "trajectory.csv")
469         write_labels_csv(spec.gts, fdir / "labels.csv")
470         freeze_fixture(spec.slug, fixtures_dir=fixtures_dir)
471         slugs.append(spec.slug)
472     _write_manifest(specs, fixtures_dir)
473     return slugs
474
475
476     def refresh_snapshot(slug: Optional[str] = None, all_slugs: bool = False,
477                         fixtures_dir: Path = FIXTURES_DIR) -> List[str]:
478         """Re-freeze expected.json (+ provenance.json) from the COMMITTED trajectory.csv +
479         labels.csv ?
480         NO regeneration of the trajectory/labels. ? §6: this can rubber-stamp a real
481         regression; the
482         mitigation is reviewing the visible diff + recording a reason (the CLI requires
483         one)."""
484         if all_slugs:
485             targets = [d.name for d in sorted(fixtures_dir.iterdir())
486                       if d.is_dir() and (d / "trajectory.csv").is_file()]
487         elif slug:
488             targets = [slug]
489         else:
490             raise ValueError("refresh_snapshot needs --slug S or --all")
491         for s in targets:
492             freeze_fixture(s, fixtures_dir=fixtures_dir)
493         return targets
494
495     # =====
496     # P1 ? De-attribution + minimization: OWNED real trajectory+labels ? committable Tier-0
497     # fixture, reusing the SAME replay_fixture snapshot. (docs/GOLDEN_REGRESSION_FIXTURES.md
498     # §4c anti-attribution, §5 threat model, §7 row P1, §6 honest limits.)
499     # =====
500
501     class RefusalError(ValueError):
502         """The provenance/biometric refusal gate (M2) rejected an unsafe freeze.
503
504         A subclass of ``ValueError`` (so existing ``except ValueError`` paths still catch
505         it) that
506         names *why* the unsafe public-fixture path was refused. The refusal is the code-
507         enforced
508         provenance gate (§4c "provenance hard-fail gate", §7 ?): the owner-recorded /
509         minors-excluded
510         / license flags are human-asserted, but the gate makes the unsafe path impossible to
511         reach
512         *silently* ? a fixture cannot be written unless all three are affirmatively set.
513         """
514
515     def _new_surrogate_slug() -> str:
516         """An opaque RANDOM surrogate slug ? ``fx_<16 hex>`` from ``os.urandom`` (§4c).
517
518         Deliberately NOT the MD5 ``video_id`` and NOT ``HMAC(salt, video_id?name)`` ? a
519         random,
520         NON-reversible id with no algebraic link to the source, so it can't be brute-forced
521         over a

```

```

516         tiny known roster even if a salt leaked. The surrogate?source map (if kept) lives
off-git.
517         ""
518         return f"fx_{os.urandom(8).hex()}"
519
520
521     #: The fixed schema filenames the writers ALWAYS emit (as ``paths`` literals). A source-
path token
522     #: that EXACTLY equals one of these is NOT a leak (the bytes would be written
regardless), so it is
523     #: excluded. Matched by exact membership, NEVER substring containment ? a substring rule
would let
524     #: a real source named ``label.csv`` (stem ``label`` ? ``labels.csv``) leak unnoticed
(red-team #4).
525     _SCHEMA_FILENAME_LITERALS: Tuple[str, ...] = (
526         "trajectory", "trajectory.csv", "labels", "labels.csv", "expected", "expected.json",
527         "provenance", "provenance.json",
528     )
529
530
531     def _scan_forbidden(text: str, *, where: str, source_paths: Sequence[str]) -> None:
532         """POSITIVE privacy assertion (§4c, red-team ?): raise if ``text`` leaks anything
attributable.
533
534         Scans the written JSON *text* for (a) any ``FORBIDDEN_KEYS`` biometric/identity
substring,
535         (b) a 32-hex MD5 (the ``video_id`` shape), or (c) any identifying token of a source
path
536         (full path / basename / stem) ? excluding the fixed schema filename literals the
writers
537         always emit. The trajectory is shuttle-only by nature; this guards against a
*regression*
538         re-introducing a person/audio/pose stream or an id/path back-channel ? never the
silent default.
539         ""
540         low = text.lower()
541         for key in FORBIDDEN_KEYS:
542             if key in low:
543                 raise RefusalError(
544                     f"{where}: forbidden key substring {key!r} found in written output ? "
545                     f"Tier-0 fixtures are shuttle-only; person/pose/gait/face/audio/player
MUST NOT "
546                     f"appear (docs/GOLDEN_REGRESSION_FIXTURES.md §4c).")
547         m = _MD5_RE.search(text)
548         if m:
549             raise RefusalError(
550                 f"{where}: a 32-hex MD5 ({m.group(0)!r}) found in written output ? that is
the "
551                 f"video_id shape and an attribution back-channel; never write it (§4c/§5).")
552         for sp in source_paths:
553             if not sp:
554                 continue
555             p = Path(sp)
556             for token in (sp, p.name, p.stem):
557                 t = (token or "").strip().lower()
558                 # Skip ONLY a token that EXACTLY equals a schema-literal the writers always
emit
559                 # (e.g. a source named ``trajectory.csv``) ? that collision is not a leak.
We must NOT
560                 # skip on substring containment: a token like ``label`` or ``traj`` is a
SUBSTRING of
561                 # ``labels.csv``/``trajectory.csv``, so substring-skip would silently miss a
real source
562                 # named ``label.csv`` leaking its stem (red-team #4). Exact membership only.
563                 if not t or t in _SCHEMA_FILENAME_LITERALS:

```

```

564         continue
565     if t in low:
566         raise RefusalError(
567             f"{where}: source-path token {token!r} found in written output ? the
source "
568             f"filename/path must never be persisted ($4c metadata strip).")
569
570
571     def _reset_trajectory_rows(points: Sequence[Any], t0_frame: int
572                               ) -> List[Tuple[int, int, float, float]]:
573         """Shift every ``TrajectoryPoint`` frame by ``-t0_frame`` ?
574         ``(Frame,Visibility,X,Y)`` rows.
575         Visibility/X/Y are passed through untouched (the shuttle geometry is metric-
invariant under a
576         pure time shift); only the absolute frame index moves toward 0, severing the match
timeline.
577         """
578         rows: List[Tuple[int, int, float, float]] = []
579         for p in points:
580             rows.append((int(p.frame) - t0_frame, 1 if p.visible else 0, float(p.x),
float(p.y)))
581         rows.sort(key=lambda r: r[0])
582         return rows
583
584
585     def _reset_labels(gts: Sequence[Interval], t0_sec: float) -> List[Interval]:
586         """Shift every label by ``-t0_sec`` (same offset as the trajectory ? metric-
invariant).
587         ``t0_sec`` MUST already be quantized to ``_FLOAT_ROUND_DP`` decimals by the caller.
Each label
588         endpoint is ALSO rounded to 6dp BEFORE subtracting, so the shift is the difference
of two exact
589         6dp quantities ? no double-rounding (``round6(orig - round6(t0))`` vs the no-reset
590         ``round6(orig)`` accumulated error; red-team #2). This keeps the RAW reset-vs-no-
reset score
591         delta at the float-error floor (~3.3e-7, well within ``_FLOAT_TOL``); see
``freeze_real_fixture``
592         for why the SERIALIZED 6dp boundary error can still differ by one 6dp ULP on a sub-
second
593         offset (the grid, not a real metric divergence)."""
594         return [(_round(s) - t0_sec, _round(e) - t0_sec) for s, e in gts]
595
596
597
598     def freeze_real_fixture(
599         label: str,
600         trajectory_csv_path: Any,
601         labels_csv_path: Any,
602         *,
603         license: str,
604         minors_free: bool,
605         owned: bool,
606         fps: float,
607         frame_width: float,
608         fixtures_dir: Path = FIXTURES_DIR,
609         slug: Optional[str] = None,
610         time_origin_reset: bool = True,
611         source_note: str = "",
612     ) -> Dict[str, Any]:
613         """Turn an OWNED, minors-free real trajectory + golden labels into a committable,
de-attributed
614         Tier-0 fixture ? reusing M1's ``replay_fixture`` for the snapshot. ($4c / $5 / $7
row P1.)
615

```

```

616         The refusal gate (M2) makes the unsafe path impossible to reach silently: a fixture
is emitted
617         ONLY when ``minors_free is True`` AND ``owned is True`` AND ``license`` is a non-
blank string,
618         AND ``label`` is a strict slug (lowercase alnum + ``_``/``-`` ? a free-text
identifier like a
619         player/event name is refused, red-team #1). The committed bytes carry NO filename /
video_id /
620         match / player / capture-date / operator free-text ? only an opaque random surrogate
slug, the
621         shuttle-only trajectory (time-origin reset by default), the reset labels, and a
622         ``kind="cleared_real"`` provenance block. A positive privacy assertion re-reads the
written
623         ``expected.json`` + ``provenance.json`` and refuses if anything attributable slipped
in.
624
625         ``source_note`` is **accepted but NOT persisted** ? operator free-text is an
attribution
626         back-channel ``_scan_forbidden`` cannot fully police (an identifying note that
happens to
627         contain no FORBIDDEN_KEY / MD5 / source-path token would slip through), so it never
reaches the
628         committed ``provenance.json`` (red-team #1). It is still scanned defensively and the
flag is
629         retained for CLI compatibility / future logging.
630
631         NOTE ($6 honest limits): the random surrogate + reset timeline make the fixture non-
attributable
632         only *relative to the unpublished source* ? it is NOT absolutely anonymous (EDPS v.
SRB). De-
633         attribution does not cure provenance/licensing; this sits BEHIND counsel sign-off
(P2) and the
634         owner-asserted Fork-A / minors / biometric-exclusion flags.
635
636         The fixture is written into a temp dir and promoted (``os.replace``) to the final
slug dir ONLY
637         after the privacy scan passes ? a refused / failed freeze leaves NO partial fixture
dir
638         (red-team #3).
639
640         Returns the recomputed ``expected.json`` snapshot dict (identical to
``replay_fixture``).
641         """
642         # --- (a) REFUSAL GATE (M2) ? the code-enforced provenance hard-fail ($4c / $7 ?)
-----
643         reasons: List[str] = []
644         if minors_free is not True:
645             reasons.append("minors_free must be True (DPDP child = under 18; behavioural
monitoring "
646                             "of a minor is prohibited ? $3, guardrail 'exclude minors')")
647         if owned is not True:
648             reasons.append("owned must be True (only owner-recorded Fork-A source is safe to
commit; "
649                             "broadcast/scraped = Fork B, do NOT commit ? $3 D1)")
650         if not (isinstance(license, str) and license.strip()):
651             reasons.append("license must be a non-blank string (per-record provenance tag ?
$4c "
652                             "'provenance hard-fail gate')")
653         # label is committed verbatim into provenance ? it MUST be a strict slug, never
operator free
654         # text (a name/event), which _scan_forbidden cannot reliably catch (red-team #1
free-text leak).
655         if not (isinstance(label, str) and _LABEL_SLUG_RE.match(label)):
656             reasons.append(
657                 f"label must match the strict slug charset {_LABEL_SLUG_RE.pattern!r}")

```

```

(lowercase "
658         f"alnum + '_'/'-', no spaces/punctuation) ? a free-text identifier
(player/event "
659         f"name) is an attribution leak; got {label!r} ($4c metadata strip, red-team
#1)")
660     if reasons:
661         raise RefusalError(
662             "REFUSED to freeze a real Tier-0 fixture ? unsafe provenance (de-attribution
does NOT "
663             "cure provenance/licensing; this gate sits behind counsel sign-off,
$6/P2):\n - "
664             + "\n - ".join(reasons))
665
666     traj_path = Path(str(trajjectory_csv_path))
667     labels_path = Path(str(labels_csv_path))
668
669     # --- (b) opaque RANDOM surrogate slug ? never the video_id MD5, never the source
name ---
670     out_slug = slug if slug is not None else _new_surrogate_slug()
671
672     # --- (c) read via the SHIPPED readers; labels via the STRICT loader (never lenient)
-----
673     points = TrackNetRunner.parse_trajectory_csv(str(traj_path))
674     if not points:
675         raise ValueError(f"{traj_path}: no parseable trajectory rows
(Frame,Visibility,X,Y)")
676     gts = load_gt_intervals(str(labels_path), strict=True)
677
678     # --- (d) consistent time-origin reset (default ON; metric-invariant within
_FLOAT_TOL, $4c) --
679     # Subtract the SAME offset from BOTH the trajectory frames and the label seconds:
metrics.score
680     # is built from pure differences (interval_iou, |start-start|, |end-end|), so a pure
time shift
681     # leaves the score essentially unchanged ? only the absolute timeline is severed. To
keep the
682     # two shifts a consistent quantity (and avoid double-rounding past _FLOAT_TOL ? red-
team #2),
683     # t0_sec is ROUNDED to 6dp BEFORE subtracting (and _reset_labels rounds each
endpoint first), so
684     # the RAW reset-vs-no-reset score delta sits at the float-error floor (~3.3e-7 ?
_FLOAT_TOL).
685     #
686     # HONEST LIMIT (not a bug): the trajectory shifts by an INTEGER frame count while
labels shift
687     # in SECONDS, and 6dp-rounding is NOT additive (round6(8/fps) - round6(7/fps) ?
round6(1/fps)),
688     # so the *serialized* 6dp boundary error (mean_start/end_error) can differ by ONE
6dp ULP
689     # (1e-6) on a sub-second offset. That is the serialization grid, not a metric
divergence ? the
690     # score is bit-identical for whole-second offsets and within _FLOAT_TOL otherwise
(test #2).
691     if time_origin_reset:
692         t0_frame = min(int(p.frame) for p in points)
693         t0_sec = _round(t0_frame / float(fps))
694     else:
695         t0_frame, t0_sec = 0, 0.0
696     rows = _reset_trajectory_rows(points, t0_frame)
697     reset_gts = _reset_labels(gts, t0_sec)
698
699     # --- (d2) negative-label guard BEFORE any write (red-team #3a)
-----
700     # A reset label that starts < 0 means the label timeline predates the earliest
trajectory

```

```

701      # frame ? an inconsistent input that would crash the strict gt_loader on replay
(negative
702      # start). Refuse loudly up-front so the partial-write path is never even reached.
703      neg = [(s, e) for (s, e) in reset_gts if s < 0]
704      if neg:
705          raise RefusalError(
706              f"REFUSED: label timeline predates the trajectory ? {len(neg)} reset
label(s) have a "
707              f"negative start after the t0={t0_sec:.6f}s time-origin reset (e.g.
{neg[0]!r}). The "
708              f"earliest label must not precede the earliest trajectory frame ($4c time-
origin reset).")
709
710      iou = 0.5
711      # NOTE (red-team #1b): operator free-text `source_note` is DELIBERATELY NOT a key in
`prov` ?
712      # it never reaches the committed bytes (an identifying note is an attribution back-
channel
713      # _scan_forbidden cannot fully police). The fixed boilerplate `note` constant below
stays.
714      prov = {
715          "slug": out_slug,
716          "schema_version": CURRENT_SCHEMA,
717          "kind": "cleared_real",
718          "label": str(label),
719          "license": license,
720          "minors_free": True,
721          "owned": True,
722          "fps": _round(fps),
723          "frame_width": _round(frame_width),
724          "iou": _round(iou),
725          "time_origin_reset": bool(time_origin_reset),
726          "windowing": {
727              "vt": PINNED_VT,
728              "mg": PINNED_MERGE_GAP,
729              "mwd": PINNED_MIN_WINDOW,
730              "max_win": PINNED_MAX_WINDOW,
731          },
732          "paths": {
733              "trajectory": "trajectory.csv",
734              "labels": "labels.csv",
735              "expected": "expected.json",
736          },
737          "gt_hash": gt_hash(reset_gts),
738          "note": (
739              "De-attributed REAL Fork-A fixture (owner-recorded, owned, minors-excluded,
biometric-"
740              "free, shuttle-only). Opaque RANDOM surrogate slug; absolute timeline reset.
NON-"
741              "attributable only relative to the UNPUBLISHED source (EDPS v. SRB) ? never
'anonymous'. "
742              "Tier-0 / CONTROL-only. S6: behaviour-locking, not correctness; sits behind
counsel (P2).",
743          ),
744      }
745
746      # --- (e) write into a TEMP dir, replay, scan, and only THEN promote (red-team #3b)
-----
747      # Everything is written under a sibling temp dir; the final slug dir is created by
an atomic
748      # os.replace ONLY after the privacy scan passes. A refused/failed freeze therefore
leaves NO
749      # partial fixture dir behind. source_note (if any) is scanned defensively but never
written.
750      fdir = fixtures_dir / out_slug

```

```

751     fixtures_dir.mkdir(parents=True, exist_ok=True)
752     if fdir.exists():
753         raise ValueError(f"{fdir}: fixture dir already exists ? refuse to clobber
(chOOSE a new slug)")
754     tmp_parent = Path(tempfile.mkdtemp(prefix=f"{out_slug}.tmp.",
dir=str(fixtures_dir)))
755     tmp_dir = tmp_parent / out_slug
756     tmp_dir.mkdir(parents=True, exist_ok=True)
757     try:
758         write_trajectory_csv(rows, tmp_dir / "trajectory.csv")
759         write_labels_csv(reset_gts, tmp_dir / "labels.csv")
760         _write_json(tmp_dir / "provenance.json", prov)
761
762         expected = replay_fixture(tmp_dir) # REUSE M1 ? correct-by-construction
snapshot
763         _write_json(tmp_dir / "expected.json", expected)
764
765         # --- (f) POSITIVE privacy assertion BEFORE promotion (scan written text +
source_note) ---
766         source_paths = [str(traj_path), str(labels_path)]
767         _scan_forbidden((tmp_dir / "expected.json").read_text(encoding="utf-8"),
768                         where=f"{out_slug}/expected.json", source_paths=source_paths)
769         _scan_forbidden((tmp_dir / "provenance.json").read_text(encoding="utf-8"),
770                         where=f"{out_slug}/provenance.json", source_paths=source_paths)
771         # source_note is never persisted, but a leak in it is still a contract violation
? scan it.
772         if source_note:
773             _scan_forbidden(str(source_note),
774                             where=f"{out_slug}/source_note(not-persisted)",
source_paths=source_paths)
775
776         # --- (g) PROMOTE atomically: temp slug dir ? final slug dir (scan passed)
-----
777         os.replace(str(tmp_dir), str(fdir))
778     finally:
779         # Clean up the temp parent whether we promoted or refused (a refusal leaves NO
partial dir).
780         shutil.rmtree(str(tmp_parent), ignore_errors=True)
781
782         # --- (h) append to the manifest with kind="cleared_real" (don't clobber synthetic)
-----
783         _append_manifest_entry(prov, fixtures_dir)
784     return expected
785
786
787 def _manifest_entry_for(prov: Dict[str, Any]) -> Dict[str, Any]:
788     """A manifest row for a single fixture, derived from its provenance block."""
789     slug = prov["slug"]
790     return {
791         "slug": slug,
792         "fps": _round(float(prov["fps"])),
793         "frame_width": _round(float(prov["frame_width"])),
794         "windowing": dict(prov.get("windowing", {})),
795         "kind": prov.get("kind", "synthetic"),
796         "schema_version": int(prov.get("schema_version", CURRENT_SCHEMA)),
797         "paths": {
798             "dir": slug,
799             "trajectory": f"{slug}/trajectory.csv",
800             "labels": f"{slug}/labels.csv",
801             "expected": f"{slug}/expected.json",
802             "provenance": f"{slug}/provenance.json",
803         },
804     }
805
806

```



```

807     def _append_manifest_entry(prov: Dict[str, Any], fixtures_dir: Path) -> None:
808         """Add/replace one fixture's manifest row WITHOUT clobbering the others (e.g.
synthetic)."""
809         entries = load_manifest(fixtures_dir)
810         entries = [e for e in entries if e.get("slug") != prov["slug"]]
811         entries.append(_manifest_entry_for(prov))
812         entries.sort(key=lambda e: str(e.get("slug")))
813         _write_json(fixtures_dir / "manifest.json", entries)
814
815
816     # =====
817     # Manifest + loaders
818     # =====
819
820
821     def _write_manifest(specs: Sequence[FixtureSpec], fixtures_dir: Path) -> None:
822         entries = [{
823             "slug": spec.slug,
824             "fps": _round(spec.fps),
825             "frame_width": _round(spec.frame_width),
826             "windowing": {
827                 "vt": PINNED_VT, "mg": PINNED_MERGE_GAP,
828                 "mwd": PINNED_MIN_WINDOW, "max_win": PINNED_MAX_WINDOW,
829             },
830             "kind": "synthetic",
831             "schema_version": CURRENT_SCHEMA,
832             "paths": {
833                 "dir": spec.slug,
834                 "trajectory": f"{spec.slug}/trajectory.csv",
835                 "labels": f"{spec.slug}/labels.csv",
836                 "expected": f"{spec.slug}/expected.json",
837                 "provenance": f"{spec.slug}/provenance.json",
838             },
839         } for spec in specs]
840         _write_json(fixtures_dir / "manifest.json", entries)
841
842
843     def load_manifest(fixtures_dir: Path = FIXTURES_DIR) -> List[Dict[str, Any]]:
844         """Load the fixtures manifest (returns [] if absent)."""
845         path = fixtures_dir / "manifest.json"
846         if not path.is_file():
847             return []
848         data = _read_json(path)
849         if not isinstance(data, list): # pragma: no cover - corrupt manifest
850             raise ValueError(f"{path}: manifest must be a JSON array")
851         return data
852
853
854     def load_fixture(slug: str, fixtures_dir: Path = FIXTURES_DIR) -> Dict[str, Any]:
855         """Load one fixture's committed artifacts: ``{provenance, expected, dir}``. Refuses
a fixture
856         whose ``schema_version`` exceeds ``CURRENT_SCHEMA`` (forward-incompatible) on EITHER
the
857         ``expected`` OR the ``provenance`` block ? they are guarded INDEPENDENTLY so a
forward-version
858         bump in one is never masked by an in-range value in the other (red-team #7)."""
859         fdir = fixtures_dir / slug
860         prov = _read_json(fdir / "provenance.json")
861         expected = _read_json(fdir / "expected.json")
862         for which, blob in (("expected", expected), ("provenance", prov)):
863             sv = int(blob.get("schema_version", 0))
864             if sv > CURRENT_SCHEMA:
865                 raise ValueError(
866                     f"{slug}: {which} schema_version {sv} > CURRENT_SCHEMA {CURRENT_SCHEMA}
? "

```

```

867         f"upgrade the harness")
868     return {"slug": slug, "dir": fdir, "provenance": prov, "expected": expected}
869
870
871     # =====
872     # Parse-compare (NEVER byte-compare): exact ints/structure, abs-tol floats
873     # =====
874
875
876     def compare_expected(recomputed: Dict[str, Any], committed: Dict[str, Any]) ->
List[str]:
877         """Return a list of human-readable mismatch messages (empty == match).
878
879         Parse-compare per HARD CONSTRAINT 2: structural/int fields (candidate count,
net_crossings,
880         TP/FP/FN, segment count, num_pred/num_gt, slug, schema_version, gt_hash) must be
EXACTLY
881         equal; float fields compared with abs tolerance 1e-6. Never a byte compare ?
CRLF/float-format
882         safe (the repo runs with core.autocrlf=true).
883
884         Key-set equality is enforced on BOTH the top-level dict AND the ``score`` block
(mirroring the
885         per-candidate ``shuttle`` check) so an EXTRA key on one side is a mismatch, not a
silent pass ?
886         a fixed-field loop alone would ignore an added field (red-team #5)."""
887         out: List[str] = []
888
889         def exact(path: str, a: Any, b: Any) -> None:
890             if a != b:
891                 out.append(f"{path}: {a!r} != {b!r} (exact)")
892
893         def approx(path: str, a: Any, b: Any) -> None:
894             try:
895                 if abs(float(a) - float(b)) > _FLOAT_TOL:
896                     out.append(f"{path}: {a!r} != {b!r} (|?| > {_FLOAT_TOL})")
897             except (TypeError, ValueError):
898                 out.append(f"{path}: non-numeric float field {a!r} vs {b!r}")
899
900         # Top-level key-set equality ? an extra/missing top-level key is a mismatch (red-
team #5).
901         if set(recomputed) != set(committed):
902             out.append(f"(top-level): keys {sorted(recomputed)} != {sorted(committed)}
(exact)")
903
904         exact("slug", recomputed.get("slug"), committed.get("slug"))
905         exact("schema_version", recomputed.get("schema_version"),
committed.get("schema_version"))
906         exact("gt_hash", recomputed.get("gt_hash"), committed.get("gt_hash"))
907         approx("iou", recomputed.get("iou"), committed.get("iou"))
908
909         rc, cc = recomputed.get("candidates", []), committed.get("candidates", [])
910         if len(rc) != len(cc):
911             out.append(f"candidates: count {len(rc)} != {len(cc)} (exact)")
912         else:
913             for i, (r, c) in enumerate(zip(rc, cc)):
914                 approx(f"candidates[{i}].start", r.get("start"), c.get("start"))
915                 approx(f"candidates[{i}].end", r.get("end"), c.get("end"))
916                 rs, cs = r.get("shuttle", {}), c.get("shuttle", {})
917                 if set(rs) != set(cs):
918                     out.append(f"candidates[{i}].shuttle: keys {sorted(rs)} != {sorted(cs)}
(exact)")
919             else:
920                 for k in rs:
921                     if k == "net_crossings":

```

```

922             exact(f"candidates[{i}].shuttle.net_crossings", rs[k], cs[k])
923         else:
924             approx(f"candidates[{i}].shuttle.{k}", rs[k], cs[k])
925
926     rseg, cseg = recomputed.get("control_segments", []),
committed.get("control_segments", [])
927     if len(rseg) != len(cseg):
928         out.append(f"control_segments: count {len(rseg)} != {len(cseg)} (exact)")
929     else:
930         for i, (r, c) in enumerate(zip(rseg, cseg)):
931             approx(f"control_segments[{i}][0]", r[0], c[0])
932             approx(f"control_segments[{i}][1]", r[1], c[1])
933
934     rsc, csc = recomputed.get("score", {}), committed.get("score", {})
935     # Score-block key-set equality ? an extra/missing score key is a mismatch (red-team
#5).
936     if set(rsc) != set(csc):
937         out.append(f"score: keys {sorted(rsc)} != {sorted(csc)} (exact)")
938     int_fields = ("num_pred", "num_gt", "true_positives", "false_positives",
"false_negatives")
939     float_fields = ("iou_threshold", "precision", "recall", "f1", "mean_iou",
940                     "mean_start_error", "mean_end_error")
941     for k in int_fields:
942         exact(f"score.{k}", rsc.get(k), csc.get(k))
943     for k in float_fields:
944         approx(f"score.{k}", rsc.get(k), csc.get(k))
945
946     return out
947
948
949 # =====
950 # Small JSON / path helpers (deterministic writers: sort_keys, LF, trailing newline)
951 # =====
952
953
954 def _write_json(path: Path, obj: Any) -> None:
955     text = json.dumps(obj, indent=2, sort_keys=True, ensure_ascii=False)
956     path.write_text(text + "\n", encoding="utf-8", newline="\n")
957
958
959 def _read_json(path: Path) -> Any:
960     with open(path, encoding="utf-8") as f:
961         return json.load(f)
962
963
964 def _resolve_dir(slug_or_dir: Any) -> Path:
965     if isinstance(slug_or_dir, Path):
966         return slug_or_dir
967     p = Path(str(slug_or_dir))
968     if p.is_dir():
969         return p
970     return FIXTURES_DIR / str(slug_or_dir)
971
972
973 def _fps_fw(prov: Optional[Dict[str, Any]], slug: str) -> Tuple[float, float]:
974     if prov and "fps" in prov and "frame_width" in prov:
975         return float(prov["fps"]), float(prov["frame_width"])
976     spec = _spec_by_slug().get(slug)
977     if spec is not None:
978         return spec.fps, spec.frame_width
979     raise ValueError(f"{slug}: no fps/frame_width in provenance and not a built-in
spec")
980
981
982 def _fps_fw_from_prov(fdir: Path, slug: str) -> Tuple[float, float]:

```

```

983     prov_path = fdir / "provenance.json"
984     prov = _read_json(prov_path) if prov_path.is_file() else None
985     return _fps_fw(prov, slug)
986
987
988     # =====
989     # CLI
990     # =====
991
992
993     def _cmd_check(fixtures_dir: Path) -> int:
994         manifest = load_manifest(fixtures_dir)
995         if not manifest:
996             print("[golden-fixtures] no fixtures found ? run --freeze-synthetic first",
997                   file=sys.stderr)
998             return 1
999         failures = 0
1000         for entry in manifest:
1001             slug = entry["slug"]
1002             committed = _read_json(fixtures_dir / slug / "expected.json")
1003             recomputed = replay_fixture(fixtures_dir / slug)
1004             mismatches = compare_expected(recomputed, committed)
1005             if mismatches:
1006                 failures += 1
1007                 print(f"[FAIL] {slug}: {len(mismatches)} mismatch(es) "
1008                       f"(characterization = behaviour-locking, NOT correctness ? $6):")
1009                 for m in mismatches:
1010                     print(f"    - {m}")
1011             else:
1012                 sc = recomputed["score"]
1013                 print(f"[ok] {slug}: {len(recomputed['candidates'])} candidates, "
1014                       f"{len(recomputed['control_segments'])} segments, F1={sc['f1']:.6f}")
1015             print(f"[golden-fixtures] {len(manifest) - failures}/{len(manifest)} fixtures match "
1016                   f"(synthetic-tier = plumbing correctness, not field quality ? $6)")
1017         return 0 if failures == 0 else 1
1018
1019     def main(argv: Optional[Sequence[str]] = None) -> int:
1020         ap = argparse.ArgumentParser(
1021             description="Golden Regression Fixtures (M1) ? synthetic, CONTROL-only replay
1022             harness.")
1023         g = ap.add_mutually_exclusive_group(required=True)
1024         g.add_argument("--freeze-synthetic", action="store_true",
1025                        help="generate synthetic trajectory+labels for all built-in
1026                        scenarios, then "
1027                        "freeze expected.json+provenance.json + write manifest.json")
1028         g.add_argument("--refresh-snapshot", action="store_true",
1029                        help="re-freeze expected.json from COMMITTED trajectory+labels (no
1030                        regeneration)")
1031         g.add_argument("--check", action="store_true",
1032                        help="replay all fixtures + parse-compare vs committed expected.json
1033                        (exit 0/1)")
1034         g.add_argument("--freeze-real", action="store_true",
1035                        help="(P1) de-attribute an OWNED real trajectory+labels into a
1036                        committable Tier-0 "
1037                        "fixture; REFUSED without --owned --minors-free and a non-blank
1038                        --license")
1039         ap.add_argument("--slug", help="single slug for --refresh-snapshot, or the surrogate
1040                        slug for "
1041                        "--freeze-real (default: a random fx_<hex> id)")
1042         ap.add_argument("--all", action="store_true", help="all slugs for --refresh-
1043                        snapshot")
1044         ap.add_argument("--reason", help="REQUIRED recorded reason for --refresh-snapshot
1045                        (review the diff!)")

```

```

1037     # --freeze-real (P1 / M2 refusal gate) options:
1038     ap.add_argument("--trajectory", help="--freeze-real) source trajectory CSV
(Frame,Visibility,X,Y)")
1039     ap.add_argument("--labels", help="--freeze-real) source golden labels CSV
(start,end)")
1040     ap.add_argument("--license", dest="license_", default=None,
1041                     help="--freeze-real) the source license tag (required, non-blank)")
1042     ap.add_argument("--owned", action="store_true",
1043                     help="--freeze-real) assert owner-recorded Fork-A source
(required)")
1044     ap.add_argument("--minors-free", dest="minors_free", action="store_true",
1045                     help="--freeze-real) assert the source contains NO minors
(required)")
1046     ap.add_argument("--fps", type=float, help="--freeze-real) source fps")
1047     ap.add_argument("--frame-width", dest="frame_width", type=float, help="--freeze-
real) source frame width (px)")
1048     ap.add_argument("--label", default="cleared_real",
1049                     help="--freeze-real) non-identifying scenario label ? STRICT slug
only "
1050                     "(lowercase alnum + '_'/'-', no spaces/punctuation/names)")
1051     ap.add_argument("--source-note", dest="source_note", default="",
1052                     help="--freeze-real) ACCEPTED BUT NOT PERSISTED ? operator free-
text is an "
1053                     "attribution back-channel, so it never reaches the committed
provenance "
1054                     "(scanned defensively, then dropped). Retained for CLI
compatibility.")
1055     ap.add_argument("--no-time-origin-reset", dest="time_origin_reset",
action="store_false",
1056                     help="--freeze-real) DISABLE the anti-attribution time-origin reset
(default ON)")
1057     ap.set_defaults(time_origin_reset=True)
1058     ap.add_argument("--fixtures-dir", type=Path, default=FIXTURES_DIR,
1059                     help="override the fixtures directory (default: repo
eval_baselines/fixtures)")
1060     args = ap.parse_args(list(argv) if argv is not None else None)
1061
1062     fixtures_dir: Path = args.fixtures_dir
1063
1064     if args.freeze_synthetic:
1065         slugs = freeze_synthetic(fixtures_dir)
1066         print(f"[golden-fixtures] froze {len(slugs)} synthetic fixture(s): {'',
''.join(slugs)}")
1067         print(f" -> {fixtures_dir}")
1068         return 0
1069
1070     if args.refresh_snapshot:
1071         if not args.slug and not args.all:
1072             ap.error("--refresh-snapshot requires --slug S or --all")
1073         # --reason is MANDATORY (red-team #6): --refresh-snapshot can rubber-stamp a
real
1074         # regression, so the deliberate, recorded reason is a required arg, not a soft
warning.
1075         if not (args.reason and args.reason.strip()):
1076             ap.error("--refresh-snapshot requires --reason \"<why>\" ? it can rubber-
stamp a real "
1077                     "regression; record the deliberate reason and review the visible
diff ($6).")
1078         targets = refresh_snapshot(slug=args.slug, all_slugs=args.all,
fixtures_dir=fixtures_dir)
1079         print(f"[golden-fixtures] refreshed snapshot for: {'', ''.join(targets)}"
f" (reason: {args.reason})")
1080         return 0
1081
1082     if args.freeze_real:

```

```

1084         missing = [n for n, v in (("--trajectory", args.trajectory),
1085                                   ("--labels", args.labels),
1086                                   ("--fps", args.fps),
1087                                   ("--frame-width", args.frame_width)) if v is None]
1088     if missing:
1089         ap.error("--freeze-real requires " + ", ".join(missing))
1090     try:
1091         expected = freeze_real_fixture(
1092             args.label, args.trajectory, args.labels,
1093             license=args.license_ if args.license_ is not None else "",
1094             minors_free=args.minors_free, owned=args.owned,
1095             fps=args.fps, frame_width=args.frame_width,
1096             fixtures_dir=fixtures_dir, slug=args.slug,
1097             time_origin_reset=args.time_origin_reset, source_note=args.source_note)
1098     except RefusalError as e:
1099         # The code-enforced provenance gate ($4c / $7 ?): refuse loudly, exit non-
1100         zero.
1101         print(f"[golden-fixtures] REFUSED (--freeze-real provenance gate):\n{e}",
1102               file=sys.stderr)
1103         return 2
1104     print(f"[golden-fixtures] froze de-attributed REAL fixture '{expected['slug']}'")
1105     f"(kind=cleared_real) -> {fixtures_dir / expected['slug']}")
1106     print(" $6: behaviour-locking, NOT correctness; non-attributable only relative
1107 to the "
1108         "UNPUBLISHED source ? never 'anonymous'. Sits behind counsel sign-off
1109 (P2).")
1110     return 0
1111
1112 # --check
1113 return _cmd_check(fixtures_dir)
1114
1115 if __name__ == "__main__":
1116     raise SystemExit(main())

```

21. assistant (2026-06-18T01:37:43.660Z)

These are extremely carefully written with red-team annotations. Let me look at the validators, which are error paths run during ingestion, and the audio/hit detector. Let me also check `metrics.py` and `tolerance\_metrics.py` for off-by-one or division issues.

22. user (2026-06-18T01:37:45.216Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit
4     tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5     start < end. We score a list of *predicted* rally intervals against a list of
6     *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7     """
8
9     from dataclasses import dataclass, field
10    from typing import List, Tuple, Optional
11
12    Interval = Tuple[float, float]
13
14
15    def interval_iou(a: Interval, b: Interval) -> float:
16        """Temporal intersection-over-union of two intervals.
17
18        Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19        overlap / union, both measured as durations on the timeline.
20        """

```

```

21     a_start, a_end = a
22     b_start, b_end = b
23     if a_end <= a_start or b_end <= b_start:
24         return 0.0
25     inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26     if inter <= 0.0:
27         return 0.0
28     union = (a_end - a_start) + (b_end - b_start) - inter
29     return inter / union if union > 0 else 0.0
30
31
32 @dataclass
33 class Match:
34     """One matched predicted?ground-truth pair."""
35     pred_index: int
36     gt_index: int
37     iou: float
38
39
40 @dataclass
41 class MatchResult:
42     """Outcome of matching predictions to ground truth at a given IoU threshold."""
43     matches: List[Match] = field(default_factory=list)
44     unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45     unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives
(misses)
46
47
48 def match_intervals(preds: List[Interval], gts: List[Interval],
49                    iou_threshold: float = 0.5) -> MatchResult:
50     """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52     Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53     are claimed highest-IoU first, and each prediction/GT can be used once. This
54     keeps a single prediction from being credited for two ground-truth rallies
55     (and vice versa).
56     """
57     candidates = []
58     for pi, p in enumerate(preds):
59         for gi, g in enumerate(gts):
60             iou = interval_iou(p, g)
61             # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
would
62             # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63             if iou > 0.0 and iou >= iou_threshold:
64                 candidates.append((iou, pi, gi))
65     # Highest IoU first; ties broken by indices for determinism.
66     candidates.sort(key=lambda c: (-c[0], c[1], c[2]))
67
68     used_pred, used_gt = set(), set()
69     matches: List[Match] = []
70     for iou, pi, gi in candidates:
71         if pi in used_pred or gi in used_gt:
72             continue
73         used_pred.add(pi)
74         used_gt.add(gi)
75         matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76
77     unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78     unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79     return MatchResult(matches=matches,
80                        unmatched_pred_indices=unmatched_pred,
81                        unmatched_gt_indices=unmatched_gt)
82
83

```

```

84     @dataclass
85     class Score:
86         """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87         iou_threshold: float
88         num_pred: int
89         num_gt: int
90         true_positives: int
91         false_positives: int
92         false_negatives: int
93         precision: float
94         recall: float
95         f1: float
96         mean_iou: float          # over matched pairs (0.0 if none)
97         mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
98         mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
99
100        def as_dict(self) -> dict:
101            return self.__dict__.copy()
102
103
104    def score(preds: List[Interval], gts: List[Interval],
105            iou_threshold: float = 0.5,
106            match_result: Optional[MatchResult] = None) -> Score:
107        """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109        Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110        extracting the failure lists); otherwise matching is done here.
111        """
112        mr = match_result if match_result is not None else match_intervals(preds, gts,
113        iou_threshold)
114
115        tp = len(mr.matches)
116        fp = len(mr.unmatched_pred_indices)
117        fn = len(mr.unmatched_gt_indices)
118
119        precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
120        recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
121        f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
122        else 0.0
123
124        if mr.matches:
125            mean_iou = sum(m.iou for m in mr.matches) / tp
126            mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
127            mr.matches) / tp
128            mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
129            mr.matches) / tp
130        else:
131            mean_iou = mean_start_err = mean_end_err = 0.0
132
133        return Score(
134            iou_threshold=iou_threshold,
135            num_pred=len(preds), num_gt=len(gts),
136            true_positives=tp, false_positives=fp, false_negatives=fn,
137            precision=precision, recall=recall, f1=f1,
138            mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
139        )
140
141
142    def pr_curve(preds: List[Interval], gts: List[Interval],
143            thresholds: Optional[List[float]] = None) -> List[Score]:
144        """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
145
146        Lets you see strict-vs-lenient behaviour rather than a single operating point.
147        """
148        if thresholds is None:

```



```

145         thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146         return [score(preds, gts, iou_threshold=t) for t in thresholds]
147

```

23. user (2026-06-18T01:37:46.233Z)

```

1     import subprocess
2     from backend.pipeline.validators.base import VideoValidator, ValidationResult
3     from typing import Dict, Any
4
5     class PTSContinuityValidator(VideoValidator):
6         @property
7         def name(self) -> str:
8             return "pts_continuity_check"
9
10        @property
11        def description(self) -> str:
12            return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
13            timeline gaps, jumps, or editing splices."
14
15        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
16        -> ValidationResult:
17            # Command to extract packet PTS and flags (extremely fast, works on H.264 &
18            H.265)
19            cmd = [
20                "ffprobe",
21                "-v", "error",
22                "-select_streams", "v:0",
23                "-show_entries", "packet=pts_time,flags",
24                "-of", "csv=p=0",
25                video_path
26            ]
27
28            try:
29                result = subprocess.run(cmd, capture_output=True, text=True, check=True)
30                output = result.stdout.strip()
31
32                if not output:
33                    return ValidationResult(
34                        validator_name=self.name,
35                        passed=False,
36                        message="No packet timestamps could be extracted from the video
37                        stream."
38                    )
39
40                timestamps = []
41                for line in output.split("\n"):
42                    line = line.strip()
43                    if line:
44                        parts = line.split(",")
45                        if len(parts) == 2:
46                            pts_str, flags = parts
47                            if "K" in flags: # Check if keyframe
48                                try:
49                                    timestamps.append(float(pts_str))
50                                except ValueError:
51                                    continue
52
53                if len(timestamps) < 2:
54                    return ValidationResult(
55                        validator_name=self.name,
56                        passed=True,
57                        message="Video is too short to perform a PTS continuity check.",
58                        details={"keyframes_count": len(timestamps)}
59                    )
60

```

```

56
57     # Analyze gaps
58     max_gap = 0.0
59     backward_jumps = 0
60     gaps = []
61
62     for i in range(1, len(timestamps)):
63         gap = timestamps[i] - timestamps[i-1]
64         gaps.append(gap)
65         if gap < 0:
66             backward_jumps += 1
67         if gap > max_gap:
68             max_gap = gap
69
70     # An continuous GoPro recording usually has keyframe intervals (GOP size) of
71     1.0 to 4.0 seconds.
72     # If there's a keyframe gap larger than 8.0 seconds or a negative gap
73     (backward jump), it's highly likely spliced.
74     details = {
75         "keyframes_count": len(timestamps),
76         "max_keyframe_gap_seconds": round(max_gap, 2),
77         "backward_jumps_detected": backward_jumps,
78         "first_keyframe": timestamps[0],
79         "last_keyframe": timestamps[-1]
80     }
81
82     if backward_jumps > 0:
83         return ValidationResult(
84             validator_name=self.name,
85             passed=False,
86             message="Video integrity compromise: Backward timeline jumps
87             (splicing) detected.",
88             details=details
89         )
90
91     # Threshold for sudden gap (e.g. 10.0 seconds)
92     if max_gap > 10.0:
93         return ValidationResult(
94             validator_name=self.name,
95             passed=False,
96             message=f"Video integrity compromise: Detected a timestamp gap of
97             {round(max_gap, 2)}s, indicating edited or missing content.",
98             details=details
99         )
100
101     return ValidationResult(
102         validator_name=self.name,
103         passed=True,
104         message="PTS timeline is continuous with no abnormal gaps or backward
105         jumps.",
106         details=details
107     )
108
109 except subprocess.CalledProcessError as e:
110     return ValidationResult(
111         validator_name=self.name,
112         passed=False,
113         message="Failed to execute ffprobe PTS analysis.",
114         details={"error": e.stderr}
115     )
116
117 except Exception as e:
118     return ValidationResult(
119         validator_name=self.name,
120         passed=False,
121         message=f"Unexpected error in PTS validation: {str(e)}"

```

116)
117

24. assistant (2026-06-18T01:37:50.604Z)

These are clean. Let me look at the per_user_gate, promotion, and tolerance_metrics ? these have more complex logic. Let me also check the harness and windowing which are central.

25. user (2026-06-18T01:37:51.614Z)

```
1      """R2 ? task-faithful rally-boundary eval (tolerance-match + over-segmentation guard).
2
3      The headline rally metric (tIoU-F1 is kept as a SECONDARY diagnostic): a prediction
matches a
4      golden rally iff ``|?start| ? ?_start AND |?end| ? ?_end`` under a **strict 1:1
assignment**, so
5      over-production costs precision directly. Reported as a DECOMPOSITION ? ``P/R/F1@?`` +
start/end
6      boundary-error distributions + an over-seg guard (split/merge counts) + a ?-sweep ?
never one
7      number (a single ? misleads; see the design doc).
8
9      Locked decisions + rationale: ``docs/R2_EVAL_METRIC_DESIGN.md`` (owner-reviewed
2026-06-17):
10     asymmetric ? (``?_start=2.0`` forgives the ~1?1.5 s service window; ``?_end=1.5`` keeps
the
11     rally-end honest ? IAA-calibrated later); augment tIoU now, ablation-gate any eventual
swap.
12
13     Pure + stdlib; **numpy/scipy are OPTIONAL** ? used for the exact Hungarian assignment
when present,
14     else a deterministic greedy 1:1 fallback. Both agree on the temporally-ordered, near-
diagonal
15     eligibility graphs that rally intervals produce, so scores are environment-independent.
16     """
17     from __future__ import annotations
18
19     import statistics
20     from typing import Any, Dict, List, Optional, Tuple
21
22     Interval = Tuple[float, float]
23     #: (pred_idx, gt_idx, dstart_signed, dend_signed) ? signed = pred ? gt (positive = pred
is late).
24     Match = Tuple[int, int, float, float]
25
26     #: Provisional ? (the design's locked starting point; fixed by the IAA study later).
Always
27     #: report the sweep alongside, since the golden END convention carries >1.5 s of noise.
28     TAU_START: float = 2.0
29     TAU_END: float = 1.5
30     SWEEP_TAUS: Tuple[float, ...] = (1.5, 2.0, 2.5, 3.0)
31
32
33     def _eligible(p: Interval, g: Interval, tau_start: float, tau_end: float) -> bool:
34         return abs(p[0] - g[0]) <= tau_start and abs(p[1] - g[1]) <= tau_end
35
36
37     def _greedy_match_1to1(preds: List[Interval], gts: List[Interval],
38                           tau_start: float, tau_end: float) -> List[Match]:
39         """Deterministic greedy 1:1: eligible (pred, gt) pairs in ascending combined
boundary error,
40         assigned first-come and skipping used indices. Optimal-or-near on the near-diagonal
eligibility
41         graph rally intervals produce. The numpy/scipy-free path (and the CI path)."""
42         pairs: List[Tuple[float, float, int, int]] = []
```

```

43     for i, p in enumerate(preds):
44         for j, g in enumerate(gts):
45             if _eligible(p, g, tau_start, tau_end):
46                 # sort key: combined error, then (i, j) for a fully deterministic
47                 pairs.append((abs(p[0] - g[0]) + abs(p[1] - g[1]), float(i * 1e-6 + j *
1e-9), i, j))
48     pairs.sort()
49     used_p: set = set()
50     used_g: set = set()
51     matches: List[Match] = []
52     for _, _, i, j in pairs:
53         if i in used_p or j in used_g:
54             continue
55         used_p.add(i)
56         used_g.add(j)
57         matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
58     matches.sort(key=lambda mm: mm[1])
59     return matches
60
61
62 def _hungarian_match_1to1(preds: List[Interval], gts: List[Interval],
63                          tau_start: float, tau_end: float) -> Optional[List[Match]]:
64     """Exact max-cardinality (then min total boundary error) via scipy. Returns ``None``
when
65     numpy/scipy are unavailable so the caller falls back to greedy."""
66     try:
67         import numpy as np
68         from scipy.optimize import linear_sum_assignment
69     except Exception:
70         return None
71     n, m = len(preds), len(gts)
72     size = max(n, m)
73     big = 1.0e6
74     cost = np.full((size, size), big, dtype=float)
75     for i, p in enumerate(preds):
76         for j, g in enumerate(gts):
77             if _eligible(p, g, tau_start, tau_end):
78                 cost[i, j] = abs(p[0] - g[0]) + abs(p[1] - g[1])
79     rows, cols = linear_sum_assignment(cost)
80     matches: List[Match] = []
81     for i, j in zip(rows.tolist(), cols.tolist()):
82         if i < n and j < m and cost[i, j] < big: # drop forced ineligible assignments
83             matches.append((i, j, preds[i][0] - gts[j][0], preds[i][1] - gts[j][1]))
84     matches.sort(key=lambda mm: mm[1])
85     return matches
86
87
88 def match_1to1(preds: List[Interval], gts: List[Interval],
89               tau_start: float = TAU_START, tau_end: float = TAU_END
90               ) -> Tuple[List[Match], float, float, float]:
91     """Strict 1:1 tolerance match ? ``(matches, precision, recall, f1)``. ``P = matched
/ n_pred``
92     (over-production costs precision), ``R = matched / n_gt``. Exact Hungarian when
scipy is
93     present; deterministic greedy otherwise."""
94     n, m = len(preds), len(gts)
95     if n == 0 or m == 0:
96         return [], 0.0, 0.0, 0.0
97     matches = _hungarian_match_1to1(preds, gts, tau_start, tau_end)
98     if matches is None:
99         matches = _greedy_match_1to1(preds, gts, tau_start, tau_end)
100     k = len(matches)
101     precision = k / n
102     recall = k / m

```

```

103         f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else
104         0.0
105     return matches, precision, recall, f1
106
107 def _percentile(xs: List[float], p: float) -> float:
108     """Linear-interpolated percentile (pure stdlib)."""
109     if not xs:
110         return 0.0
111     s = sorted(xs)
112     k = (len(s) - 1) * p / 100.0
113     lo = int(k)
114     hi = min(lo + 1, len(s) - 1)
115     return s[lo] + (s[hi] - s[lo]) * (k - lo)
116
117
118 def _overlap(a: Interval, b: Interval) -> float:
119     return max(0.0, min(a[1], b[1]) - max(a[0], b[0]))
120
121
122 def boundary_error_report(preds: List[Interval], gts: List[Interval]) -> Dict[str,
Dict[str, float]]:
123     """Median + IQR of signed AND absolute ?start, ?end, split start/end ? the "start
solved,
124     end open" diagnostic that aims R4.
125
126     Computed over **best-overlap** matches (each GT ? its maximally-overlapping
prediction, any
127     positive overlap), NOT the within-? 1:1 matches: a ?-gated set is survivorship-
biased (loose
128     ends fail the gate and vanish from the distribution), which would HIDE the very end-
deficit
129     this diagnostic exists to surface. Unconditional best-overlap is what the day-1
validation used
130     to find |?end| ? |?start|. (The within-? deficit shows up instead as low
``tol_recall``.)"""
131     def dist(vals: List[float]) -> Dict[str, float]:
132         av = [abs(v) for v in vals]
133         return {
134             "signed_median": statistics.median(vals) if vals else 0.0,
135             "abs_median": statistics.median(av) if av else 0.0,
136             "abs_p25": _percentile(av, 25.0),
137             "abs_p75": _percentile(av, 75.0),
138             "n": float(len(vals)),
139         }
140     ds: List[float] = []
141     de: List[float] = []
142     for g in gts:
143         best = max(preds, key=lambda p: _overlap(p, g), default=None)
144         if best is not None and _overlap(best, g) > 0.0:
145             ds.append(best[0] - g[0])
146             de.append(best[1] - g[1])
147     return {"dstart": dist(ds), "dend": dist(de)}
148
149
150 def over_seg_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, int]:
151     """The over-segmentation GUARD (per-video no-regression in promotion):
``split_count`` (?2
152     preds covering one rally) + ``merge_count`` (one pred ? ?2 rallies). Reuses the
bipartite
153     overlap-graph in ``segmentation_metrics.merge_split_report``.
154     from backend.eval.segmentation_metrics import merge_split_report
155     r = merge_split_report(preds, gts)
156     return {"split_count": int(r["splits"]), "merge_count": int(r["merges"])}
157

```

```

158
159 # ----- #
160 # R1 ? net over-segmentation promotion guard
161 # ----- #
162 #: Default over-seg cost weights. A MERGE hides a rally (?2 golden rallies collapse into
one
163 #: predicted window ? the user can't reach the hidden one ? recall loss at rally
granularity); a
164 #: SPLIT only fragments one rally (it is still found, just in pieces). So a merge is the
strictly
165 #: worse fault and is weighted higher. (Equal weights also pass the milestone ? see the
R1 evidence;
166 #: the asymmetry is the principled default, not a number tuned to force a verdict.)
167 WEIGHT_MERGE: float = 2.0
168 WEIGHT_SPLIT: float = 1.0
169
170
171 def over_seg_guard(base: List[Dict[str, Any]], cand: List[Dict[str, Any]], *,
172                 weight_merge: float = WEIGHT_MERGE, weight_split: float =
WEIGHT_SPLIT,
173                 use_rates: bool = True, eps: float = 1e-9) -> Dict[str, Any]:
174     """**Net** over-segmentation promotion guard (the R1 refinement of the strict per-
video rule).
175
176     ``base``/``cand`` are aligned per-video over-seg dicts (the rows ``rally_seg_eval``
already
177     emits): each needs ``split_count``/``merge_count`` and ? when ``use_rates``
(default) ?
178     ``merge_rate``/``split_rate``. Optional ``name`` keys align the two lists by video
(else by
179     position). Returns the promote-or-block verdict for promoting ``cand`` over
``base``.
180
181     **Why the strict rule needed refining.** R2 §2.3.3's guard blocks promotion if
``cand`` raises
182     ``split_count`` OR ``merge_count`` on ANY video. That is right for an *incremental*
cue (same
183     windowing structure, an increase = a real regression) but mis-fires on a
*structural* change
184     (mega-windows ? bounded): splitting a single mega-window necessarily lifts
``split_count`` from
185     ~0, and the raw merge **count** is non-monotonic ? one mega-window swallowing 40
rallies is
186     ``merge_count=1`` yet ``merge_rate=1.0`` (every rally hidden), whereas bounding it
into pieces
187     that each glue 2 neighbours is ``merge_count=9`` yet ``merge_rate=0.5`` (FEWER
rallies hidden).
188     So the strict count rule blocks a config that strictly *reduces* the fraction of
rallies lost to
189     over-merge. (Measured: the cap6+R4 milestone trips strict on 10/11 videos yet cuts
mean
190     ``merge_rate`` 0.694 ? 0.150 with NO per-video merge_rate regression ?
RALLY_QUALITY_RESEARCH §6.)
191
192     **The net rule (default verdict ``passes``):** score over-seg as ONE weighted cost
per video and
193     require BOTH:
194     1. the corpus-mean net cost does not rise (``cand_net ? base_net + eps``), AND
195     2. NO video's **merge_rate** rises beyond ``eps`` ? reintroducing a mega-window
that hides MORE
196     of a video's rallies is the one over-merge regression that is never acceptable,
the honest
197     "no NEW merges" rule expressed on the *rate* (the recall-meaningful quantity),
not the count.
198     With ``use_rates`` the per-video cost uses ``merge_rate``/``split_rate`` (fraction

```

```

of GT rallies
199         affected, ? [0,1] ? comparable across a structural change); ``use_rates=False``
scores raw counts.
200
201     **Revert (one line):** read ``strict_passes`` instead of ``passes`` to fall back to
the original
202     strict per-video count rule. Both verdicts are always reported, so the choice is
auditable.
203     """
204     def _aligned() -> List[Tuple[Dict[str, Any], Dict[str, Any]]]:
205         if base and cand and all("name" in b for b in base) and all("name" in c for c in
cand):
206             cmap = {c["name"]: c for c in cand}
207             return [(b, cmap[b["name"]]) for b in base if b["name"] in cmap]
208             n = min(len(base), len(cand))
209             return [(base[i], cand[i]) for i in range(n)]
210
211     def _cost(row: Dict[str, Any]) -> float:
212         if use_rates:
213             return weight_merge * float(row["merge_rate"]) + weight_split *
float(row["split_rate"])
214             return weight_merge * float(row["merge_count"]) + weight_split *
float(row["split_count"])
215
216     pairs = _aligned()
217     n = len(pairs)
218     base_costs = [_cost(b) for b, _ in pairs]
219     cand_costs = [_cost(c) for _, c in pairs]
220     base_net = (sum(base_costs) / n) if n else 0.0
221     cand_net = (sum(cand_costs) / n) if n else 0.0
222     net_delta = cand_net - base_net
223
224     # Per-video regressions, reported for transparency.
225     new_merges: List[Dict[str, Any]] = []           # raw COUNT increases (what strict
trips on)
226     new_splits: List[Dict[str, Any]] = []          # raw COUNT increases (what strict
trips on)
227     merge_rate_regress: List[Dict[str, Any]] = []  # RATE increases (the real over-
merge regression)
228     for b, c in pairs:
229         if float(c["merge_count"]) > float(b["merge_count"]) + eps:
230             new_merges.append({"name": b.get("name", ""),
231                               "base": float(b["merge_count"]), "cand":
float(c["merge_count"])})
232         if float(c["split_count"]) > float(b["split_count"]) + eps:
233             new_splits.append({"name": b.get("name", ""),
234                               "base": float(b["split_count"]), "cand":
float(c["split_count"])})
235         if use_rates and float(c["merge_rate"]) > float(b["merge_rate"]) + eps:
236             merge_rate_regress.append({"name": b.get("name", ""),
237                                       "base": float(b["merge_rate"]), "cand":
float(c["merge_rate"])})
238
239     strict_passes = (not new_merges) and (not new_splits)
240     net_cost_ok = net_delta <= eps
241     no_merge_rate_regress = not merge_rate_regress
242     passes = bool(net_cost_ok and no_merge_rate_regress)
243     return {
244         "passes": passes,
245         "strict_passes": strict_passes,
246         "net_cost_ok": net_cost_ok,
247         "no_merge_rate_regress": no_merge_rate_regress,
248         "base_net": base_net,
249         "cand_net": cand_net,
250         "net_delta": net_delta,

```

```

251         "n": n,
252         "use_rates": use_rates,
253         "weight_merge": weight_merge,
254         "weight_split": weight_split,
255         "new_merges": new_merges,
256         "new_splits": new_splits,
257         "merge_rate_regress": merge_rate_regress,
258     }
259
260
261     def tolerance_report(preds: List[Interval], gts: List[Interval],
262                         tau_start: float = TAU_START, tau_end: float = TAU_END) ->
Dict[str, object]:
263         """The full R2 block for one video: P/R/F1@? + boundary-error distributions + over-
seg guard."""
264         matches, precision, recall, f1 = match_1to1(preds, gts, tau_start, tau_end)
265         return {
266             "tau_start": tau_start, "tau_end": tau_end,
267             "n_pred": len(preds), "n_gt": len(gts), "matched": len(matches),
268             "precision": precision, "recall": recall, "f1": f1,
269             "boundary_error": boundary_error_report(preds, gts),
270             "over_seg": over_seg_report(preds, gts),
271         }
272
273
274     def tolerance_sweep(preds: List[Interval], gts: List[Interval],
275                        taus: Tuple[float, ...] = SWEEP_TAUS) -> Dict[float, float]:
276         """Symmetric-? F1 sweep ? the trend curve (a single ? misleads). Monotonic non-
decreasing in ?."""
277         return {t: match_1to1(preds, gts, t, t)[3] for t in taus}
278

```

26. user (2026-06-18T01:37:52.181Z)

```

1     """Per-user promotion gate ? the safety rail for flag-gated personalization.
2
3     The owner adopted personalization as a flag-gated feature on a generalization-
first
4     baseline: a cue/variant B may be flipped ON for one user only when that user's own
5     held-out evidence honestly clears the same small-N bar the global eval uses ? otherwise
we
6     fall back to the shipped baseline (variant A). This module is the decision logic for
that
7     flip; it is pure (no I/O) and reuses ``backend.eval.eval_stats`` for every statistic
8     (``paired_delta_ci`` + ``paired_sign_test``) so the math is never reinvented.
9
10    The counter-case this guards against (and its mitigations, implemented here):
11
12    1. Minimum-N floor. Personalizing off 1-2 of a user's videos is the small-N-lies
regime
13        (see ``eval_stats`` C1). Refuse to promote below ``min_n`` videos ? point estimates
from
14        too few held-out videos are noise, not signal.
15    2. Honest lift, not a single mean. Promote only when the paired ?F1 95% bootstrap CI
16        excludes 0 and the paired sign test agrees (majority of videos win, p <=
threshold).
17        This is the global promotion bar applied per user.
18    3. Structural guards are never disabled on "no lift". A structural guard (e.g.
Gate-A
19        net-crossings ? the held-shuttle false-positive killer) earns its keep through
failure
20        modes a given user's corpus may not contain yet. Measuring "no per-user lift" on a
corpus
21        that simply lacks those failure cases must NEVER turn the guard OFF. For
``structural=True``

```



```

22         the decision is therefore **always "keep ON"**, independent of the measured delta.
23
24     Below the floor / insufficient evidence ? fall back to the shipped baseline (the
conservative
25     default), never an unsafe promotion. All additive, pure, deterministic (the bootstrap
seed is
26     threaded through). Nothing here changes existing eval behaviour; callers opt in.
27     """
28     from __future__ import annotations
29
30     from dataclasses import dataclass, field
31     from typing import Any, Dict, Sequence
32
33     from backend.eval.eval_stats import paired_delta_ci, paired_sign_test
34
35     __all__ = ["PromotionDecision", "should_promote_for_user"]
36
37
38     @dataclass
39     class PromotionDecision:
40         """Structured result of a per-user promotion decision.
41
42         - ``promote`` ? flip variant B ON for this user (only ever ``True`` for a non-
structural
43         cue that cleared the full bar).
44         - ``keep_on`` ? the cue/guard should be ON for this user. For a structural guard
this is
45         always ``True`` (it is never disabled on "no lift"); for a non-structural cue it
equals
46         ``promote``.
47         - ``reason`` ? short human-readable explanation of the decision.
48         - the remaining fields surface the evidence (n, mean delta, CI bounds and the two
tests)
49         so a caller can log/audit exactly why a user was or wasn't personalized.
50         """
51
52         promote: bool
53         keep_on: bool
54         reason: str
55         n: int
56         mean_delta: float
57         ci_low: float
58         ci_high: float
59         ci_excludes_zero: bool
60         sign_test: Dict[str, float] = field(default_factory=dict)
61         floor_met: bool = False
62         structural_protected: bool = False
63
64         def as_dict(self) -> Dict[str, Any]:
65             """Plain-dict view (for JSON logging / telemetry)."""
66             return {
67                 "promote": self.promote,
68                 "keep_on": self.keep_on,
69                 "reason": self.reason,
70                 "n": self.n,
71                 "mean_delta": self.mean_delta,
72                 "ci_low": self.ci_low,
73                 "ci_high": self.ci_high,
74                 "ci_excludes_zero": self.ci_excludes_zero,
75                 "sign_test": self.sign_test,
76                 "floor_met": self.floor_met,
77                 "structural_protected": self.structural_protected,
78             }
79
80

```

```

81     def should_promote_for_user(
82         a_scores: Sequence[float],
83         b_scores: Sequence[float],
84         *,
85         structural: bool = False,
86         min_n: int = 5,
87         p_threshold: float = 0.05,
88         confidence: float = 0.95,
89         n_boot: int = 2000,
90         seed: int = 0,
91         eps: float = 0.0,
92     ) -> PromotionDecision:
93         """Decide whether to flip variant B ON for ONE user from that user's held-out
94         scores.
95         ``a_scores`` / ``b_scores`` are the user's per-video held-out F1 (or any score)
96         for
97         variant A (control / shipped baseline) and variant B (with the cue), aligned by
98         video
99         (same order). The decision rules (faithful to the module docstring):
100         1. ``structural=True`` ? the guard is never disabled on "no lift": return
101         ``keep_on=True, promote=False`` regardless of the measured per-user delta. Its
102         value is
103         in failure modes the user's corpus may not yet contain.
104         2. Otherwise promote iff ALL hold ? the minimum-N floor (``n >= min_n``), the
105         paired
106         ?F1 bootstrap CI excludes 0, and the paired sign test agrees (majority of
107         videos
108         win, ``p_value <= p_threshold``). Any miss ? fall back to the shipped baseline.
109         Statistics come straight from ``eval_stats`` (``paired_delta_ci`` /
110         ``paired_sign_test``);
111         deterministic given ``seed``. Returns a :class:`PromotionDecision`.
112         """
113         n = min(len(a_scores), len(b_scores))
114         floor_met = n >= min_n
115
116         # --- Evidence (reuse eval_stats; never reinvent the statistics)
117         -----
118         if n == 0:
119             # No paired videos at all ? nothing to test. Surface an empty-but-honest result.
120             sign_test: Dict[str, float] = paired_sign_test([], eps)
121             mean_delta, ci_low, ci_high, ci_excludes_zero = 0.0, 0.0, 0.0, False
122         else:
123             delta = paired_delta_ci(
124                 a_scores, b_scores,
125                 confidence=confidence, n_boot=n_boot, seed=seed, eps=eps,
126             )
127             mean_delta = float(delta["mean_delta"])
128             ci_low = float(delta["ci_low"])
129             ci_high = float(delta["ci_high"])
130             ci_excludes_zero = bool(delta["ci_excludes_zero"])
131             sign_test = delta["sign_test"]
132
133             sign_p = float(sign_test.get("p_value", 1.0))
134             sign_wins = float(sign_test.get("wins", 0.0))
135             sign_losses = float(sign_test.get("losses", 0.0))
136             # The sign test "agrees" with a promotion only if it is significant AND the majority
137             # leans the winning way (more B-wins than B-losses) ? a significant net loss
138             must
139             # never satisfy the promotion bar.
140             sign_agrees = (sign_p <= p_threshold) and (sign_wins > sign_losses)
141
142             # --- Rule 1: structural guards are never disabled on "no lift"

```

```

-----
137         if structural:
138             return PromotionDecision(
139                 promote=False,          # nothing to "promote" ? it ships ON by design
140                 keep_on=True,           # ALWAYS kept ON, regardless of measured per-user
lift
141                 reason=(
142                     "structural guard kept ON regardless of measured per-user lift "
143                     "(value is in failure modes this user's corpus may not yet contain)"
144                 ),
145                 n=n,
146                 mean_delta=mean_delta,
147                 ci_low=ci_low,
148                 ci_high=ci_high,
149                 ci_excludes_zero=ci_excludes_zero,
150                 sign_test=sign_test,
151                 floor_met=floor_met,
152                 structural_protected=True,
153             )
154
155         # --- Rule 2: minimum-N floor (refuse before even looking at lift)
-----
156         if not floor_met:
157             return PromotionDecision(
158                 promote=False,
159                 keep_on=False,          # fall back to the shipped baseline (variant A)
160                 reason=(
161                     f"below minimum-N floor (n={n} < min_n={min_n}); "
162                     "falling back to shipped baseline"
163                 ),
164                 n=n,
165                 mean_delta=mean_delta,
166                 ci_low=ci_low,
167                 ci_high=ci_high,
168                 ci_excludes_zero=ci_excludes_zero,
169                 sign_test=sign_test,
170                 floor_met=False,
171                 structural_protected=False,
172             )
173
174         # --- Rule 2 (cont.): honest lift ? CI excludes 0 AND sign test agrees
-----
175         if ci_excludes_zero and sign_agrees:
176             promote = True
177             reason = (
178                 f"promoted: n={n} >= {min_n}, ?F1 95% CI "
179                 f"[{ci_low:.4f}, {ci_high:.4f}] excludes 0, "
180                 f"sign test {int(sign_wins)}-{int(sign_losses)} (p={sign_p:.4f} <=
{p_threshold})"
181             )
182         else:
183             promote = False
184             misses = []
185             if not ci_excludes_zero:
186                 misses.append(f"?F1 95% CI [{ci_low:.4f}, {ci_high:.4f}] spans 0")
187             if not sign_agrees:
188                 misses.append(
189                     f"sign test does not agree ({int(sign_wins)}-{int(sign_losses)}, "
190                     f"p={sign_p:.4f})"
191                 )
192             reason = (
193                 "insufficient evidence (" + "; ".join(misses) + "); "
194                 "falling back to shipped baseline"
195             )
196

```

```

197         return PromotionDecision(
198             promote=promote,
199             keep_on=promote,          # non-structural: keep_on tracks promote exactly
200             reason=reason,
201             n=n,
202             mean_delta=mean_delta,
203             ci_low=ci_low,
204             ci_high=ci_high,
205             ci_excludes_zero=ci_excludes_zero,
206             sign_test=sign_test,
207             floor_met=True,
208             structural_protected=False,
209         )
210

```

27. assistant (2026-06-18T01:37:55.725Z)

These are clean. Let me focus on the actual production runtime path that's exercised: the detectors (wasb_runner, tracknet_runner) and the segmenters. These have CSV parsing and trajectory math that's error-prone. Let me look at tracknet_runner's parse and windowing, plus the audio hit_detector.

28. user (2026-06-18T01:37:57.034Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7            stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16
17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                          # WSL path to the .h5/.keras weights
(REQUIRED)
38         queue_length: int = 5

```

```

39         stage_in_wsl: bool = True                                # copy video into WSL fs first
(perf)
40         wsl_stage_dir: str = "~/clips"                            # where staged videos/outputs live
in WSL
41         timeout_sec: int = 1800                                  # 30 min; kills a hung call (0 = no
timeout)
42         keep_frames: bool = False                                # retain staged video after success
(debug only)
43         min_free_gb: float = 0.0                                # warn if WSL free space below this
(0 = off)
44
45     @classmethod
46     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
47         tn = (idx_cfg or {}).get("tracknet", {}) or {}
48         return cls(
49             distro=tn.get("wsl_distro", "Ubuntu"),
50             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
51             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
52             conda_env=tn.get("conda_env", "TrackNetV4"),
53             weights_path=tn.get("weights_path", ""),
54             queue_length=int(tn.get("queue_length", 5)),
55             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
56             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
57             timeout_sec=int(tn.get("timeout_sec", 1800)),
58             keep_frames=bool(tn.get("keep_frames", False)),
59             min_free_gb=float(tn.get("min_free_gb", 0.0)),
60         )
61
62
63     def to_wsl_mnt_path(win_path: str) -> str:
64         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
65
66         Already-POSIX paths (starting with / or ~) are returned unchanged so the
67         same helper is safe to call on either kind of path.
68         """
69         p = str(win_path)
70         if p.startswith("/") or p.startswith("~"):
71             return p
72         p = p.replace("\\", "/")
73         if len(p) >= 2 and p[1] == ":":
74             drive = p[0].lower()
75             return f"/mnt/{drive}{p[2:]}"
76         return p
77
78
79     @dataclass
80     class TrajectoryPoint:
81         frame: int
82         visible: bool
83         x: float
84         y: float
85
86
87     class TrackNetRunner(WSLCommandMixin, DetectorRunner):
88         """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90         Implements the common DetectorRunner contract so a future Linux-native or
91         alternative trajectory runner can be substituted without touching the hybrids.
92         """
93
94         def __init__(self, cfg: TrackNetConfig):
95             self.cfg = cfg
96
97         def healthcheck(self) -> Tuple[bool, str]:
98             """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.

```

```

99
100     Returns (ok, message). Cheap to call before a long run.
101     """
102     cmd = (
103         f"conda activate {self.cfg.conda_env} && "
104         f"python -c \"import tensorflow as tf; "
105         f"print('TF', tf.__version__, 'GPUs', "
106         f"len(tf.config.list_physical_devices('GPU')))\n"
107     )
108     try:
109         res = self._wsl_bash(cmd)
110     except FileNotFoundError:
111         return False, "wsl` not found ? is this running on Windows with WSL2
112         installed?"
113     except subprocess.TimeoutExpired:
114         return False, "WSL healthcheck timed out."
115     out = (res.stdout or "").strip()
116     if res.returncode != 0:
117         return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118     return True, out
119
120 # ----- inference
121
122 def run_predict(self, video_win_path: str, output_win_dir: str,
123                 video_id: Optional[str] = None) -> Optional[str]:
124     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125     None.
126
127     ``output_win_dir`` is a Windows directory (under the repo's output/) that
128     WSL writes into via its /mnt view, so the Windows process can read the CSV
129     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131     a filename cannot collide on the output CSV.
132     """
133     if not self.cfg.weights_path:
134         logger.error(
135             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136             "in config.json to the WSL path of your trained TrackNetV4 weights."
137         )
138     return None
139
140     # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
141     # relative paths through unchanged, so WSL would resolve them against the
142     # predict.py cwd instead of the repo (same bug class as wasb_runner).
143     output_win_dir = os.path.abspath(output_win_dir)
144     os.makedirs(output_win_dir, exist_ok=True)
145     # Normalize separators so basename/splitext work when tests (or collector)
146     # pass Windows-style paths on a Linux host.
147     video_win_path = str(video_win_path).replace("\\", "/")
148     base = os.path.basename(video_win_path)
149     stem, ext = os.path.splitext(base)
150
151     # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
152     if ext.lower() not in (".mp4", ".avi"):
153         logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
154         return None
155
156     wsl_out = to_wsl_mnt_path(output_win_dir)
157     # Namespace by video_id so two same-filename videos don't collide on the CSV.
158     # predict.py derives its output name from the (staged) video stem, so staging
159     # under the namespaced name propagates into "<key>_predict.csv".
160     key = f"{stem}_{video_id[:12]}" if video_id else stem
161     expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
162
163     # Resolve the video path WSL will read.

```

```

161         if self.cfg.stage_in_wsl:
162             staged = self._stage_video(video_win_path, f"{key}{ext}")
163             if staged is None:
164                 return None
165             wsl_video = staged
166         else:
167             # No staging: predict.py reads /mnt directly and writes
168             # We cannot rename its output, so fall back to the un-namespaced name.
169             wsl_video = to_wsl_mnt_path(video_win_path)
170             expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
171
172         if self.cfg.min_free_gb > 0:
173             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
174             if free is not None and free < self.cfg.min_free_gb:
175                 logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
176                                "consider running tools/cleanup_caches.py.",
177                                free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
178
179         cmd = (
180             f"conda activate {self.cfg.conda_env} && "
181             f"cd {self.cfg.repo_dir}/src && "
182             f"python predict.py "
183             f"--video_path {wsl_tilde_quote(wsl_video)} "
184             f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
185             f"--output_dir {wsl_tilde_quote(wsl_out)} "
186             f"--queue_length {self.cfg.queue_length}"
187         )
188         logger.info("Running TrackNet inference on %s ...", base)
189         try:
190             res = self._wsl_bash(cmd)
191         except subprocess.TimeoutExpired:
192             logger.error("TrackNet inference timed out after %ss.",
193                         self.cfg.timeout_sec)
194             return None
195
196         if res.returncode != 0:
197             logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
198                     res.stdout)[-2000:])
199             return None
200
201         if not os.path.exists(expected_csv_win):
202             logger.error("TrackNet finished but expected CSV not found at %s",
203                         expected_csv_win)
204             return None
205
206         logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
207
208         # Success ? the CSV is the durable output; drop the staged video copy (a pure,
209         # regenerable intermediate). On earlier failure we return above without
210         cleaning.
211
212         if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
213             logger.info("Cache hygiene: removing staged video for %s "
214                         "(set indexing.tracknet.keep_frames=true to retain).", key)
215             self._wsl_rm_rf([wsl_video])
216             return expected_csv_win
217
218     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
219         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
220
221         Returns the WSL path to the staged copy, or None on failure.
222         """
223         src_mnt = to_wsl_mnt_path(video_win_path)
224         reason = self.wsl_source_unreadable_reason(src_mnt)
225         if reason:
226             logger.error("Cannot stage video into WSL: %s", reason)

```

```

221         return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
224         {wsl_tilde_quote(dest)} && echo OK"
225         try:
226             res = self._wsl_bash(cmd)
227             except subprocess.TimeoutExpired:
228                 logger.error("Staging video into WSL timed out.")
229                 return None
230                 if res.returncode != 0 or "OK" not in (res.stdout or ""):
231                     logger.error("Failed to stage video into WSL: %s", (res.stderr or
232 res.stdout).strip())
233                     return None
234                     return dest
235
236 # ----- CSV -> windows
237
238 @staticmethod
239 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
240     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
241     points: List[TrajectoryPoint] = []
242     with open(csv_win_path, newline="") as f:
243         reader = csv.DictReader(f)
244         for row in reader:
245             try:
246                 points.append(TrajectoryPoint(
247                     frame=int(row["Frame"]),
248                     visible=int(row["Visibility"]) == 1,
249                     x=float(row["X"]),
250                     y=float(row["Y"]),
251                 ))
252             except (KeyError, ValueError):
253                 continue
254     points.sort(key=lambda p: p.frame)
255     return points
256
257 @staticmethod
258 def trajectory_to_action_windows(
259     points: List[TrajectoryPoint],
260     fps: float,
261     frame_width: float,
262     chunk_start: float = 0.0,
263     velocity_thresh: float = 0.02,
264     merge_gap: float = 2.0,
265     min_window_duration: float = 2.0,
266     chunk_index: Optional[int] = None,
267     max_window_duration: float = 0.0,
268     min_split_gap: float = 0.5,
269     window_hard_cap: bool = False,
270     window_inactivity_end: float = 0.0,
271     inactivity_min_density: float = 0.34,
272     inactivity_rally_thresh: float = 0.08,
273     window_blob_fallback: bool = False,
274     window_blob_factor: float = 4.0,
275 ) -> List[Dict[str, Any]]:
276     """Convert a shuttle trajectory into candidate rally windows.
277
278     A frame is "active" when the shuttle is visible AND its inter-frame
279     displacement (normalised by frame width, per second) exceeds
280     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
281     ``merge_gap`` seconds of each other are grouped into one window; short
282     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
283     shape feeding the AI-handoff phase is identical.
284
285     ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard

```


against
the *over-merge* failure where "shuttle moving" is conflated with "rally in progress"
and a single window swallows several rallies + the dead time between them. When set, any window longer than this is recursively SPLIT at its largest internal inactivity longest stretch with no in-flight shuttle ? the most likely rally boundary),
that gap is at least ``min\_split\_gap`` seconds. This is the bounded-windowing
fix from ``docs/RALLY\_QUALITY\_RESEARCH.md`` §6 (W1); it never merges, only cuts, so recall cannot drop, and it is config-gated default-OFF until measured on the golden set.

``window\_hard\_cap`` (default OFF) makes ``max\_window\_duration`` a TRUE cap: when an over-long window has NO internal inactivity gap to split on (a continuously-active trajectory ? the shuttle never stops moving, e.g. the real-footage
mahadevpura-1 blob where the gap-splitter alone left a single 174 s window), it is recursively hard-chopped at its temporal midpoint until every piece is ? the cap. Still only cuts (never merges), so recall cannot drop; gated default-OFF until measured.

``window\_blob\_fallback`` (default OFF ? the R5 last-resort blob splitter): unlike
``window\_hard\_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on every gap-less over-cap window and over-segments normal clips), this runs AFTER the R4 inactivity trim and force-chops ONLY a genuine *blob* ? a group still longer than
``window\_blob\_factor`` x ``max\_window\_duration`` once the gap-split and the principled rally-end trim have both had their chance (e.g. mahadevpura-1's ~65 s residual ? 11x a 6 s cap; a 7 s rally is left alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are logged, and each resulting window is flagged ``forced\_cut=True`` ? surfacing the clip as one that needs rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts (recall can't drop). Requires ``max\_window\_duration`` > 0; gated default-OFF until measured.

```

"""
if fps <= 0:
    fps = 30.0
if frame_width <= 0:
    frame_width = 1280.0

# Compute per-frame normalised velocity from the last *visible* point.
active = [] # (frame_idx, velocity)
prev: Optional[TrajectoryPoint] = None
for p in points:
    if not p.visible:
        prev = None # break the trajectory; absent shuttle = not in flight
        continue
    if prev is not None:
        dframes = p.frame - prev.frame
        if dframes > 0:
            dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
            velocity = (dist / frame_width) * (fps / dframes)
            if velocity > velocity_thresh:
```

```

328         active.append((p.frame, velocity))
329     prev = p
330
331     if not active:
332         return []
333
334     max_gap_frames = int(merge_gap * fps)
335
336     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337         vels = [v for _, v in group]
338         return {
339             "start": group[0][0] / fps + chunk_start,
340             "end": group[-1][0] / fps + chunk_start,
341             "active_frame_count": len(group),
342             "peak_velocity": max(vels),
343             "mean_velocity": sum(vels) / len(vels),
344             "track_id_count": 1, # single shuttle; kept for window-shape parity
345             "chunk_index": chunk_index,
346         }
347
348     groups: List[List[Tuple[int, float]]] = []
349     group = [active[0]]
350     for af in active[1:]:
351         if af[0] - group[-1][0] <= max_gap_frames:
352             group.append(af)
353         else:
354             groups.append(group)
355             group = [af]
356     groups.append(group)
357
358     if max_window_duration > 0:
359         groups = TrackNetRunner._split_overlong_groups(
360             groups, int(max_window_duration * fps), int(min_split_gap * fps),
361             hard_cap=window_hard_cap)
362
363     # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364     # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365     # "active" while the shuttle drifts/gets-picked-up above threshold, over-
extending the END
366     # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
367     # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
368     if window_inactivity_end > 0:
369         tw = int(window_inactivity_end * fps)
370         groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
371             inactivity_rally_thresh) for g
in groups]
372
373     # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
374     # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
375     # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
376     # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
377     forced_flags = [False] * len(groups)
378     if window_blob_fallback and max_window_duration > 0:
379         groups, forced_flags = TrackNetRunner._blob_fallback_chop(
380             groups, int(max_window_duration * fps), window_blob_factor)
381
382     windows = []

```

```

383         for g, forced in zip(groups, forced_flags):
384             w = _finalize(g)
385             if forced:
386                 w["forced_cut"] = True
387             windows.append(w)
388         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
389
390     @staticmethod
391     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
392                             min_density: float, rally_thresh: float) -> List[Tuple[int,
393 float]]:
394         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
395         *slowly*
396         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
397         windowing
398         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
399         ``rally_thresh``
400         (higher than the active threshold). The rally is "on" at frame ``i`` while its
401         trailing
402         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
403         back to the
404         LAST such frame ? everything after is sustained slow drift (rally over). Only
405         cuts (recall
406         can't rise); a window whose tail never goes slow is untouched, and if no fast-
407         dense point
408         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
409         pointer; the START
410         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
411         if trail_frames <= 0 or len(group) < 2:
412             return group
413         frames = [f for f, _ in group]
414         fast = [1 if v > rally_thresh else 0 for _, v in group]
415         lo = 0
416         fast_sum = 0
417         last_dense = -1
418         for i in range(len(frames)):
419             fast_sum += fast[i]
420             while frames[lo] <= frames[i] - trail_frames:
421                 fast_sum -= fast[lo]
422                 lo += 1
423             win_count = i - lo + 1
424             if win_count > 0 and fast_sum / win_count >= min_density:
425                 last_dense = i
426         return group[: last_dense + 1] if last_dense >= 0 else group
427
428     @staticmethod
429     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
430 int,
431                                 min_split_frames: int,
432                                 hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
433         """Recursively split any active-frame group longer than ``max_win_frames`` at
434         its largest
435         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
436         likely rally
437         boundary. Splitting only subdivides existing groups (never merges/extends), so
438         it cannot
439         invent or drop coverage; a group with no gap ? the floor is left intact (a
440         genuinely long
441         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
442         its
443         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
444         true upper
445         bound on continuously-active trajectories). Iterative worklist to bound stack
446         depth."""
447         if max_win_frames <= 0:

```

```

431         return groups
432     out: List[List[Tuple[int, float]]] = []
433     work = list(groups)
434     while work:
435         g = work.pop()
436         if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
437             out.append(g)
438             continue
439         best_i, best_gap = -1, -1
440         for i in range(1, len(g)):
441             gap = g[i][0] - g[i - 1][0]
442             if gap > best_gap:
443                 best_gap, best_i = gap, i
444         if best_i < 0 or best_gap < min_split_frames:
445             # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
446             # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
447             if hard_cap:
448                 mid = (g[0][0] + g[-1][0]) / 2.0
449                 split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
450                 work.append(g[:split_i])
451                 work.append(g[split_i:])
452             else:
453                 out.append(g)
454             continue
455             work.append(g[:best_i])
456             work.append(g[best_i:])
457     out.sort(key=lambda grp: grp[0][0])
458     return out
459
460     @staticmethod
461     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
462                             blob_factor: float = 4.0
463                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
464         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
465         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
466         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no
rally-end
467         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
468         11x a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2x the cap)
469         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
470         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
471         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
472         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
473         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
474         that piece came from a forced chop. Iterative worklist to bound stack depth."""
475         if max_win_frames <= 0:
476             return groups, [False] * len(groups)
477         blob_frames = max_win_frames * max(1.0, blob_factor)
478         out: List[List[Tuple[int, float]]] = []
479         flags: List[bool] = []
480         forced = 0
481         for g in groups:
482             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
483                 out.append(g)
484                 flags.append(False)

```

```

485         continue
486     pieces: List[List[Tuple[int, float]]] = []
487     work = [g]
488     while work:
489         h = work.pop()
490         if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
491             pieces.append(h)
492             continue
493         mid = (h[0][0] + h[-1][0]) / 2.0
494         split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) #
noqa: B023
495         work.append(h[:split_i])
496         work.append(h[split_i:])
497         pieces.sort(key=lambda grp: grp[0][0])
498         out.extend(pieces)
499         flags.extend([True] * len(pieces))
500         forced += len(pieces) - 1
501     if forced:
502         logger.warning(
503             "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
504                 "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
505                 "motion), not a bigger cap", forced)
506     return out, flags
507

```

29. assistant (2026-06-18T01:38:00.123Z)

Let me examine the `windowing.py` and `distill_local.py` modules (the production CSV-to-candidate path), plus check `segmentation_metrics` which `over_seg_guard` depends on for `merge_rate/split_rate` computation.

30. user (2026-06-18T01:38:01.377Z)

```

1     """Segmentation-aware metrics (course-correction C4) ? make over-segmentation visible.
2
3     Temporal IoU (metrics.score`) is nearly blind to *fragmentation*: a single rally
4     chopped into three predicted pieces, or two rallies merged across a dead-time gap, can
5     still post a moderate mean-IoU. The temporal-action-segmentation / -localization
6     literature grades those failure modes with **F1@k** (segmental F1 at several overlap
7     thresholds) and **mAP@tIoU** (ranked, confidence-aware average precision over a tIoU
8     sweep). Reporting these alongside temporal-IoU is what lets a reconciler's split/merge/
9     tighten win actually show up ? and stops an IoU-chasing rule from shredding rally
10    structure invisibly. See docs/ANALYZERS_AND_RECONCILERS.md` §13/C4.
11
12    This is **additive**: new functions over the existing metrics` matcher; nothing here
13    changes score`/compare`/LOVO behaviour. Pure (stdlib only), deterministic, no I/O
14    ?
15    so it runs in the GPU-free / numpy-free lanes too.
16
17    Conventions match metrics`: an Interval` is a (start, end)` tuple of seconds.
18    A ScoredInterval` is (interval, confidence)` ? the confidence is whatever the
scorer
19    emits per predicted rally (e.g. the classifier probability), used only to *rank* for AP.
20
21    Note on the segmental *edit* score: the classic TAS edit/Levenshtein score operates on
the
22    ordered sequence of segment *labels* and is degenerate for a single action class ?
rally/
23    background segments strictly alternate, so the edit distance collapses to (twice) the
24    segment-count difference, adding nothing over segment_count_ratio`. So we
intentionally
25    omit it and instead make over-segmentation explicit with merge_split_report` (a
bipartite

```

```

25 overlap-graph breakdown into merge/split/matched/missed/spurious ? see below), alongside
26 ``f1_at_overlaps`` (F1@k) + ``mean_average_precision`` (mAP@tIoU). Those, not a single-
class
27 edit score, are the over-segmentation-sensitive metrics for this binary rally setting.
28 """
29 from __future__ import annotations
30
31 from typing import Dict, List, Sequence, Tuple
32
33 from backend.eval.metrics import Interval, interval_iou, score
34
35 #: A predicted interval plus the confidence used to rank it for average precision.
36 ScoredInterval = Tuple[Interval, float]
37
38 #: Default overlap thresholds for F1@k ? the TAS convention F1@{10,25,50}.
39 DEFAULT_OVERLAPS: Tuple[float, ...] = (0.1, 0.25, 0.5)
40 #: Default tIoU sweep for mean average precision ? the action-detection convention.
41 DEFAULT_TIOUS: Tuple[float, ...] = (0.3, 0.4, 0.5, 0.6, 0.7)
42
43
44 def f1_at_overlaps(preds: List[Interval], gts: List[Interval],
45                   overlaps: Sequence[float] = DEFAULT_OVERLAPS) -> Dict[float, float]:
46     """Segmental F1 at each overlap threshold (``{overlap: f1}``) ? the TAS F1@k.
47
48     Reuses the canonical greedy one-to-one matcher, so a fragmented prediction (3 pieces
49     for 1 rally) scores extra false positives and a merged prediction misses a rally ?
50     both drag F1 down where temporal-IoU alone would not flinch.
51     """
52     return {ov: score(preds, gts, iou_threshold=ov).f1 for ov in overlaps}
53
54
55 def average_precision(scored_preds: Sequence[ScoredInterval], gts: List[Interval],
56                       iou_threshold: float = 0.5) -> float:
57     """Average precision at a single tIoU (all-points / VOC-2010 interpolation).
58
59     Predictions are ranked by descending confidence; each is a true positive iff it
60     claims a still-unmatched ground-truth rally at IoU >= ``iou_threshold`` (highest
61     available IoU wins, one GT per prediction). Returns AP in ``[0, 1]``. ``0.0`` when
62     there are no ground-truth rallies or no predictions reach the threshold.
63     """
64     n_gt = len(gts)
65     if n_gt == 0 or not scored_preds:
66         return 0.0
67
68     order = sorted(range(len(scored_preds)),
69                   key=lambda i: (-scored_preds[i][1], i)) # conf desc, stable by
index
70     matched_gt = set()
71     precisions: List[float] = []
72     recalls: List[float] = []
73     cum_tp = 0
74     cum_fp = 0
75     for i in order:
76         interval, _conf = scored_preds[i]
77         best_iou, best_gi = 0.0, -1
78         for gi, g in enumerate(gts):
79             if gi in matched_gt:
80                 continue
81             iou = interval_iou(interval, g)
82             if iou >= iou_threshold and iou > best_iou:
83                 best_iou, best_gi = iou, gi
84         if best_gi >= 0:
85             matched_gt.add(best_gi)
86             cum_tp += 1
87         else:

```

```

88         cum_fp += 1
89         precisions.append(cum_tp / (cum_tp + cum_fp))
90         recalls.append(cum_tp / n_gt)
91
92     # Monotone precision envelope (take the max precision to the right), then integrate
93     # over recall steps: AP = sum_k (recall_k - recall_{k-1}) * precision_env_k.
94     for i in range(len(precisions) - 2, -1, -1):
95         precisions[i] = max(precisions[i], precisions[i + 1])
96     ap = 0.0
97     prev_recall = 0.0
98     for p, r in zip(precisions, recalls):
99         if r > prev_recall:
100             ap += p * (r - prev_recall)
101             prev_recall = r
102     return ap
103
104
105 def mean_average_precision(scored_preds: Sequence[ScoreInterval], gts: List[Interval],
106                             iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> float:
107     """Mean of ``average_precision`` over a tIoU sweep (the standard mAP@tIoU)."""
108     tious = list(iou_thresholds)
109     if not tious:
110         return 0.0
111     return sum(average_precision(scored_preds, gts, t) for t in tious) / len(tious)
112
113
114 def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
115     """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.
116
117     ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
118     merging/misses. Only meaningful with ground truth present; returns ``0.0`` if
119     ``gts``
120     is empty (use F1@k / mAP for the graded picture).
121     """
122     if not gts:
123         return 0.0
124     return len(preds) / len(gts)
125
126 def _overlaps(a: Interval, b: Interval) -> bool:
127     """True iff two intervals share any positive temporal extent."""
128     return min(a[1], b[1]) - max(a[0], b[0]) > 0.0
129
130
131 def merge_split_report(preds: List[Interval], gts: List[Interval]) -> Dict[str, float]:
132     """Explicit over-/under-segmentation breakdown via the bipartite overlap graph.
133
134     Where ``segment_count_ratio`` only compares *counts* and ``f1_at_overlaps`` blends
135     merge
136     and split into one F1, this names the two failure modes directly ? the diagnostic
137     that
138     surfaced our local-segmenter over-MERGE (one giant window swallowing many rallies).
139     Build
140     the bipartite graph (edge = any positive temporal overlap between a pred and a gt),
141     take its
142     connected components, and classify each by ``(#preds, #gts)``:
143
144     - ``(1,1)`` matched    ``(1,0)`` spurious (pred, no gt)    ``(0,1)`` missed (gt, no
145     pred)
146     - ``(1,>1)`` **merge** ? one predicted window covers ?2 rallies (over-MERGE; our
147     failure)
148     - ``(>1,1)`` **split** ? one rally cut across ?2 predictions (over-segmentation)
149     - ``(>1,>1)`` complex (tangled many-to-many)
150
151     Returns counts plus ``merge_rate`` (fraction of GT rallies swallowed inside a merge

```

```

group)
146 and ``split_rate`` (fraction of GT rallies cut by ?2 predictions) ? the two headline
147 over-segmentation rates. Pure, deterministic, stdlib-only.
148 """
149 n, m = len(preds), len(gts)
150 p_adj: List[List[int]] = [[] for _ in range(n)]
151 g_adj: List[List[int]] = [[] for _ in range(m)]
152 for i, p in enumerate(preds):
153     for j, g in enumerate(gts):
154         if _overlaps(p, g):
155             p_adj[i].append(j)
156             g_adj[j].append(i)
157
158 seen_p: set = set()
159 seen_g: set = set()
160 comps: List[Tuple[set, set]] = []
161 for start in range(n):
162     if start in seen_p:
163         continue
164     ps: set = set()
165     gs: set = set()
166     stack: List[Tuple[str, int]] = [("p", start)]
167     while stack:
168         side, k = stack.pop()
169         if side == "p":
170             if k in ps:
171                 continue
172             ps.add(k)
173             seen_p.add(k)
174             stack.extend(("g", j) for j in p_adj[k] if j not in gs)
175         else:
176             if k in gs:
177                 continue
178             gs.add(k)
179             seen_g.add(k)
180             stack.extend(("p", i) for i in g_adj[k] if i not in ps)
181     comps.append((ps, gs))
182     comps.extend((set(), {j}) for j in range(m) if j not in seen_g) # gts with no
overlapping pred
183
184 counts = {"matched": 0, "merges": 0, "splits": 0, "spurious": 0, "missed": 0,
"complex": 0}
185 gts_merged = gts_split = 0
186 for ps, gs in comps:
187     a, b = len(ps), len(gs)
188     if (a, b) == (1, 1):
189         counts["matched"] += 1
190     elif (a, b) == (1, 0):
191         counts["spurious"] += 1
192     elif (a, b) == (0, 1):
193         counts["missed"] += 1
194     elif a == 1 and b > 1:
195         counts["merges"] += 1
196         gts_merged += b
197     elif b == 1 and a > 1:
198         counts["splits"] += 1
199         gts_split += 1
200     else:
201         counts["complex"] += 1
202 out: Dict[str, float] = {k: float(v) for k, v in counts.items()}
203 out["merge_rate"] = (gts_merged / m) if m else 0.0
204 out["split_rate"] = (gts_split / m) if m else 0.0
205 return out
206
207

```



```

208     def segmentation_report(scored_preds: Sequence[ScoreInterval], gts: List[Interval],
209                             overlaps: Sequence[float] = DEFAULT_OVERLAPS,
210                             iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> Dict[str,
object]:
211         """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio + merge/split
breakdown.
212         ``scored_preds`` carry confidences for AP; F1@k + the merge/split breakdown use the
bare
213         intervals. Returns a plain dict (JSON-friendly, string keys) so it can ride next to
a
214         ``compare`` result or land in the experiment leaderboard.
215         """
216         preds = [iv for iv, _conf in scored_preds]
217         return {
218             "f1_at_overlaps": {f"{ov:g}": v for ov, v in f1_at_overlaps(preds, gts,
overlaps).items()},
219             "ap_at_tiou": {f"{t:g}": average_precision(scored_preds, gts, t) for t in
iou_thresholds},
220             "mean_ap": mean_average_precision(scored_preds, gts, iou_thresholds),
221             "segment_count_ratio": segment_count_ratio(preds, gts),
222             "merge_split": merge_split_report(preds, gts),
223             "num_pred": len(preds),
224             "num_gt": len(gts),
225         }
226
227

```

31. assistant (2026-06-18T01:38:06.650Z)

I notice a potential issue in `merge\_split\_report`. The `split\_rate` is computed as `gts\_split / m`, but `gts\_split` is incremented by `1` per split component (line 199: `gts\_split += 1`), while a split component is `( >1, 1 )` ? exactly one GT. So `gts\_split` counts the number of GTs that were split, which is correct. And `merge\_rate` uses `gts\_merged += b` (the number of GTs in the merge), correct. That's fine.

Let me look at `distill\_local.py` (build_candidates, the production path) and `windowing.py`.

32. user (2026-06-18T01:38:07.607Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7
8     Flow (all local at runtime):
9     WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10     filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11     active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12     -> rally windows.
13
14     Training labels come from Gemini full-context labels via the SAME IoU matching the
15     evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16     other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17     compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18     rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20     Run:
21     python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22     """
23     from __future__ import annotations
24
25     import json

```

```

26     import argparse
27     from typing import Dict, List, Optional, Tuple
28
29
30     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31     from backend.pipeline.detectors import rally_gate
32     from backend.eval.calibrate_wasb import load_golden_gts
33     from backend.eval.training import label_candidates
34     from backend.eval.classifier import LogisticRegression
35     from backend.eval.metrics import score
36     from backend.eval.gemini_refine import merge_overlaps
37     from backend.eval import windowing as _windowing
38
39     Interval = Tuple[float, float]
40
41     FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
42 "net_crossings"]
43     # Recall-oriented proposal: deliberately permissive so real rallies are among the
44     # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur).
45     # Single-sourced from backend.eval.windowing so eval and serve can never drift ? see
46     that
47     # module's docstring for the eval?serve bug these named presets close. The bare tuples
48     are
49     # preserved (byte-identical: PROPOSAL == (0.005, 1.0, 0.5), BASELINE_LOCAL == SERVED ==
50     # (0.02, 2.0, 2.0)) so every existing call site / golden JSON stays valid.
51     PROPOSAL = _windowing.PROPOSAL.as_tuple()
52     BASELINE_LOCAL = _windowing.SERVED.as_tuple() # the served (production) windowing, no
53     learning
54
55
56
57     def build_candidates(trajecory_csv: str, fps: float, frame_width: float,
58                         proposal: Tuple[float, float, float] = PROPOSAL) ->
59     Tuple[List[Interval], List[List[float]]]:
60     """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
61     rows)."""
62     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
63     vt, mg, mwd = proposal
64     wins = TrackNetRunner.trajectory_to_action_windows(
65         points, fps=fps, frame_width=frame_width,
66         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
67     intervals = [(w["start"], w["end"]) for w in wins]
68     gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
69     crossings
70     X: List[List[float]] = []
71     for w, g in zip(wins, gated):
72         dur = max(1e-6, w["end"] - w["start"])
73         win_frames = max(1.0, dur * fps)
74         X.append([
75             dur, # rally length (sec) ?
76             float(w["peak_velocity"]), # already normalized by
77             float(w["mean_velocity"]),
78             float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?
79             float(g.crossings), # net crossings (count) ?
80             the rally signal
81         ])
82     return intervals, X
83
84
85
86     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
87     List[Interval]:
88     points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
89     vt, mg, mwd = BASELINE_LOCAL

```

```

79     wins = TrackNetRunner.trajectory_to_action_windows(
80         points, fps=fps, frame_width=frame_width,
81         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
82     return [(w["start"], w["end"]) for w in wins]
83
84
85     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
86         fps, fw = float(spec["fps"]), float(spec["frame_width"])
87         intervals, X = build_candidates(spec["trajectory"], fps, fw)
88         gts = load_golden_gts(spec["labels"])
89         y = label_candidates(intervals, gts, iou)
90         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
91             X,
92             "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
93             fw)}
94
95     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
96     List[Interval],
97     threshold: float = 0.5) -> List[Interval]:
98         if not intervals:
99             return []
100         proba = model.predict_proba(X)
101         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
102         return merge_overlaps(pos, gap_tol=1.0)
103
104     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
105     model_out: Optional[str] = None) -> dict:
106         videos = [load_video(s, iou) for s in specs]
107         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
108             f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
109         for v in videos:
110             ceil = score(v["intervals"], v["gts"], iou).recall
111             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
112                 f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
113
114         result: dict = {"videos": [v["name"] for v in videos]}
115         if len(videos) >= 2:
116             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
117             out) ---")
118             clf, base = [], []
119             for i, held in enumerate(videos):
120                 others = [v for j, v in enumerate(videos) if j != i]
121                 X = [row for v in others for row in v["X"]]
122                 y = [t for v in others for t in v["y"]]
123                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
124                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
125                 sc = score(preds, held["gts"], iou)
126                 bsc = score(held["baseline"], held["gts"], iou)
127                 clf.append(sc)
128                 base.append(bsc)
129                 print(f"    held-out {held['name']}: LEARNED P={sc.precision:.3f}
130                 R={sc.recall:.3f} F1={sc.f1:.3f}"
131                     f"    | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
132                     F1={bsc.f1:.3f}")
133             def mean(xs, a):
134                 return sum(getattr(s, a) for s in xs) / len(xs)
135             print(f"    MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
136                 R={mean(clf, 'recall'):.3f} "
137                     f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
138                     f"R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
139             result["mean_learned_f1"] = mean(clf, "f1")
140             result["mean_baseline_f1"] = mean(base, "f1")

```

```

137     # Final model on ALL videos (for production / running on a fresh held-out video).
138     Xall = [row for v in videos for row in v["X"]]
139     yall = [t for v in videos for t in v["y"]]
140     final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
141     assert final.w is not None # set by fit()
142     weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
143     print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
144     if model_out:
145         final.save(model_out)
146         print(f"final model (all videos) -> {model_out}")
147         result["model_out"] = model_out
148     return result
149
150
151     def main() -> None:
152         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
153         ap.add_argument("--manifest", required=True,
154                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
155         ap.add_argument("--iou", type=float, default=0.5)
156         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
157         ap.add_argument("--model-out", default="output/local_rally_model.json")
158         args = ap.parse_args()
159         with open(args.manifest, encoding="utf-8") as f:
160             specs = json.load(f)
161         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
162
163
164     if __name__ == "__main__":
165         main()
166

```

33. assistant (2026-06-18T01:38:12.883Z)

Let me look at the rally_gate (net-crossings, the structural guard, central to production) and the audio hit_detector, plus the geometry util.

34. user (2026-06-18T01:38:13.890Z)

```

1     """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3     WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4     the opponent for service after losing a point, that single trajectory looks like a
5     rally. The discriminator is structural, not duration-based: a real rally has the
6     shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7     single service feed crosses -once.
8
9     This module is camera-agnostic and needs NO new model ? it works on the trajectory
10    we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11    axis with the larger shuttle spread) and the net position (median shuttle coord on
12    that axis ? the shuttle clusters at the two ends and crosses the net region often,
13    so the median sits near the net), then counts sign changes of (coord - net) across
14    the visible points inside a candidate window, with a deadband to ignore jitter.
15
16    Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17    Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18    windows from trajectory_to_action_windows, or to fold into it later.
19    """
20    from __future__ import annotations
21
22    from dataclasses import dataclass
23    from typing import List, Optional, Sequence, Tuple
24

```

```

25     Interval = Tuple[float, float]
26
27
28     @dataclass
29     class GatedWindow:
30         start: float
31         end: float
32         crossings: int
33         visible_points: int
34         kept: bool
35
36
37     def _spread(vals: Sequence[float]) -> float:
38         """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39         if len(vals) < 2:
40             return 0.0
41         s = sorted(vals)
42         lo = s[max(0, int(0.05 * (len(s) - 1)))]
43         hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44         return hi - lo
45
46
47     def _median(vals: Sequence[float]) -> float:
48         if not vals:
49             return 0.0
50         s = sorted(vals)
51         n = len(s)
52         return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55     def estimate_play_axis(points) -> Tuple[str, float, float]:
56         """Return (axis 'x'/'y', net_value, span) from visible points.
57
58         Play axis = the image axis along which the shuttle travels most (larger spread).
59         For a baseline camera that's Y (players top/bottom); for a side camera, X.
60         """
61         xs = [p.x for p in points if p.visible]
62         ys = [p.y for p in points if p.visible]
63         sx, sy = _spread(xs), _spread(ys)
64         if sx >= sy:
65             return "x", _median(xs), sx
66         return "y", _median(ys), sy
67
68
69     def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70         """Count sign changes of (coord - net) across visible points, ignoring the
71         deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72         crossings = 0
73         last_side = None
74         for p in window_points:
75             if not p.visible:
76                 continue
77             v = (p.x if axis == "x" else p.y) - net
78             if abs(v) < deadband:
79                 continue
80             side = v > 0
81             if last_side is not None and side != last_side:
82                 crossings += 1
83             last_side = side
84         return crossings
85
86
87     def _points_in(points, fps: float, start: float, end: float):
88         return [p for p in points if start * fps <= p.frame <= end * fps]
89

```

```

90
91     def gate_windows(points, windows: List[Interval], fps: float,
92                       min_crossings: int = 2, deadband_frac: float = 0.06,
93                       estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94         """Annotate each (start,end) window with its shuttle net-crossing count and a
95         keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96         axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98         `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99         `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100         per window ? same result, avoids the per-window re-estimation."""
101         axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102         deadband = deadband_frac * span
103         out: List[GatedWindow] = []
104         for (s, e) in windows:
105             wp = _points_in(points, fps, s, e)
106             c = count_net_crossings(wp, axis, net, deadband)
107             vis = sum(1 for p in wp if p.visible)
108             out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                                   kept=c >= min_crossings))
110         return out
111
112
113     def apply_gate(points, windows: List[Interval], fps: float,
114                   min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115         """Convenience: return only the windows that pass the crossing gate."""
116         return [(g.start, g.end) for g in
117                 gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

35. assistant (2026-06-18T01:38:21.383Z)

Clean. Let me now look at the API request boundary (main.py) and the segmenters' time parsing (gemini.py parse_time) which are runtime-exercised. Also the `eval\_stats` module that promotion depends on for the bootstrap CI ? a subtle stats bug there would be real.

36. user (2026-06-18T01:38:22.418Z)

```

1     """Honest small-N evaluation statistics (course-corrections C1 + C3).
2
3     The ~6-match golden corpus + a regularised logistic arbiter is exactly the regime where
a
4     single leave-one-video-out *mean* lies: leave-one-out anti-correlates each fold's
training
5     and held-out label means at small N, and one lucky/unlucky fold swings the headline. The
6     ``compare``/LOVO machinery already produces **per-video held-out scores** ? this module
7     turns those into an honest summary:
8
9     - **C1 ? never a single mean.** ``mean_with_ci`` / ``bootstrap_ci`` report a confidence
10     interval; ``paired_sign_test`` and ``paired_delta_ci`` grade a B?A improvement by how
11     many *videos* it wins on (the distribution-free test the repo already leans on:
12     "sign test needs n ? ~n=10/9-wins for p<0.05"), so the promotion gate can't promote
noise.
13     - **C3 ? honest stacking.** ``out_of_fold_predictions`` / ``grouped_folds`` give a
learned
14     producer's *out-of-fold* outputs so feeding them to the arbiter doesn't leak (group by
15     match, never a held-out window from the same video).
16
17     See ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C1+C3. All **additive**, pure (stdlib
only),
18     deterministic (bootstrap takes an explicit ``seed``) ? nothing here changes existing
19     ``compare``/``calibration.leave_one_video_out`` behaviour; callers opt in.
20     """

```

```

21     from __future__ import annotations
22
23     import math
24     import random
25     import statistics
26     from typing import Any, Callable, Dict, List, Optional, Sequence, Tuple
27
28     Interval = Tuple[float, float]
29     FitPredict = Callable[[List[int], List[int]], List[Any]]
30
31
32     def bootstrap_ci(values: Sequence[float], confidence: float = 0.95,
33                     n_boot: int = 2000, seed: int = 0) -> Tuple[float, float]:
34         """Percentile bootstrap CI for the mean of ``values``. Deterministic given ``seed``.
35
36         ``n<=1`` returns a degenerate ``(v, v)`` / ``(0, 0)`` interval ? there is no spread
37         to
38         resample. With n=6 (our corpus) this is the honest "how wide could the mean be?"
39         band.
40         """
41         vals = [float(v) for v in values]
42         n = len(vals)
43         if n == 0:
44             return (0.0, 0.0)
45         if n == 1:
46             return (vals[0], vals[0])
47         rng = random.Random(seed)
48         means: List[float] = []
49         for _ in range(max(1, n_boot)):
50             means.append(sum(vals[rng.randrange(n)] for _ in range(n)) / n)
51         means.sort()
52         alpha = (1.0 - confidence) / 2.0
53         last = len(means) - 1
54         lo = means[max(0, int(math.floor(alpha * last)))]
55         hi = means[min(last, int(math.ceil((1.0 - alpha) * last)))]
56         return (lo, hi)
57
58     def mean_with_ci(values: Sequence[float], confidence: float = 0.95,
59                     n_boot: int = 2000, seed: int = 0) -> Dict[str, float]:
60         """``{mean, std, n, ci_low, ci_high, confidence}`` ? report this, never a bare mean
61         (C1)."""
62         vals = [float(v) for v in values]
63         n = len(vals)
64         mean = statistics.mean(vals) if vals else 0.0
65         std = statistics.pstdev(vals) if n > 1 else 0.0
66         lo, hi = bootstrap_ci(vals, confidence, n_boot, seed)
67         return {"mean": mean, "std": std, "n": float(n),
68                 "ci_low": lo, "ci_high": hi, "confidence": confidence}
69
70     def paired_sign_test(deltas: Sequence[float], eps: float = 0.0) -> Dict[str, float]:
71         """Two-sided exact sign test over per-fold deltas (B ? A).
72
73         ``wins`` = folds where B beat A by more than ``eps``; ``losses`` the reverse; ties
74         excluded. ``p_value`` is the exact two-sided binomial tail under p=0.5 ? the
75         distribution-free "did B win on enough *videos*?" check.
76         """
77         wins = sum(1 for d in deltas if d > eps)
78         losses = sum(1 for d in deltas if d < -eps)
79         ties = len(deltas) - wins - losses
80         n_eff = wins + losses
81         if n_eff == 0:
82             p = 1.0
83         else:

```

```

83         k = min(wins, losses)
84         tail = sum(math.comb(n_eff, j) for j in range(k + 1)) * (0.5 ** n_eff)
85         p = min(1.0, 2.0 * tail)
86     return {"wins": float(wins), "losses": float(losses), "ties": float(ties),
87           "n_effective": float(n_eff), "p_value": p}
88
89
90 def paired_delta_ci(a_values: Sequence[float], b_values: Sequence[float],
91                    confidence: float = 0.95, n_boot: int = 2000, seed: int = 0,
92                    eps: float = 0.0) -> Dict[str, Any]:
93     """Paired (by fold) B ? A summary: mean delta + bootstrap CI + the sign test (C1).
94
95     ``a_values``/``b_values`` are per-fold held-out scores for variants A and B, aligned
96     by
97     fold (same video order). A promotion is honest when the delta CI clears 0 *and* the
98     sign
99     test agrees ? not when a single mean ticked up.
100     """
101     n = min(len(a_values), len(b_values))
102     deltas = [float(b_values[i]) - float(a_values[i]) for i in range(n)]
103     summary = mean_with_ci(deltas, confidence, n_boot, seed)
104     return {
105         "mean_delta": summary["mean"],
106         "ci_low": summary["ci_low"],
107         "ci_high": summary["ci_high"],
108         "n": float(n),
109         "ci_excludes_zero": (summary["ci_low"] > 0.0) or (summary["ci_high"] < 0.0),
110         "sign_test": paired_sign_test(deltas, eps),
111     }
112
113 def grouped_folds(groups: Sequence[Any]) -> List[Tuple[List[int], List[int]]]:
114     """Leave-one-group-out splits ``(train_idx, test_idx)`` ? group by match, the honest
115     unit.
116
117     The field's name for "LOVO by match" is leave-one-group-out (LOGO). Group order is
118     first-seen (deterministic). Groups with no complement (a single group) yield no
119     fold.
120
121     """
122     seen: List[Any] = []
123     for g in groups:
124         if g not in seen:
125             seen.append(g)
126     n = len(groups)
127     folds: List[Tuple[List[int], List[int]]] = []
128     for g in seen:
129         test = [i for i in range(n) if groups[i] == g]
130         train = [i for i in range(n) if groups[i] != g]
131         if test and train:
132             folds.append((train, test))
133     return folds
134
135 def out_of_fold_predictions(groups: Sequence[Any], fit_predict: FitPredict) ->
136 List[Optional[Any]]:
137     """Out-of-fold predictions for honest stacking (C3).
138
139     For each leave-one-group-out fold, ``fit_predict(train_idx, test_idx)`` trains on
140     the
141     other groups and returns predictions for ``test_idx`` (in that order); results are
142     scattered back to a list aligned to the input items. So a *learned* producer's
143     outputs,
144     when used as arbiter features, never include in-sample predictions ? the leakage
145     path
146     that inflates measured lift on a tiny corpus. Items in a singleton group stay

```



```

``None``
140         (needs >= 2 groups to predict).
141         """
142         n = len(groups)
143         oof: List[Optional[Any]] = [None] * n
144         for train_idx, test_idx in grouped_folds(groups):
145             preds = fit_predict(train_idx, test_idx)
146             for pos, ti in enumerate(test_idx):
147                 if pos < len(preds):
148                     oof[ti] = preds[pos]
149         return oof
150

```

37. assistant (2026-06-18T01:38:26.434Z)

These are clean and well-tested. Let me look at the API surface (main.py) for the request validation/path handling ? that's the externally-reachable error path. Also the run_telemetry util.

38. user (2026-06-18T01:38:27.720Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import threading
6     import traceback
7     import uuid
8     from concurrent.futures import ThreadPoolExecutor
9     import uvicorn
10    from dotenv import load_dotenv
11
12    load_dotenv()
13
14    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
15    logging.basicConfig(
16        level=logging.INFO,
17        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
18        datefmt='%H:%M:%S'
19    )
20    logger = logging.getLogger(__name__)
21    from fastapi import FastAPI, HTTPException, Request
22    from fastapi.middleware.cors import CORSMiddleware
23    from pydantic import BaseModel
24    from typing import List, Dict, Any, Optional
25
26    from fastapi.staticfiles import StaticFiles
27    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
28    from backend.pipeline.validators.base import registry
29    from backend.infrastructure.database import Database
30    from backend.pipeline.segmenters.base import segmenter_registry
31    from backend.pipeline.stitcher.compiler import VideoCompiler
32    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
33    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
34    from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
35    from backend.results import Failure # standardized error/result contract
36    from backend.reporting import emit_indexer_report
37
38    app = FastAPI(title="Sports Highlight Indexer API")
39
40    # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is

```

```

41     # rejected by browsers per the fetch spec and is an unsafe default to carry into the
42     # auth/tenancy work; this tool uses no cookies, so credentials stay off.
43     app.add_middleware(
44         CORSMiddleware,
45         allow_origins=["*"],
46         allow_credentials=False,
47         allow_methods=["*"],
48         allow_headers=["*"],
49     )
50
51     # Serves static frontend files directly (now under api/ per approved structure)
52     os.makedirs("backend/api/static", exist_ok=True)
53     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
54
55     @app.get("/", response_class=HTMLResponse)
56     def serve_index():
57         index_path = "backend/api/static/index.html"
58         if os.path.exists(index_path):
59             with open(index_path, "r", encoding="utf-8") as f:
60                 return f.read()
61         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
62
63     # Global variables initialized at startup
64     db_instance: Optional[Database] = None
65     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
66
67     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
68     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
69     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
70     # already parallelize their AI handoff internally via their own pools.
71     _job_executor: Optional[ThreadPoolExecutor] = None
72
73     def _get_executor() -> ThreadPoolExecutor:
74         """Lazy module-global executor; tests monkeypatch this for inline execution."""
75         global _job_executor
76         if _job_executor is None:
77             _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
78         return _job_executor
79
80     # Legacy thin wrapper for any remaining direct calls during transition.
81     # New code should import load_app_config / AppConfig from backend.config directly.
82     def load_config(config_path: str = "config.json") -> Dict[str, Any]:
83         cfg = load_app_config(config_path)
84         return cfg.model_dump(mode="json")
85
86     # --- API Models ---
87     class ProcessRequest(BaseModel):
88         video_path: str
89         player_pool: Optional[List[str]] = None
90         max_frames: Optional[int] = None
91         segmenter: Optional[str] = None
92
93     class QueryRequest(BaseModel):
94         video_id: str
95         server: Optional[str] = None
96         receiver: Optional[str] = None
97         duration_min: Optional[float] = None
98         event_type: Optional[str] = None
99
100     class CompileRequest(BaseModel):
101         video_id: str
102         segment_ids: List[int]
103         output_filename: str

```

```

104
105 def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
106     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
107
108     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
109     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
110     and
111     keep the prior good results intact ? a failed/empty re-run never destroys prior
112     data.
113     """
114     if segments:
115         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
116     else:
117         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
118
119 # --- API Endpoints ---
120 @app.get("/api/config")
121 def get_config():
122     return global_config
123
124 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
125     """The full hash ? validate ? segment ? report pipeline for one video.
126
127     This is the former synchronous /api/process body, unchanged in behavior, now run on
128     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
129     the sync endpoint used to return (UI compatibility); the per-video observability
130     report is still emitted in `finally:` and grafted as response["reports"].
131
132     `progress` is an optional callable(stage: str) used to surface coarse job progress.
133     Raises on unexpected internal errors ? the caller records those as a job failure
134     (after this function has already restored the pre-run segments, see below).
135     """
136     notify = progress or (lambda stage: None)
137
138     notify("hashing")
139     video_id = compute_video_id(req.video_path)
140     was_reingest = db_instance.get_video(video_id) is not None
141
142     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
143     report)
144     notify("validating")
145     logger.info(f"Running validations for {req.video_path}...")
146     val_results, val_context = registry.run_all_with_context(req.video_path,
147     global_config)
148     failures = [r for r in val_results if not r.passed]
149
150     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
151     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
152     "motion")
153     seg_name = req.segmenter or default_seg
154     segmenter_actual = None
155     segmentation_ran = False
156     response: Optional[Dict[str, Any]] = None
157     try:
158         if failures:
159             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
160             # status; it no longer destroys the video's prior good segments.
161             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
162     for r in val_results])
163             response = {
164                 "video_id": video_id,
165                 "status": "failed",
166                 "message": "Video failed validation checks.",
167                 "results": [r.model_dump() for r in val_results],

```

```

163         }
164         return response
165
166     # 2. Run segmenter & indexer
167     logger.info("[API] Video passed checks. Segmenting and indexing...")
168     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
169
170     segmenter_cls = segmenter_registry.get(seg_name)
171     if not segmenter_cls:
172         # Submit-time validation already 400s unknown names; this is the defensive
173         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
174         available = list(segmenter_registry.list_available().keys())
175         response = {
176             "video_id": video_id,
177             "status": "failed",
178             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
179             "results": [r.model_dump() for r in val_results],
180             "play_segments": [],
181         }
182         return response
183
184     logger.info(f"[API] Using segmenter: {seg_name}")
185     notify(f"segmenting ({seg_name})")
186     segmenter_actual = seg_name
187     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
188     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
189     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
190     segmenter = segmenter_cls(db_instance, global_config)
191     try:
192         result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
193     except Exception:
194         # A raising segmenter must not leave half-written rows: restore the pre-run
195         # state (same destroy-after-confirm contract as the Failure branch), then
let
196         # the job worker record the failure.
197         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
198         raise
199     segmentation_ran = True
200
201     if isinstance(result, Failure):
202         logger.error(f"[API] Segmenter failed: {result.message}")
203         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
204         response = {
205             "video_id": video_id,
206             "status": "failed",
207             "message": f"Segmenter '{seg_name}' failed: {result.message}",
208             "results": [r.model_dump() for r in val_results],
209             "play_segments": [],
210         }
211         return response
212
213     segments = result.value if hasattr(result, "value") else result
214     notify("finalizing")
215     _finalize_reingest(video_id, segments, play_wm, cand_wm)
216
217     response = {
218         "video_id": video_id,
219         "status": "success",
220         "message": f"Successfully indexed {len(segments)} play segments using

```

```

'{{seg_name}}' segmenter.",
221         "results": [r.model_dump() for r in val_results],
222         "play_segments": segments,
223     }
224     return response
225 finally:
226     # Always emit the per-video observability report (next to the video, or to
227     # report.output_dir). Never raises. Surface the paths in the response.
228     paths = emit_indexer_report(
229         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
230         val_results=val_results, val_context=val_context,
231         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
232         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
233     )
234     if paths and isinstance(response, dict):
235         response["reports"] = paths
236
237
238     class JobWaiting(Exception):
239         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
240         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`",
241         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
242         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
243         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
244
245         The wiring is additive: the production segmenter path never raises this today (zero
behaviour change), so a job runs exactly as before. The hook is what a later slice
246         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
247         """
248
249         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
250             super().__init__(f"job parked for {delay_sec:.0f}s")
251             self.delay_sec = max(0.0, float(delay_sec))
252             self.progress = progress
253
254
255     def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:
256         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
257         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
258         the original submit. The row stores exactly the ProcessRequest fields."""
259         return ProcessRequest(
260             video_path=job["video_path"],
261             player_pool=job.get("player_pool"),
262             max_frames=job.get("max_frames"),
263             segmenter=job.get("segmenter"),
264         )
265
266
267     def _run_claimed_job(job: Dict[str, Any]) -> None:
268         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its
269         terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
270
271         Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
272         the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
273         the
274         job self-scheduled and will be re-claimed when `next_poll_at` is due.
275         """
276         job_id = job["id"]

```

```

278     req = _job_to_request(job)
279     try:
280         result = _execute_processing(
281             req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
282         if result.get("video_id"):
283             db_instance.set_job_video_id(job_id, result["video_id"])
284             db_instance.mark_job_done(job_id, result)
285     except JobWaiting as w:
286         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
287         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
288         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
289     except Exception as e:
290         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
291         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
292
293
294     def _drain_due_jobs() -> int:
295         """One worker tick: claim and run every currently-runnable job (queued, or waiting
296         whose
297         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
298         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
299         picked
300         up by a FUTURE drain once due; we schedule that follow-up drain via
301         `_schedule_drain` so
302         a single-worker box keeps making progress with no central timer (the DB is the
303         clock).
304         Returns the number of jobs that reached a terminal/parked state this tick.
305         """
306         ran = 0
307         while True:
308             job = db_instance.claim_due_job()
309             if job is None:
310                 break
311             ran += 1
312             _run_claimed_job(job)
313         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
314         new
315         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
316         relies
317         # on the next submit/drain. Best-effort ? never raises into the worker.
318         _schedule_next_drain_if_waiting()
319         return ran
320
321     def _schedule_next_drain_if_waiting() -> None:
322         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
323         due, so a
324         self-scheduled job resumes with no further client submit. The drain itself re-checks
325         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
326         fine."""
327         try:
328             delay = db_instance.seconds_until_next_due()
329         except Exception as e: # pragma: no cover - defensive; never wedge the worker
330             logger.error(f"Could not compute next-due delay: {e}")
331             return
332         if delay is None:
333             return # nothing waiting
334         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
335         ness.
336         _arm_wake_timer(max(0.0, min(delay, _MAX_DRAIN_WAKE_SEC)))
337
338     def _arm_wake_timer(delay_sec: float) -> None:
339         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central

```

```

334         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
335         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting. """
336         timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
337         timer.daemon = True
338         timer.start()
339
340
341     #: A parked job further out than this is re-checked by a capped wake rather than one
long
342     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
343     _MAX_DRAIN_WAKE_SEC = 60.0
344
345
346     @app.post("/api/process", status_code=202)
347     def process_video(req: ProcessRequest):
348         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
349
350         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
351         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
352         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
353         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
354         """
355         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
356         try:
357             validate_input_path(req.video_path, allowed_root=allowed_root)
358         except ValueError as e:
359             raise HTTPException(status_code=400, detail=str(e)) from e
360         if not os.path.exists(req.video_path):
361             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
362         if req.segmenter and not segmenter_registry.get(req.segmenter):
363             available = list(segmenter_registry.list_available().keys())
364             raise HTTPException(
365                 status_code=400,
366                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
367         )
368
369         job_id = uuid.uuid4().hex
370         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
371                                     req.player_pool, req.max_frames):
372             raise HTTPException(status_code=500, detail="Failed to persist job record.")
373         _get_executor().submit(_drain_due_jobs)
374         return JSONResponse(
375             status_code=202,
376             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
377         )
378
379
380     @app.get("/api/jobs/{job_id}")
381     def get_job(job_id: str):
382         """Job state: {status, progress, result, reports}. `status` = execution state
383         (queued|running|done|failed); the pipeline verdict is result["status"]. """
384         job = db_instance.get_job(job_id)
385         if not job:
386             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
387         result = job.get("result")
388         return {
389             "job_id": job["id"],
390             "status": job["status"],
391             "progress": job.get("progress"),
392             "video_id": job.get("video_id"),

```

```

393         "error": job.get("error"),
394         "result": result,
395         "reports": (result or {}).get("reports"),
396         "created_at": job.get("created_at"),
397         "started_at": job.get("started_at"),
398         "finished_at": job.get("finished_at"),
399     }
400
401     # --- Chunked upload (B2, PR-2) -----
402     # Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
403     # proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
404     # under security.upload_dir, then submitted to /api/process by the returned path.
405     # Upload state is in-process by design (this is the single-box alpha): a server restart
406     # aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
407     # arrives with PR-3 identity scoping.
408
409     _uploads: Dict[str, Dict[str, Any]] = {}
410     _uploads_lock = threading.Lock()
411
412
413     class UploadInitRequest(BaseModel):
414         filename: str
415         total_size_bytes: Optional[int] = None
416
417
418     class UploadCompleteRequest(BaseModel):
419         total_parts: int
420
421
422     def _upload_cfg():
423         sec = getattr(global_config, "security", None)
424         upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
425         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
426         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
427         exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
428         None) or ["mp4", "mov"])]
429         return upload_dir, max_bytes, part_bytes, exts
430
431
432     def _abort_upload(upload_id: str) -> None:
433         with _uploads_lock:
434             st = _uploads.pop(upload_id, None)
435             if st:
436                 try:
437                     os.remove(st["tmp_path"])
438                 except OSError:
439                     pass
440
441
442     @app.post("/api/upload/init")
443     def upload_init(req: UploadInitRequest):
444         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
445         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
446         try:
447             name = safe_output_filename(req.filename)
448         except ValueError as e:
449             raise HTTPException(status_code=400, detail=str(e)) from e
450         ext = os.path.splitext(name)[1].lower().lstrip(".")
451         if ext not in exts:
452             raise HTTPException(status_code=400,
453             detail=f"Extension '{ext}' not allowed. Allowed:
454             {sorted(exts)}")
455         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
456             raise HTTPException(status_code=413,
457             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB

```



```

upload limit.")
456     os.makedirs(upload_dir, exist_ok=True)
457     upload_id = uuid.uuid4().hex
458     tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
459     open(tmp_path, "wb").close()
460     with _uploads_lock:
461         _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
462     return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
463           "next_part": f"/api/upload/{upload_id}/part/0"}
464
465
466 @app.put("/api/upload/{upload_id}/part/{index}")
467 async def upload_part(upload_id: str, index: int, request: Request):
468     """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
469     _, max_bytes, part_bytes, _ = _upload_cfg()
470     with _uploads_lock:
471         st = _uploads.get(upload_id)
472         if st is None:
473             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
474         if index != st["parts"]:
475             raise HTTPException(status_code=409,
476                               detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
477
478         received = 0
479         overflow = None
480         with open(st["tmp_path"], "ab") as fh:
481             async for chunk in request.stream():
482                 received += len(chunk)
483                 if received > part_bytes:
484                     overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
485                     break
486                 if st["bytes"] + received > max_bytes:
487                     overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
488                     break
489                 fh.write(chunk)
490         if overflow:
491             _abort_upload(upload_id)
492             raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
493
494         with _uploads_lock:
495             st["parts"] += 1
496             st["bytes"] += received
497         return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
498
499
500 @app.post("/api/upload/{upload_id}/complete")
501 def upload_complete(upload_id: str, req: UploadCompleteRequest):
502     """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
503     upload_dir, _, _, _ = _upload_cfg()
504     with _uploads_lock:
505         st = _uploads.get(upload_id)
506         if st is None:
507             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
508         if st["parts"] == 0 or st["parts"] != req.total_parts:
509             raise HTTPException(status_code=409,
510                               detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
511         final_path = os.path.join(upload_dir, st["filename"])

```

```

512         if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
513             stem, ext = os.path.splitext(st["filename"])
514             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
515         os.replace(st["tmp_path"], final_path)
516         with _uploads_lock:
517             _uploads.pop(upload_id, None)
518         video_id = compute_video_id(final_path)
519         return {"path": os.path.abspath(final_path), "video_id": video_id,
520               "size_bytes": os.path.getsize(final_path),
521               "next": "POST /api/process with this path"}
522
523
524     @app.delete("/api/upload/{upload_id}")
525     def upload_abort(upload_id: str):
526         """Abort an in-flight upload and delete its partial file."""
527         with _uploads_lock:
528             known = upload_id in _uploads
529         if not known:
530             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
531         _abort_upload(upload_id)
532         return {"status": "aborted", "upload_id": upload_id}
533
534
535     # --- Highlight download (B3, PR-2) -----
536
537     @app.get("/api/highlights/{video_id}/{filename}")
538     def download_highlight(video_id: str, filename: str):
539         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
540         (which only writes into output_dir; the browser previously got back an unreachable
541         server-local path)."""
542         try:
543             name = safe_output_filename(filename)
544         except ValueError as e:
545             raise HTTPException(status_code=400, detail=str(e)) from e
546         if not db_instance.get_video(video_id):
547             raise HTTPException(status_code=404, detail="Indexed video record not found.")
548         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
549         path = os.path.join(output_dir, name)
550         try:
551             path = resolve_under_root(path, output_dir)
552         except ValueError as e:
553             raise HTTPException(status_code=400, detail=str(e)) from e
554         if not os.path.isfile(path):
555             raise HTTPException(status_code=404,
556                               detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
557         media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
558         return FileResponse(path, media_type=media, filename=name)
559
560
561     @app.post("/api/query")
562     def query_play_segments(req: QueryRequest):
563         filters = {
564             "server": req.server,
565             "receiver": req.receiver,
566             "duration_min": req.duration_min,
567             "event_type": req.event_type
568         }
569         segments = db_instance.query_play_segments(req.video_id, filters)
570         return {"play_segments": segments}
571
572     @app.post("/api/compile")

```

```

573     def compile_highlights(req: CompileRequest):
574         video = db_instance.get_video(req.video_id)
575         if not video:
576             raise HTTPException(status_code=404, detail="Indexed video record not found.")
577
578         # Retrieve matching play segments from db
579         all_segments = db_instance.query_play_segments(req.video_id, {})
580         matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
581
582         if not matched_segments:
583             raise HTTPException(status_code=400, detail="No matching play segments found to
584 stitch.")
585
586         # Reject path traversal / absolute output names (os.path.join would otherwise let an
587 # absolute or '..'-laden output_filename write anywhere on disk).
588         try:
589             out_name = safe_output_filename(req.output_filename)
590         except ValueError as e:
591             raise HTTPException(status_code=400, detail=str(e)) from e
592
593         # The configured shot-count floor (stitching.min_shot_count) applies on the API
594 # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
595 # metadata are exempt: the floor filters known counts only, so explicitly
596 # requested manual/legacy segments can't silently vanish under the default floor.
597         stitching = getattr(global_config, "stitching", None) or StitchingConfig()
598         kept = []
599         for s in matched_segments:
600             meta = s.get("metadata") or {}
601             sc = meta.get("shot_count") if isinstance(meta, dict) else None
602             try:
603                 if sc is not None and int(sc) < stitching.min_shot_count:
604                     continue
605             except (TypeError, ValueError):
606                 pass # unparseable count -> treat as unknown, keep
607             kept.append(s)
608         if len(kept) < len(matched_segments):
609             logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
610 dropped "
611 f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
612 segments below the floor.")
613         if not kept:
614             raise HTTPException(status_code=400,
615 detail=f"All {len(matched_segments)} matching segments fall
616 below "
617 f"stitching.min_shot_count={stitching.min_shot_count}.")
618
619         # Extract start/end time pairs
620         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
621
622         # Run compiler with the configured stitching paddings + optional branding watermark
623         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
624         or "output"
625         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
626 end_padding=stitching.end_padding,
627 watermark=stitching.watermark)
628         try:
629             out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
630             return {"status": "success", "filepath": out_path}
631         except Exception as e:
632             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
633 {str(e)}") from e
634
635     # --- CLI Debug Utility & Orchestration ---

```

```

629 def run_cli_diagnose_clip(video_path: str, timestamp: float):
630     """CLI debugging utility to examine segment properties around a timestamp."""
631     print("=" * 60)
632     print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
633     print("=" * 60)
634
635     cfg = load_config() # legacy wrapper still works, returns dict for now
636
637     # 1. Validation check diagnostics
638     print("[DIAGNOSTIC] Running validation framework...")
639     results = registry.run_all(video_path, cfg)
640     for r in results:
641         status_symbol = "[PASS]" if r.passed else "[FAIL]"
642         print(f" {status_symbol} {r.validator_name}: {r.message}")
643         if r.details:
644             print(f"         Details: {r.details}")
645
646     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
647     print("-" * 60)
648     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
649     import cv2
650     cap = cv2.VideoCapture(video_path)
651     if not cap.isOpened():
652         print("[FAIL] OpenCV could not open video.")
653         return
654
655     fps = cap.get(cv2.CAP_PROP_FPS)
656     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
657
658     start_frame = max(0, int((timestamp - 3.0) * fps))
659     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
660
661     # Measure frame-by-frame changes in sample region
662     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
663     prev_gray = None
664     motions = []
665
666     for _ in range(start_frame, end_frame):
667         ret, frame = cap.read()
668         if not ret:
669             break
670         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
671         gray = cv2.GaussianBlur(gray, (21, 21), 0)
672
673         if prev_gray is not None:
674             diff = cv2.absdiff(prev_gray, gray)
675             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
676             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
677             motions.append(motion_ratio)
678             prev_gray = gray
679
680     cap.release()
681
682     if motions:
683         avg_motion = sum(motions) / len(motions)
684         # Support both legacy dict (from wrapper) and future direct model
685         if hasattr(cfg, "indexing"):
686             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
687         else:
688             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
689         print(f" Sample Frame Count: {len(motions)}")
690         print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")

```

```

691         if avg_motion > threshold:
692             print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
693         else:
694             print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
695     else:
696         print(" [ERROR] No visual frames were extracted in the sample range.")
697
698     print("=" * 60)
699
700     def main():
701         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
702         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
703         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
704         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
705         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
706         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
707         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
708         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
709
710         available_segmenters = list(segmenter_registry.list_available().keys())
711         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
712         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
713         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
714         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
715
716         args = parser.parse_args()
717
718         if args.setup:
719             # Invoke the diagnostics/bootstrap script (renamed from the old root
720             # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
721             # build system). Resolve its path from this file so `indexer --setup`
722             # works regardless of the caller's cwd.
723             import subprocess
724             doctor_path = os.path.join(
725                 os.path.dirname(os.path.dirname(os.path.abspath(__file__))),
726                 "scripts", "doctor.py",
727             )
728             print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
729             subprocess.run([sys.executable, doctor_path])
730             return
731
732         if args.trim_start is not None and args.trim_end is not None:
733             if not args.video_path:
734                 print("[ERROR] Please provide --video-path to extract a segment.")
735                 return
736
737             # Determine output path
738             out_filename = args.trim_output or "trimmed_segment.mp4"
739             if os.path.isabs(out_filename):
740                 out_path = out_filename
741                 out_dir = os.path.dirname(out_path)
742                 out_name = os.path.basename(out_path)

```

```

743         else:
744             out_dir = args.output_dir
745             out_path = os.path.join(out_dir, out_filename)
746             out_name = out_filename
747
748             os.makedirs(out_dir, exist_ok=True)
749             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
750
751             # global_config isn't initialized this early; load the typed config so the trim
752             # clip carries the same configured stitching paddings + watermark as a real
753             # highlight (watermark default-on).
754             stitching = load_app_config().stitching
755             compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
756                                     end_padding=stitching.end_padding,
watermark=stitching.watermark)
757             try:
758                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
759                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
760             except Exception as e:
761                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
762             return
763
764         if args.debug_clip is not None:
765             if not args.video_path:
766                 print("[ERROR] Please provide --video-path to debug a specific clip.")
767             return
768             run_cli_diagnose_clip(args.video_path, args.debug_clip)
769             return
770
771         # Initialize Global Config and DB
772         global global_config, db_instance
773         global_config = load_app_config() # returns typed AppConfig
774
775         # The CLI --output-dir overrides the configured output directory. It now lives
776         # on the typed model (OutputConfig.output_dir), so every consumer ? including
777         # /api/compile ? reads the same value instead of a write-only runtime hack.
778         global_config.output.output_dir = args.output_dir
779         os.makedirs(global_config.output.output_dir, exist_ok=True)
780
781         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
782         db_instance = Database(db_path)
783
784         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
785         # the executor is in-process. Mark them failed so pollers see a terminal state.
786         swept = db_instance.fail_stale_jobs()
787         if swept:
788             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
789
790         # CLI processing mode (no web UI needed)
791         if args.video_path and not args.server:
792             print(f"[CLI] Processing video file: {args.video_path}")
793             if not os.path.exists(args.video_path):
794                 print(f"[FAIL] Video file not found: {args.video_path}")
795             return
796
797             video_id = compute_video_id(args.video_path)
798             was_reingest = db_instance.get_video(video_id) is not None
799
800             # Validate (with-context variant surfaces ffmpeg_meta for the report)
801             print(f"[CLI] Validating...")
802             val_results, val_context = registry.run_all_with_context(args.video_path,

```

```

global_config)
803     failures = [r for r in val_results if not r.passed]
804
805     # Save video status
806     status = "failed" if failures else "processed"
807     db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
808
809     seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
810     segmenter_actual = None
811     segmentation_ran = False
812     try:
813         print("\n" + "="*40)
814         print("VALIDATION SUMMARY")
815         print("="*40)
816         for r in val_results:
817             status_symbol = "[PASS]" if r.passed else "[FAIL]"
818             print(f"{status_symbol} {r.validator_name}: {r.message}")
819         print("="*40 + "\n")
820
821         if failures:
822             print("[FAIL] Video failed checks. Indexing halted.")
823             return
824
825         # Index
826         segmenter_cls = segmenter_registry.get(seg_name)
827         if not segmenter_cls:
828             print(f"[FAIL] Segmenter '{seg_name}' not found.")
829             return
830
831         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
832         segmenter_actual = seg_name
833         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
834         segmenter = segmenter_cls(db_instance, global_config)
835         result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
836         segmentation_ran = True
837
838         if isinstance(result, Failure):
839             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
840             print("[FAIL] Indexing halted for video.")
841             segments = []
842         else:
843             segments = result.value if hasattr(result, "value") else result
844             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
845             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
846             _finalize_reingest(video_id, segments, play_wm, cand_wm)
847         finally:
848             # Always emit the per-video observability report. Never raises.
849             paths = emit_indexer_report(
850                 db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
851                 val_results=val_results, val_context=val_context,
852                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
853                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
854             )
855         if paths:
856             print(f"[REPORT] Technical report : {paths.get('technical_md')}")
857             print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
858             print(f"[REPORT] Machine JSON      : {paths.get('json')}")
859     return
860

```

```

861         # Start Local Server Mode
862         if args.server or not args.video_path:
863             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
864             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
865             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
866             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
867             _get_executor().submit(_drain_due_jobs)
868             uvicorn.run(app, host="127.0.0.1", port=args.port)
869
870     if __name__ == "__main__":
871         main()
872

```

39. assistant (2026-06-18T01:38:32.140Z)

I want to look closely at one thing in the upload flow. In `upload\_part`, when overflow occurs mid-stream, the code has already written some chunks to the file before hitting the limit, but `\_abort\_upload` removes the file, so that's fine. However, there's a subtle issue: the `received` counter resets per part, but the overflow check `st["bytes"] + received > max\_bytes` is correct. Let me look at the paths util (the security boundary) since that's the externally-reachable validation.

40. user (2026-06-18T01:38:33.102Z)

```

1      """Path-safety helpers for the API boundary (untrusted request fields)."""
2      import os
3      from typing import Optional
4
5
6      def safe_output_filename(name: str) -> str:
7          """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
8
9          `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
10         and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
11         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
12         """
13         if not name or not name.strip():
14             raise ValueError("output_filename must not be empty")
15         if "\x00" in name:
16             raise ValueError("output_filename must not contain a null byte")
17         if "/" in name or "\\" in name:
18             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
19         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
20             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
21         return name
22
23
24     def resolve_under_root(path: str, root: str) -> str:
25         """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
26         rp = os.path.realpath(path)
27         rr = os.path.realpath(root)
28         try:
29             contained = os.path.commonpath([rp, rr]) == rr
30         except ValueError: # different drives on Windows
31             contained = False
32         if not contained:
33             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
34         return rp

```



```

35
36
37 def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
38     """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
39
40     With no allowed_root (default, single-user local mode) only the null-byte check
41     applies,
42     so the local 'point it at any of my videos' workflow is preserved. Setting an
43     allowed_root (a future hosted/tenant deployment) enforces containment.
44     """
45     if "\x00" in path:
46         raise ValueError("video_path must not contain a null byte")
47     if allowed_root:
48         resolve_under_root(path, allowed_root)

```

41. assistant (2026-06-18T01:38:36.477Z)

These boundaries are carefully built. Let me look at the audio hit_detector and the gemini segmenter's parse_time, plus the wasb_runner which has trajectory math. Let me also check `windowing.py` and `harness.py`.

42. user (2026-06-18T01:38:37.436Z)

```

1     """Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).
2
3     Stages 0?1 of docs/AUDIO_RALLY_DETECTION_PLAN.md: extract the audio track, build a
4     spectral-flux onset envelope (with pre-emphasis to boost the broadband HF transient
5     a racket?shuttle contact makes), threshold it adaptively, and return (timestamp,
6     strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
7     only for extraction. Consumed by the E1 go/no-go probe; later feeds rally fusion
8     (ADR-007) if E1 passes. WASB is never modified ? this is a separate signal.
9     """
10    from __future__ import annotations
11
12    import os
13    import wave
14    import tempfile
15    import subprocess
16    from dataclasses import dataclass
17    from typing import List, Optional, Tuple
18
19    import numpy as np
20
21
22    @dataclass
23    class HitEvent:
24        t: float          # seconds
25        strength: float    # onset-envelope peak value (arbitrary units)
26
27
28    def extract_audio(video_path: str, out_wav: Optional[str] = None, sr: int = 16000) ->
29    str:
30        """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
31        Audio-only decode, so it's fast even for multi-GB video."""
32        if out_wav is None:
33            fd, out_wav = tempfile.mkstemp(suffix=".wav")
34            os.close(fd)
35            cmd = ["ffmpeg", "-y", "-i", video_path, "-vn", "-ac", "1", "-ar", str(sr),
36                  "-c:a", "pcm_s16le", out_wav]
37            subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
38                          stderr=subprocess.DEVNULL)
39            return out_wav

```

```

40     def load_wav(path: str) -> Tuple[np.ndarray, int]:
41         """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
42         with wave.open(path, "rb") as w:
43             sr, n, sw, ch = w.getframerate(), w.getnframes(), w.getsampwidth(),
w.getnchannels()
44             raw = w.readframes(n)
45             if sw == 2:
46                 a = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768.0
47             elif sw == 4:
48                 a = np.frombuffer(raw, dtype=np.int32).astype(np.float32) / 2147483648.0
49             elif sw == 1:
50                 a = np.frombuffer(raw, dtype=np.uint8).astype(np.float32) / 128.0 - 1.0
51             else:
52                 raise ValueError(f"unsupported sample width {sw}")
53             if ch > 1:
54                 a = a.reshape(-1, ch).mean(axis=1)
55             return a, sr
56
57
58     def _preemphasis(x: np.ndarray, coef: float = 0.97) -> np.ndarray:
59         if len(x) < 2:
60             return x.copy()
61         return np.append(x[0], x[1:] - coef * x[:-1])
62
63
64     def onset_envelope(samples: np.ndarray, sr: int, frame: int = 1024,
65                         hop: int = 256, band: Optional[Tuple[float, float]] = None) ->
Tuple[np.ndarray, float]:
66         """Spectral-flux onset strength per hop (pure-numpy STFT). Returns (env,
hop_seconds).
67
68         `band=(lo_hz, hi_hz)` restricts the flux to a frequency band ? a shuttle "thwack"
69         is HF-rich while voices/footsteps live lower, so band-limiting (e.g. 3-9 kHz)
70         suppresses non-shuttle transients. Needs sr high enough for hi_hz (Nyquist =
sr/2)."""
71         x = _preemphasis(np.asarray(samples, dtype=np.float32))
72         hop_sec = hop / float(sr)
73         if len(x) < frame:
74             return np.zeros(0, dtype=np.float32), hop_sec
75         win = np.hanning(frame).astype(np.float32)
76         n_frames = 1 + (len(x) - frame) // hop
77         idx = np.arange(frame)[None, :] + hop * np.arange(n_frames)[: , None]
78         frames = x[idx] * win
79         mag = np.abs(np.fft.rfft(frames, axis=1))
80         if band is not None:
81             freqs = np.fft.rfftfreq(frame, d=1.0 / sr)
82             keep = (freqs >= band[0]) & (freqs <= band[1])
83             if keep.any():
84                 mag = mag[:, keep] # band-limited flux
85             diff = np.diff(mag, axis=0) # frame-to-frame change
86             flux = np.sqrt((np.maximum(diff, 0.0) ** 2).sum(axis=1)) # positive (onset) flux
87             env = np.concatenate([[0.0], flux]).astype(np.float32) # align to n_frames
88             return env, hop_sec
89
90
91     def _moving_stats(env: np.ndarray, win: int) -> Tuple[np.ndarray, np.ndarray]:
92         """Sliding-window mean and std (reflect-padded, via cumsum). win forced odd."""
93         if win % 2 == 0:
94             win += 1
95         pad = win // 2
96         p = np.pad(env.astype(np.float64), pad, mode="reflect")
97         c1 = np.cumsum(np.insert(p, 0, 0.0))
98         c2 = np.cumsum(np.insert(p * p, 0, 0.0))
99         s1 = c1[win:] - c1[:-win]
100        s2 = c2[win:] - c2[:-win]

```

```

101     mean = s1 / win
102     var = np.maximum(s2 / win - mean * mean, 0.0)
103     return mean[:len(env)], np.sqrt(var)[:len(env)]
104
105
106     def detect_hits(samples: np.ndarray, sr: int, k: float = 2.5, win_frames: int = 41,
107                     min_gap_sec: float = 0.08, band: Optional[Tuple[float, float]] = None)
-> List[HitEvent]:
108         """Detect impact transients: local maxima of the onset envelope that exceed an
109         adaptive threshold (moving mean + k*std), keeping the strongest within
110         `min_gap_sec`.
111         `k` is the sensitivity knob (lower = more hits); `band` band-limits the onset flux
112         to the shuttle-click frequencies (e.g. (3000, 9000))."""
113         env, hop_sec = onset_envelope(samples, sr, band=band)
114         if len(env) < 3:
115             return []
116         mean, std = _moving_stats(env, win_frames)
117         thr = mean + k * std
118         is_peak = (env[1:-1] >= env[:2]) & (env[1:-1] > env[2:]) & (env[1:-1] > thr[1:-1])
119         peak_idx = np.nonzero(is_peak)[0] + 1
120         if peak_idx.size == 0:
121             return []
122         # enforce min inter-peak gap: greedily keep strongest first
123         min_gap = max(1, int(round(min_gap_sec / hop_sec)))
124         order = peak_idx[np.argsort(-env[peak_idx])]
125         taken = np.zeros(len(env), dtype=bool)
126         kept = []
127         for i in order:
128             lo, hi = max(0, i - min_gap), min(len(env), i + min_gap + 1)
129             if taken[lo:hi].any():
130                 continue
131             taken[i] = True
132             kept.append(i)
133         kept.sort()
134         return [HitEvent(t=float(i * hop_sec), strength=float(env[i])) for i in kept]
135
136     def detect_hits_in_video(video_path: str, sr: int = 16000, **kw) -> List[HitEvent]:
137         """Convenience: extract audio from a video and detect hits. Cleans up the temp
138         wav."""
139         wav = extract_audio(video_path, sr=sr)
140         try:
141             samples, real_sr = load_wav(wav)
142             return detect_hits(samples, real_sr, **kw)
143         finally:
144             try:
145                 os.remove(wav)
146             except OSError:
147                 pass

```

43. user (2026-06-18T01:38:37.896Z)

```

1     """Single-sourced candidate-windowing presets ? the eval?serve foundation fix.
2
3     **The bug this closes.** The offline eval substrate (`distill_local.build_candidates`
4     ?
5     `fusion_golden` ? the golden feature JSONs the ablation/compare path re-scores) was
6     built
7     on a *recall-oriented PROPOSAL* windowing (`vt=0.005, merge_gap=1.0, min_window=0.5`):
8     deliberately permissive so a learned scorer/gate can filter the junk. But the SERVED
9     pipeline
10    (`TrajectoryHybridSegmenter.process_video`) windows COARSELY (`vt=0.02,
11    merge_gap=2.0,
12    min_window=2.0` ? the `config.indexing` defaults). A cue measured on the proposal set

```

```

is
9     measured on candidates **production never serves**, so a "win" need not transfer. This
is
10    exactly how Gate-A scored **+0.074** on the proposal set yet **?0.008** on the served
set
11    (`scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md` vs `output/gateA_served_verify/`).
12
13    **The fix.** One module owns the two named windowings as :class:`WindowingPreset`s, and
the
14    SERVED preset is **single-sourced from the same Pydantic config defaults production
reads**
15    (:class:`backend.config.models.IndexingConfig`) ? eval and serve can no longer drift.
Helpers
16    build candidates under either preset, and :func:`served_windowing_from_config` lifts the
17    *actual* served windowing out of a live config (overrides included), so an A/B can
target the
18    operating point a given deployment truly serves.
19
20    The companion rule ? *a cue is promotable only if it does NOT regress at the SERVED
21    windowing* ? lives in :mod:`backend.eval.promotion` (which composes ``eval_stats`` +
22    ``per_user_gate``). This module is the **what windowing**; that one is the **promotion
gate**.
23
24    Pure, offline, stdlib + config only. Nothing here changes existing eval behaviour until
a
25    caller opts in (`distill_local` re-points its module-level presets here, byte-
identically).
26    """
27    from __future__ import annotations
28
29    from dataclasses import dataclass
30    from typing import Any, Dict, List, Mapping, Tuple
31
32    from backend.config.models import IndexingConfig
33
34    Interval = Tuple[float, float]
35
36    __all__ = [
37        "WindowingPreset",
38        "SERVED",
39        "PROPOSAL",
40        "PRESETS",
41        "served_windowing_from_config",
42        "build_windows",
43        "windows_to_intervals",
44    ]
45
46
47    @dataclass(frozen=True)
48    class WindowingPreset:
49        """A named (velocity_thresh, merge_gap, min_window_duration) windowing operating
point.
50
51        ``velocity_thresh`` ? normalised per-second shuttle velocity above which a frame
counts as
52        "in flight"; ``merge_gap`` ? seconds within which active frames coalesce into one
window;
53        ``min_window_duration`` ? windows shorter than this are dropped. These are exactly
the three
54        knobs ``TrackNetRunner.trajectory_to_action_windows`` takes, so a preset *is* the
windowing.
55        """
56
57        name: str
58        velocity_thresh: float

```

```

59         merge_gap: float
60         min_window_duration: float
61         #: 0 = OFF (default). When > 0, windows longer than this are split at their largest
internal
62         #: inactivity gap ? the bounded-windowing over-merge fix (W1,
RALLY_QUALITY_RESEARCH.md §6).
63         max_window_duration: float = 0.0
64         #: When True, ``max_window_duration`` is a HARD cap ? an over-long window with no
internal
65         #: inactivity gap is chopped at its midpoint until ? the cap (continuously-active
fix). OFF by default.
66         hard_cap: bool = False
67         #: R4 rally-END inactivity trim (s, 0 = OFF): trim a window's post-rally slow-drift
tail back to
68         #: the last frame whose trailing window stays dense with FAST motion. The measured
rally-end fix.
69         inactivity_end: float = 0.0
70         #: velocity above which motion counts as "rally" (vs slow drift) for the R4 trim;
min fast-fraction.
71         inactivity_rally_thresh: float = 0.08
72         inactivity_min_density: float = 0.34
73         #: R5 blob-fallback (default OFF). After the cap split + R4 trim, midpoint-chop any
group still
74         #: longer than ``blob_factor`` × ``max_window_duration`` (the gap-less, R4-survived
blob ?
75         #: mahadevpura-1's ~11× residual) and flag it. Runs AFTER R4 (vs ``hard_cap``
before) and the
76         #: factor keeps it surgical (a normal long rally is left alone).
77         blob_fallback: bool = False
78         blob_factor: float = 4.0
79
80         def as_tuple(self) -> Tuple[float, float, float]:
81             """``(velocity_thresh, merge_gap, min_window_duration)`` ? the legacy tuple
shape the
82             ``trajectory_to_action_windows`` / ``distill_local`` call sites already speak.
(The
83             bounded-windowing knob ``max_window_duration`` is passed separately by
``build_windows``.)"""
84             return (self.velocity_thresh, self.merge_gap, self.min_window_duration)
85
86         def to_dict(self) -> Dict[str, Any]:
87             return {
88                 "name": self.name,
89                 "velocity_thresh": self.velocity_thresh,
90                 "merge_gap": self.merge_gap,
91                 "min_window_duration": self.min_window_duration,
92                 "max_window_duration": self.max_window_duration,
93                 "hard_cap": self.hard_cap,
94                 "inactivity_end": self.inactivity_end,
95                 "inactivity_rally_thresh": self.inactivity_rally_thresh,
96                 "inactivity_min_density": self.inactivity_min_density,
97                 "blob_fallback": self.blob_fallback,
98                 "blob_factor": self.blob_factor,
99             }
100
101
102     def _served_preset_from_defaults() -> WindowingPreset:
103         """The SERVED windowing, lifted from the SAME Pydantic config defaults production
reads.
104
105         ``TrajectoryHybridSegmenter.process_video`` reads exactly these three:
106         - velocity: ``idx_cfg[<family>].velocity_threshold`` (default 0.02 ?
wasb/fusion/tracknet)
107         - merge_gap: ``idx_cfg.dead_time_merge_gap`` (default 2.0)
108         - min window: ``idx_cfg.min_action_window_duration`` (default 2.0)

```

```

109
110     Sourcing them from :class:`IndexingConfig`'s defaults (not a hand-typed tuple) is
the whole
111     point: change a served default in ``config/models.py`` and the eval SERVED preset
follows,
112     so the two can never silently diverge again.
113     """
114     idx = IndexingConfig() # construct with defaults ? the served operating point
115     return WindowingPreset(
116         name="served",
117         velocity_thresh=float(idx.wasb.velocity_threshold), # == fusion/tracknet
default 0.02
118         merge_gap=float(idx.dead_time_merge_gap),
119         min_window_duration=float(idx.min_action_window_duration),
120         max_window_duration=float(idx.max_window_duration), # W1 bounded-windowing
(default 0=OFF)
121         hard_cap=bool(idx.window_hard_cap), # W1 hard-cap fallback
(default OFF)
122         inactivity_end=float(idx.window_inactivity_end), # R4 rally-end trim
(default 0=OFF)
123         inactivity_rally_thresh=float(idx.inactivity_rally_thresh),
124         inactivity_min_density=float(idx.inactivity_min_density),
125         blob_fallback=bool(idx.window_blob_fallback), # R5 blob-fallback
(default OFF)
126         blob_factor=float(idx.window_blob_factor),
127     )
128
129
130     #: The COARSE windowing production actually serves (config defaults; ~0.02/2.0/2.0). A
cue is
131     #: only promotable if it holds up HERE ? the operating point real users get.
132     SERVED: WindowingPreset = _served_preset_from_defaults()
133
134     #: The recall-oriented PROPOSAL windowing the offline substrate is built on
(deliberately
135     #: permissive: propose many, let the classifier/gate filter). NOT what production serves
? a
136     #: win here must be re-confirmed on SERVED before promotion. Mirrors the historical
137     #: ``distill_local.PROPOSAL`` tuple (0.005/1.0/0.5) so the existing golden JSONs stay
valid.
138     PROPOSAL: WindowingPreset = WindowingPreset(
139         name="proposal", velocity_thresh=0.005, merge_gap=1.0, min_window_duration=0.5
140     )
141
142     #: Name ? preset, so a CLI / config string ("served" | "proposal") selects one.
143     PRESETS: Dict[str, WindowingPreset] = {SERVED.name: SERVED, PROPOSAL.name: PROPOSAL}
144
145
146     def served_windowing_from_config(config: Any, family: str = "wasb") -> WindowingPreset:
147         """The SERVED windowing a *specific* live config serves (overrides honoured).
148
149         Reproduces ``TrajectoryHybridSegmenter.process_video``'s knob reads exactly, against
an
150         arbitrary config (the typed :class:`AppConfig` or a plain dict) and segmenter
``family``
151         (``"wasb"`` / ``"fusion"`` / ``"tracknet"``). Use this when a deployment overrode a
served
152         default in ``config.json`` ? the module-level :data:`SERVED` is the *default*
operating point;
153         this is *this config's* operating point. Falls back to the served defaults for any
absent key.
154         """
155         idx = _as_mapping(config, "indexing")
156         family_cfg = _as_mapping(idx, family)
157         return WindowingPreset(

```

```

158         name=f"served:{family}",
159         velocity_thresh=float(family_cfg.get("velocity_threshold",
SERVED.velocity_thresh)),
160         merge_gap=float(idx.get("dead_time_merge_gap", SERVED.merge_gap)),
161         min_window_duration=float(idx.get("min_action_window_duration",
SERVED.min_window_duration)),
162         max_window_duration=float(idx.get("max_window_duration",
SERVED.max_window_duration)),
163         hard_cap=bool(idx.get("window_hard_cap", SERVED.hard_cap)),
164         inactivity_end=float(idx.get("window_inactivity_end", SERVED.inactivity_end)),
165         inactivity_rally_thresh=float(idx.get("inactivity_rally_thresh",
SERVED.inactivity_rally_thresh)),
166         inactivity_min_density=float(idx.get("inactivity_min_density",
SERVED.inactivity_min_density)),
167         blob_fallback=bool(idx.get("window_blob_fallback", SERVED.blob_fallback)),
168         blob_factor=float(idx.get("window_blob_factor", SERVED.blob_factor)),
169     )
170
171
172 def _as_mapping(obj: Any, key: str) -> Mapping[str, Any]:
173     """`obj[key]` as a plain mapping, whether `obj` is a dict or a typed config
section
174     (`AppConfig.get` returns nested sections as dicts). Empty mapping if absent/non-
mapping."""
175     if obj is None:
176         return {}
177     if isinstance(obj, Mapping):
178         sub = obj.get(key, {})
179     elif hasattr(obj, "get"):
180         sub = obj.get(key, {})
181     else:
182         sub = getattr(obj, key, {})
183     return sub if isinstance(sub, Mapping) else {}
184
185
186 def build_windows(points: List[Any], fps: float, frame_width: float,
187                 preset: WindowingPreset) -> List[Dict[str, Any]]:
188     """Trajectory `points` ? candidate windows under `preset` (the one windowing
call site).
189
190     A thin wrapper over `TrackNetRunner.trajectory_to_action_windows` that takes a
named preset
191     instead of three loose floats, so eval candidate-building and the served path
reference the
192     SAME knobs by name. Imported lazily to keep this module import-light (no detector
deps)."""
193     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
194
195     vt, mg, mwd = preset.as_tuple()
196     return TrackNetRunner.trajectory_to_action_windows(
197         points, fps=fps, frame_width=frame_width,
198         velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
199         max_window_duration=preset.max_window_duration,
200         window_hard_cap=preset.hard_cap,
201         window_inactivity_end=preset.inactivity_end,
202         inactivity_rally_thresh=preset.inactivity_rally_thresh,
203         inactivity_min_density=preset.inactivity_min_density,
204         window_blob_fallback=preset.blob_fallback,
205         window_blob_factor=preset.blob_factor)
206
207
208 def windows_to_intervals(windows: List[Dict[str, Any]]) -> List[Interval]:
209     """`[{start, end, ...}]` ? `[(start, end)]` ? the interval shape the scorer/gate
speak."""
210     return [(w["start"], w["end"]) for w in windows]

```

44. assistant (2026-06-18T01:38:44.049Z)

I noticed a potential issue in `\_moving\_stats` in `hit_detector.py`. When `win > len(env)`, the `reflect-pad` and `cumsum` slicing could go wrong. Let me think: `np.pad(p, pad, mode="reflect")` requires `pad < len(env)` for reflect mode (numpy raises if pad exceeds array length minus 1 in reflect mode... actually numpy reflect requires the pad width to be at most `len-1`). But `detect_hits` only calls `_moving_stats` when `len(env) >= 3`, with `win_frames=41` default. If `env` has length 3 and `win=41`, `pad=20`, and `np.pad(array_of_len_3, 20, mode="reflect")` raises a `ValueError`. Let me verify this is reachable and whether numpy actually raises.

Let me check numpy's reflect behavior and whether this path is reachable.

45. user (2026-06-18T01:38:45.820Z)

```
[0. 1. 2. 1. 0. 1. 2. 1. 0. 1. 2. 1. 0. 1. 2. 1. 0. 1. 2. 1.
 0. 1. 2. 1. 0. 1. 2. 1. 0. 1. 2. 1. 0. 1. 2. 1. 0. 1. 2.]
```

46. assistant (2026-06-18T01:38:51.102Z)

Modern numpy handles `reflect-pad` beyond array length fine (it just keeps reflecting). So that's not a bug. The `_moving_stats` slicing `mean[:len(env)]` is correct since the padded array is `len(env)+2*pad`, and after the windowed-sum there are `len(env)+2*pad-win+1 = len(env)+1` elements when `win=2*pad+1`; slicing to `len(env)` is correct. Good.

Let me look at the remaining unexamined modules: the `calibrate/harness` modules, `run_telemetry`, and the `wasb_runner`. Let me check `run_telemetry` and `geometry` (small utils, easy to have edge bugs) and `harness`.

47. user (2026-06-18T01:38:51.566Z)

```
1     import math
2     from typing import Tuple
3
4     def get_center(bbox: Tuple[float, float, float, float]) -> Tuple[float, float]:
5         """Calculate the center (x, y) of a bounding box.
6         bbox format: (x1, y1, x2, y2)
7         """
8         x1, y1, x2, y2 = bbox
9         return ((x1 + x2) / 2.0, (y1 + y2) / 2.0)
10
11    def get_distance(p1: Tuple[float, float], p2: Tuple[float, float]) -> float:
12        """Calculate the Euclidean distance between two points (x, y)."""
13        return math.sqrt((p2[0] - p1[0])**2 + (p2[1] - p1[1])**2)
14
15    def get_velocity(bbox1: Tuple[float, float, float, float], bbox2: Tuple[float, float,
float, float], fps: float) -> float:
16        """Calculate velocity (pixels per second) of a bounding box moving between two
consecutive frames.
17        """
18        c1 = get_center(bbox1)
19        c2 = get_center(bbox2)
20        dist = get_distance(c1, c2)
21        return dist * fps
22
23    def get_velocity_normalised(bbox1: Tuple[float, float, float, float], bbox2:
Tuple[float, float, float, float],
24                                fps: float, frame_width: float) -> float:
25        """Calculate velocity as a fraction of frame width per second.
26
27        This makes the threshold resolution-independent:
28        - 0.15 means the center moved 15% of the frame width in one second.
29        - Works identically on 720p, 1080p, and 4K footage.
```



```

30
31     Args:
32         bbox1: Previous bounding box (x1, y1, x2, y2).
33         bbox2: Current bounding box (x1, y1, x2, y2).
34         fps: Effective frames per second (after frame-skipping).
35         frame_width: Width of the video frame in pixels.
36     """
37     raw_velocity = get_velocity(bbox1, bbox2, fps)
38     if frame_width <= 0:
39         return raw_velocity # Fallback to raw pixels if width unknown
40     return raw_velocity / frame_width
41
42     def calculate_iou(boxA: Tuple[float, float, float, float], boxB: Tuple[float, float,
float, float]) -> float:
43         """Calculate the Intersection over Union (IoU) of two bounding boxes.
44         bbox format: (x1, y1, x2, y2)
45         """
46         xA = max(boxA[0], boxB[0])
47         yA = max(boxA[1], boxB[1])
48         xB = min(boxA[2], boxB[2])
49         yB = min(boxA[3], boxB[3])
50
51         interArea = max(0, xB - xA + 1) * max(0, yB - yA + 1)
52         if interArea == 0:
53             return 0.0
54
55         boxAArea = (boxA[2] - boxA[0] + 1) * (boxA[3] - boxA[1] + 1)
56         boxBArea = (boxB[2] - boxB[0] + 1) * (boxB[3] - boxB[1] + 1)
57
58         iou = interArea / float(boxAArea + boxBArea - interArea)
59         return iou
60

```

48. user (2026-06-18T01:38:52.096Z)

```

1     """Structured run telemetry ? a compact, rotating JSONL "black box" for long/flaky runs.
2
3     A long GPU/CV run (the fusion-golden extraction, a WSL/WASB pass, an ffmpeg stitch) can
die with
4     **no Python traceback** ? the OS kills it (a console-control event, an OOM, a thermal
event). When
5     that happens the only forensic evidence is whatever was last flushed to disk. This
module keeps a
6     tiny (~10 KB) rotating runlog of *execution velocity + machine health* so a post-mortem
is a
7     bullseye instead of a guess: "it was at 26 fps, GPU 85 °C throttled to 1125/2100 MHz,
4.8 GB VRAM
8     free, CPU 60 %, RAM 21/32 GB ? then vanished."
9
10    It is the **observability** companion to the `ExecutionPolicy` *control* layer
11    (`docs/EXECUTION_POLICY.md`): the policy decides wait / abort / resume; this records the
signals
12    that explain *why* a run slowed or died, and lets us compare execution velocity across
quality
13    iterations.
14
15    Design:
16    - **Caller-driven snapshots** (no background thread): call ``snapshot(done, total)``
from an
17    existing progress callback; fps is derived from the delta since the previous snapshot.
18    ``event(kind, msg)`` records discrete moments (start, video-done, error) WITH a health
sample, so
19    the last line before a kill captures the machine state at death.
20    - **Capped + rotating**: a fixed ``meta`` header line (machine / GPU / start) is always
kept; recent

```

```

21     snapshot/event lines form an on-disk ring trimmed to ``max_bytes`` (~10 KB). The file
is rewritten
22     atomically (tmp + ``os.replace``) on each write, so a hard kill never leaves it half-
written.
23     - **Graceful degradation**: GPU stats come from ``nvidia-smi`` (absent -> ``None``);
system stats
24     from ``psutil`` (not installed -> ``None``). Collection never raises into the caller ?
a telemetry
25     hiccup must not break the run it observes.
26     - **Deterministic / testable**: the clock and the two collectors are injectable, so fps,
rotation,
27     and parsing are unit-tested with no real GPU, no psutil, and no wall-clock.
28
29     All stdlib + optional ``psutil`` (BSD-3, commercial-clean). No network, no training-data
path.
30     """
31     from __future__ import annotations
32
33     import json
34     import os
35     import shutil
36     import subprocess
37     import time
38     from typing import Any, Callable, Dict, List, Optional
39
40     GpuFn = Callable[[], Optional[Dict[str, Any]]]
41     SysFn = Callable[[], Optional[Dict[str, Any]]]
42
43     # nvidia-smi fields pulled in one query, in this order (see gpu_stats parsing).
44     _GPU_QUERY =
"temperature.gpu,utilization.gpu,power.draw,clocks.sm,clocks.max.sm,memory.used,memory.free"
45     _THROTTLE_CLOCK_FRAC = 0.8 # SM clock below this fraction of max => flagged throttled
46     _THROTTLE_TEMP_C = 84.0 # ... or temperature at/above this (laptop Max-Q throttle
territory)
47
48
49     def _num(s: str) -> Optional[float]:
50         try:
51             return float(s)
52         except (TypeError, ValueError):
53             return None
54
55
56     def gpu_stats(timeout: float = 4.0) -> Optional[Dict[str, Any]]:
57         """One ``nvidia-smi`` sample as a dict, or ``None`` if nvidia-smi is unavailable /
errors.
58
59         Returns util %, temperature, power (W), current + max SM clock (MHz), VRAM used /
free (MB), and a
60         derived ``throttled`` flag (SM clock well below max, or hot). Pure read; never
raises.
61         """
62         exe = shutil.which("nvidia-smi")
63         if not exe:
64             return None
65         try:
66             out = subprocess.run(
67                 [exe, f"--query-gpu={_GPU_QUERY}", "--format=csv,noheader,nounits"],
68                 capture_output=True, text=True, timeout=timeout, check=True,
69                 ).stdout.strip()
70         except (subprocess.SubprocessError, OSError):
71             return None
72         row = out.splitlines()[0] if out else ""
73         parts = [p.strip() for p in row.split(",")]
74         if len(parts) < 7:

```

```

75         return None
76     vals = [_num(p) for p in parts[:7]]
77     temp, util, power, sm, sm_max, used, free = vals
78     throttled = bool(temp is not None and temp >= _THROTTLE_TEMP_C)
79     if sm is not None and sm_max:
80         throttled = throttled or sm < _THROTTLE_CLOCK_FRAC * sm_max
81     return {
82         "util_pct": util, "temp_c": temp, "power_w": power,
83         "sm_mhz": sm, "sm_max_mhz": sm_max,
84         "vram_used_mb": used, "vram_free_mb": free,
85         "throttled": throttled,
86     }
87
88
89     def system_stats() -> Optional[Dict[str, Any]]:
90         """CPU % + RAM via ``psutil``, or ``None`` if psutil isn't installed. Never
91         raises."""
92         try:
93             import psutil # optional dep (BSD-3); only used when present
94         except ImportError:
95             return None
96         try:
97             vm = psutil.virtual_memory()
98             return {
99                 "cpu_pct": psutil.cpu_percent(interval=None),
100                 "ram_used_gb": round(vm.used / 1e9, 2),
101                 "ram_total_gb": round(vm.total / 1e9, 2),
102                 "ram_pct": vm.percent,
103             }
104         except Exception: # noqa: BLE001 - psutil edge cases must not break the run
105             return None
106
107     def gpu_name() -> Optional[str]:
108         """The GPU model string (for the runlog header), or ``None`` without nvidia-smi."""
109         exe = shutil.which("nvidia-smi")
110         if not exe:
111             return None
112         try:
113             out = subprocess.run([exe, "--query-gpu=name", "--format=csv,noheader"],
114                                 capture_output=True, text=True, timeout=4.0,
115                                 check=True).stdout
116         except (subprocess.SubprocessError, OSError):
117             return None
118         return out.splitlines()[0].strip() if out else None
119
120     class RunTelemetry:
121         """A small rotating JSONL runlog of execution velocity + machine health.
122
123         Call ``snapshot(done, total, phase=...)`` from a progress callback and ``event(kind,
124         msg)`` at
125         milestones. The file holds a ``meta`` header line plus the most recent snapshot /
126         event lines
127         that fit within ``max_bytes`` (~10 KB), rewritten atomically on every write.
128         """
129
130         def __init__(self, path: str, label: str = "run", max_bytes: int = 10_000,
131                     gpu_fn: GpuFn = gpu_stats, sys_fn: SysFn = system_stats,
132                     clock: Callable[[], float] = time.time) -> None:
133             self.path = path
134             self.label = label
135             self.max_bytes = max_bytes
136             self._gpu_fn = gpu_fn
137             self._sys_fn = sys_fn

```

```

136         self._clock = clock
137         self._t0 = clock()
138         self._last_done: Optional[int] = None
139         self._last_t: Optional[float] = None
140         self._lines: List[str] = [] # recent body lines (on-disk ring, trimmed to
max_bytes)
141         self._meta = json.dumps({"meta": {
142             "label": label, "started": self._iso(self._t0), "pid": os.getpid(),
143             "gpu_name": gpu_name(), "cpu_count": os.cpu_count(),
144         }}, separators=(",", ":"))
145         self._write()
146
147     # -- public API -----
148     def snapshot(self, done: Optional[int] = None, total: Optional[int] = None,
149                 phase: Optional[str] = None, **extra: Any) -> Dict[str, Any]:
150         """Record a velocity + health sample; ``fps`` is derived from the previous
snapshot."""
151         now = self._clock()
152         rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
1)
153         if phase is not None:
154             rec["phase"] = phase
155         if done is not None:
156             rec["done"] = done
157             if total:
158                 rec["total"] = total
159                 rec["pct"] = round(100.0 * done / total, 1)
160             if self._last_done is not None and self._last_t is not None and now >
self._last_t:
161                 rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
162             self._last_done, self._last_t = done, now
163         rec.update(extra)
164         self._attach_health(rec)
165         self._append(rec)
166         return rec
167
168     def event(self, kind: str, msg: str = "", **extra: Any) -> Dict[str, Any]:
169         """Record a milestone (start, video_done, error, restart) WITH a health
sample."""
170         now = self._clock()
171         rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
1),
172                                "event": kind}
173         if msg:
174             rec["msg"] = msg
175         rec.update(extra)
176         self._attach_health(rec)
177         self._append(rec)
178         return rec
179
180     # -- internals -----
181     def _attach_health(self, rec: Dict[str, Any]) -> None:
182         gpu = self._safe(self._gpu_fn)
183         sys_ = self._safe(self._sys_fn)
184         if gpu is not None:
185             rec["gpu"] = gpu
186         if sys_ is not None:
187             rec["sys"] = sys_
188
189     def _append(self, rec: Dict[str, Any]) -> None:
190         self._lines.append(json.dumps(rec, separators=(",", ":")))
191         budget = self.max_bytes - len(self._meta) - 1
192         while len(self._lines) > 1 and sum(len(s) + 1 for s in self._lines) > budget:
193             self._lines.pop(0) # drop oldest body line; the meta header is always kept
194         self._write()

```

```

195
196     def _write(self) -> None:
197         try:
198             tmp = self.path + ".tmp"
199             with open(tmp, "w", encoding="utf-8") as f:
200                 f.write(self._meta + "\n")
201                 for s in self._lines:
202                     f.write(s + "\n")
203             os.replace(tmp, self.path)
204         except OSError:
205             pass # telemetry must never break the run it observes
206
207     @staticmethod
208     def _safe(fn: GpuFn) -> Optional[Dict[str, Any]]:
209         try:
210             return fn()
211         except Exception: # noqa: BLE001 - a collector hiccup must not break the run
212             return None
213
214     @staticmethod
215     def _iso(t: float) -> str:
216         return time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(t))
217

```

49. assistant (2026-06-18T01:38:57.453Z)

In `gpu\_stats`, line 77: `temp, util, power, sm, sm\_max, used, free = vals` ? but `\_GPU\_QUERY` order is
`temperature.gpu,utilization.gpu,power.draw,clocks.sm,clocks.max.sm,memory.used,memory.free`,
which maps to temp, util, power, sm, sm_max, used, free. That's correct.

These are all clean. Let me look at the remaining modules I haven't seen ? the gemini segmenter (parse_time, runtime path), wasb_runner, and the calibration modules. Let me also check `regression.py` (gt_hash) and `annotations.py` (parse_timestamp, the central parser).

50. user (2026-06-18T01:38:58.403Z)

```

1     """Load hand-annotated ground-truth rally timestamps from CSV.
2
3     Annotation file format (one file per video):
4
5         start,end
6         00:01:12,00:01:39
7         102.5,131.0
8
9     - A header row `start,end` is optional (auto-detected and skipped).
10    - Each timestamp is either decimal seconds (`102.5`) or colon-separated
11      `MM:SS` / `HH:MM:SS` (`00:01:12`). Both forms may be mixed in one file.
12    - Blank lines and `#` comment lines are ignored.
13    """
14
15    import csv
16    import logging
17    from typing import List, Tuple
18
19    logger = logging.getLogger(__name__)
20
21    Interval = Tuple[float, float]
22
23
24    def parse_timestamp(value: str) -> float:
25        """Parse a timestamp string to float seconds.
26
27        Accepts decimal seconds or colon-separated MM:SS / HH:MM:SS.
28        Raises ValueError on anything unparseable so bad annotations fail loudly.

```

```

29     """
30     value = value.strip()
31     if not value:
32         raise ValueError("empty timestamp")
33     if ":" in value:
34         parts = value.split(":")
35         if len(parts) > 3:
36             raise ValueError(f"too many ':' segments in timestamp '{value}'")
37         total = 0.0
38         for part in parts:                # most-significant first (H, M, S)
39             total = total * 60 + float(part)
40         return total
41     return float(value)
42
43
44 def load_annotations(csv_path: str) -> List[Interval]:
45     """Load and validate ground-truth rally intervals from a CSV file.
46
47     Delegates to the canonical loader (gt_loader.py, ADR-008 prereq), which accepts BOTH
48     golden conventions: positional/headered ``start,end`` (the collector export) and the
49     rally-annotator ``rally_number,start_time,end_time,...`` schema. Strict: raises
50     ValueError on malformed rows (bad timestamp, start >= end) so labeling mistakes
51     surface immediately rather than silently skewing scores. Returns intervals sorted by
52     start time.
53     """
54     from backend.eval.gt_loader import load_gt_intervals
55     intervals = load_gt_intervals(csv_path, strict=True)
56     logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
57     return intervals
58
59
60 def write_scaffold(csv_path: str) -> None:
61     """Write an empty annotation file with just the header, for hand-filling."""
62     with open(csv_path, "w", newline="") as f:
63         writer = csv.writer(f)
64         writer.writerow(["start", "end"])
65

```

51. user (2026-06-18T01:38:58.908Z)

```

1     """Regression runner ? verify the production tool against golden-set ground truth.
2
3     Implements the explicit ask: *"given a video, get a golden set created and verify
4     its precision and recall against the production tool."* This is deliberately a thin
5     wrapper over machinery that already exists:
6
7     ground truth ? AnnotationProvider (file / vendor / oracle) [backend.annotations]
8     predictions  ? the production pipeline's stored DB rows   [backend.eval.harness]
9     comparison   ? interval matching + P/R/F1                 [backend.eval.metrics]
10    baseline     ? a per-video snapshot of "known good" numbers [this module]
11
12    A "regression" is a precision/recall drop beyond `tolerance` versus the snapshotted
13    baseline. Tiers map to annotation providers: e.g. tier "file" ? FileProvider (CI),
14    later tier "vendor" ? HumanVendorProvider (authoritative), tier "auto" ? an
15    independent-lineage oracle (fast smoke alarm). See docs/GOLDEN_SET_IMPLEMENTATION.md.
16    """
17    import os
18    import json
19    import hashlib
20    import logging
21    from dataclasses import dataclass, asdict
22    from typing import Dict, List, Optional
23
24    from backend.infrastructure.database import Database
25    from backend.eval import metrics

```

```

26     from backend.eval.harness import get_predictions
27     from backend.annotations import annotation_registry
28
29     logger = logging.getLogger(__name__)
30
31     # Tracked location (NOT under the gitignored output/), so a fresh checkout / CI run can
32     # actually load a committed baseline to compare against.
33     DEFAULT_BASELINE_PATH = os.path.join("eval_baselines", "regression_baseline.json")
34     DEFAULT_TOLERANCE = 0.05
35
36
37     def gt_hash(ground_truth: List[metrics.Interval]) -> str:
38         """Stable short hash of the GT intervals ? detects a baseline taken against
different labels."""
39         norm = sorted((round(float(s), 3), round(float(e), 3)) for s, e in ground_truth)
40         return hashlib.sha1(json.dumps(norm).encode()).hexdigest()[:12]
41
42
43     @dataclass
44     class RegressionResult:
45         video_id: str
46         stage: str
47         iou_threshold: float
48         precision: float
49         recall: float
50         f1: float
51         # Baseline values compared against (None when no baseline existed yet).
52         baseline_precision: Optional[float]
53         baseline_recall: Optional[float]
54         tolerance: float
55         regressed: bool
56         reasons: List[str]
57         # True only when a baseline existed to compare against. Distinguishes a genuine PASS
58         # from "nothing to compare" so the CLI can refuse to silently green-light the
latter.
59         has_baseline: bool = False
60         gt_hash: Optional[str] = None
61
62         def as_dict(self) -> dict:
63             return asdict(self)
64
65
66     # ----- baseline store
67
68     def load_baselines(path: str = DEFAULT_BASELINE_PATH) -> Dict[str, dict]:
69         """Load the per-video baseline snapshot file (returns {} if absent)."""
70         if not os.path.isfile(path):
71             return {}
72         with open(path) as f:
73             return json.load(f)
74
75
76     def _baseline_key(video_id: str, stage: str) -> str:
77         return f"{video_id}::{stage}"
78
79
80     def save_baseline(video_id: str, stage: str, precision: float, recall: float,
81                      f1: float, path: str = DEFAULT_BASELINE_PATH,
82                      iou_threshold: Optional[float] = None, detector: Optional[str] = None,
83                      gt_fingerprint: Optional[str] = None) -> None:
84         """Snapshot a video/stage's current numbers as the new 'known good' baseline.
85
86         Records the iou_threshold, detector, and a GT fingerprint alongside the metrics so a
87         later run computed under different settings (or against different labels) is
detected

```

```

88         as incommensurable rather than silently compared.
89         """
90         baselines = load_baselines(path)
91         baselines[_baseline_key(video_id, stage)] = {
92             "video_id": video_id, "stage": stage,
93             "precision": precision, "recall": recall, "f1": f1,
94             "iou_threshold": iou_threshold, "detector": detector, "gt_hash": gt_fingerprint,
95         }
96         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
97         with open(path, "w") as f:
98             json.dump(baselines, f, indent=2, sort_keys=True)
99         logger.info("Saved baseline for %s (precision=%.3f recall=%.3f)",
100 _baseline_key(video_id, stage), precision, recall)
101
102 # ----- the runner
103
104 def regress(db: Database, video_id: str, ground_truth: List[metrics.Interval],
105            stage: str = "final", iou_threshold: float = 0.5,
106            detector: Optional[str] = None,
107            tolerance: float = DEFAULT_TOLERANCE,
108            baseline_path: str = DEFAULT_BASELINE_PATH) -> RegressionResult:
109     """Score stored predictions for one video against ground truth and compare to
baseline.
110
111     `ground_truth` is a list of (start, end) intervals ? typically obtained from an
112     AnnotationProvider (see `regress_with_provider`). A regression is flagged when
113     precision OR recall falls more than `tolerance` below the snapshotted baseline.
114     If no baseline exists yet, `regressed` is False (nothing to compare to) and the
115     caller can choose to snapshot the current numbers via `save_baseline`.
116     """
117     preds = get_predictions(db, video_id, stage, detector=detector)
118     s = metrics.score(preds, ground_truth, iou_threshold)
119     fp = gt_hash(ground_truth)
120
121     baselines = load_baselines(baseline_path)
122     b = baselines.get(_baseline_key(video_id, stage))
123
124     reasons: List[str] = []
125     regressed = False
126     has_baseline = b is not None
127     base_p = base_r = None
128     if b is not None:
129         base_p, base_r = b["precision"], b["recall"]
130         # Incommensurable comparison guard: a baseline taken at a different IoU/detector
or
131         # against different labels is not a valid reference ? flag it instead of
trusting it.
132         b_iou, b_det, b_hash = b.get("iou_threshold"), b.get("detector"),
b.get("gt_hash")
133         if b_iou is not None and abs(float(b_iou) - iou_threshold) > 1e-9:
134             regressed = True
135             reasons.append(f"baseline IoU {b_iou} != run IoU {iou_threshold}
(incommensurable)")
136         if b_det is not None and b_det != detector:
137             regressed = True
138             reasons.append(f"baseline detector {b_det!r} != run detector {detector!r}
(incommensurable)")
139         if b_hash is not None and b_hash != fp:
140             regressed = True
141             reasons.append(f"baseline GT hash {b_hash} != run GT hash {fp} (labels
changed)")
142         if s.precision < base_p - tolerance:
143             regressed = True
144             reasons.append(f"precision {s.precision:.3f} < baseline {base_p:.3f} -

```



```

{tolerance}")
145         if s.recall < base_r - tolerance:
146             regressed = True
147             reasons.append(f"recall {s.recall:.3f} < baseline {base_r:.3f} -
{tolerance}")
148         else:
149             reasons.append("no baseline recorded for this video/stage ? nothing to compare")
150
151         return RegressionResult(
152             video_id=video_id, stage=stage, iou_threshold=iou_threshold,
153             precision=s.precision, recall=s.recall, f1=s.f1,
154             baseline_precision=base_p, baseline_recall=base_r,
155             tolerance=tolerance, regressed=regressed, reasons=reasons,
156             has_baseline=has_baseline, gt_hash=fp,
157         )
158
159
160     def regress_with_provider(db: Database, video_id: str, video_ref: str,
161                             tier: str = "file", stage: str = "final",
162                             iou_threshold: float = 0.5,
163                             detector: Optional[str] = None,
164                             tolerance: float = DEFAULT_TOLERANCE,
165                             baseline_path: str = DEFAULT_BASELINE_PATH,
166                             guideline: Optional[str] = None,
167                             sport: str = "badminton",
168                             provider_kwargs: Optional[dict] = None) -> RegressionResult:
169         """End-to-end: obtain ground truth from the `tier` annotation provider, then
regress.
170
171         `tier` is an annotation-provider registry key ("file" today; "vendor"/"auto"
172         once those providers land). `video_ref` is whatever that provider expects (a CSV
173         path for FileProvider, a video_id/URL for a vendor). `video_id` keys the stored
174         predictions in the DB (the file MD5).
175         """
176         provider = annotation_registry.get(tier, **(provider_kwargs or {}))
177         gt = provider.annotate(video_ref, guideline=guideline, sport=sport)
178         return regress(db, video_id, gt, stage=stage, iou_threshold=iou_threshold,
179                       detector=detector, tolerance=tolerance, baseline_path=baseline_path)
180
181
182     def format_result(r: RegressionResult) -> str:
183         verdict = "REGRESSED" if r.regressed else "ok"
184         head = (f"[{verdict}] {r.video_id} ({r.stage}) "
185               f"precision={r.precision:.3f} recall={r.recall:.3f} f1={r.f1:.3f}")
186         if r.baseline_precision is not None:
187             head += f" | baseline p={r.baseline_precision:.3f} r={r.baseline_recall:.3f}
tol={r.tolerance}"
188         return head + ".join(f'\n    - {why}' for why in r.reasons)
189

```

52. assistant (2026-06-18T01:39:02.834Z)

I want to double-check `gt\_hash` here vs the one in golden_fixtures. Note `gt\_hash` uses `json.dumps(norm)` where `norm` is a list of tuples; `json.dumps` serializes tuples as lists, so deterministic. Fine.

Let me look at the gemini segmenter and the wasb_runner ? the remaining big production paths I haven't seen. Also the calibrate_wasb's `load\_golden\_gts` which distill_local uses.

53. user (2026-06-18T01:39:04.169Z)

```

1     import os
2     import time
3     import logging
4     import subprocess

```

```

5     import concurrent.futures
6     from typing import List, Dict, Any, Tuple
7     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8     from backend.utils.video import get_video_duration, extract_video_chunk
9     from backend.sports import sport_registry
10    from backend.providers import provider_registry
11    from backend.reporting import run_signals
12
13    logger = logging.getLogger(__name__)
14
15
16    def _chunk_workdir(video_id: str) -> str:
17        """Per-video chunk staging dir under output/chunks/<video_id>.
18
19        Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20        otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21        in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22        """
23        return os.path.join("output", "chunks", video_id)
24
25
26    class GeminiPlaySegmenter(BasePlaySegmenter):
27        @property
28        def name(self) -> str:
29            return "gemini"
30
31        @property
32        def description(self) -> str:
33            return "High-precision AI-powered semantic play segment detection using Google
Gemini Multimodal API."
34
35        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
36            """
37            Processes a video to detect play segments using Google Gemini Multimodal Video
API,
38            populates SQLite, and indexes detailed events.
39            """
40            # Get sport profile config
41            sport_name = self.config.get("sport", "badminton")
42            try:
43                sport_profile = sport_registry.get(sport_name)
44            except KeyError as e:
45                logger.error(str(e))
46                return []
47
48            # Get AI provider and model config
49            idx_cfg = self.config.get("indexing", {})
50            provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
51            model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
52
53            # Get appropriate API key based on the provider
54            api_key_env_var = f"{provider_name.upper()}_API_KEY"
55            api_key = os.environ.get(api_key_env_var)
56            if not api_key:
57                logger.error(
58                    f"{api_key_env_var} environment variable is not set! "
59                    f"To run the {provider_name} segmenter, set your API key. "
60                    f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
61                )
62            return []
63
64            try:

```

```

65         provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
66     except KeyError as e:
67         logger.error(str(e))
68         return []
69
70     # A: Check for debug_duration to trim the video first
71     debug_duration = self.config.get("indexing", {}).get("debug_duration")
72     video_to_upload = video_path
73     is_trimmed = False
74     temp_trimmed_path = None
75
76     if debug_duration:
77         logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
78         temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
79         os.makedirs("output", exist_ok=True)
80         if os.path.exists(temp_trimmed_path):
81             try:
82                 os.remove(temp_trimmed_path)
83             except Exception:
84                 pass
85
86         cmd = [
87             "ffmpeg", "-y",
88             "-ss", "0",
89             "-t", str(debug_duration),
90             "-i", video_path,
91             "-c", "copy",
92             temp_trimmed_path
93         ]
94         logger.debug(f"Running command: {' '.join(cmd)}")
95         try:
96             subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
97             logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
98             video_to_upload = temp_trimmed_path
99             is_trimmed = True
100         except Exception as e:
101             logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
102
103
104
105     # Helper to clean up local temp files
106     def cleanup_temp_files():
107         if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
108             try:
109                 os.remove(temp_trimmed_path)
110             except Exception:
111                 pass
112
113     # 1. Determine actual video duration
114     video_duration = get_video_duration(video_to_upload)
115     if video_duration > 0:
116         logger.info(f"Detected video duration: {video_duration:.1f}s")
117     else:
118         logger.warning("Could not detect video duration. Prompt will not include
duration context.")
119
120     # 2. Decide whether to use chunked processing
121     idx_cfg = self.config.get("indexing", {})
122     chunk_duration = idx_cfg.get("chunk_duration", 300) # 5 minutes default
123     chunk_overlap = idx_cfg.get("chunk_overlap", 30) # 30 seconds overlap

```

```

124         use_chunking = video_duration > 0 and video_duration > chunk_duration
125         # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
126         # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
127         # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
128         ai_scale_width = int(idxcfg.get("ai_chunk_scale_width", 1280) or 0)
129
130         ai_segments = []
131         # Per-video chunk workdir (concurrency isolation): a concurrent run for another
132         # video keeps its own dir, so cleanup here never removes another run's chunks.
133         chunks_dir = _chunk_workdir(video_id)
134
135         if use_chunking:
136             # --- CHUNKED PROCESSING ---
137             step = chunk_duration - chunk_overlap
138             chunk_starts = []
139             t = 0.0
140             while t < video_duration:
141                 chunk_starts.append(t)
142                 t += step
143             num_chunks = len(chunk_starts)
144             logger.info(
145                 f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
146                 f"({chunk_duration}s each, {chunk_overlap}s overlap).")
147             )
148
149             os.makedirs(chunks_dir, exist_ok=True)
150
151             def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
152                 chunk_end = min(chunk_start + chunk_duration, video_duration)
153                 actual_chunk_dur = chunk_end - chunk_start
154                 chunk_label = f"Chunk {ci+1}/{num_chunks}"
155                 chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
156
157                 # Extract chunk with ffmpeg
158                 logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
159                 success = extract_video_chunk(
160                     video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
161                     reencode=ai_scale_width > 0,
162                     scale_width=ai_scale_width if ai_scale_width > 0 else None,
163                 )
164                 if not success:
165                     logger.error(f"Failed to extract {chunk_label}. Skipping.")
166                     return chunk_label, [], {}
167
168                 # Measure chunk size
169                 try:
170                     chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
171                 except Exception:
172                     chunk_size_mb = 0.0
173
174                 # Get prompt from sport profile
175                 prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
176
177                 # Process this chunk through the provider
178                 chunk_segments, metrics = provider.analyze_video(
179                     chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
180                 )
181                 metrics["file_size_mb"] = chunk_size_mb
182                 metrics["segments_found"] = len(chunk_segments)
183

```

```

184         # Offset timestamps back to full-video coordinates
185         for segment in chunk_segments:
186             if "start_time" in segment:
187                 try:
188                     segment["start_time"] = float(segment["start_time"]) +
chunk_start
189                 except (ValueError, TypeError):
190                     pass
191             if "end_time" in segment:
192                 try:
193                     segment["end_time"] = float(segment["end_time"]) +
chunk_start
194                 except (ValueError, TypeError):
195                     pass
196
197         logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
198
199         # Clean up chunk file
200         try:
201             os.remove(chunk_path)
202         except Exception:
203             pass
204
205         return chunk_label, chunk_segments, metrics
206
207     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same
knob
208     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
209     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns
a
210     # misleading "credits depleted" 429 under 5 simultaneous uploads.
211     all_metrics = {}
212     max_workers = max(1, int(self.config.get("indexing",
{}}).get("ai_max_workers", 5)))
213     logger.info("Starting parallel processing with up to %d concurrent
workers...", max_workers)
214     t_chunking_start = time.time()
215     with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
executor:
216         futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
217         for future in concurrent.futures.as_completed(futures):
218             label, segments, metrics = future.result()
219             ai_segments.extend(segments)
220             if metrics:
221                 all_metrics[label] = metrics
222
223     # Oversize refusals must be LOUD: each one is a time range with NO analysis,
224     # so "0 segments / success" here would be a silent failure, not "no
rallies".
225     oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
226     if oversize_skips:
227         logger.error(
228             f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload
cap and were "
229             f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time
ranges. "
230             f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
231             f"downscaled before upload."
232         )
233     run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)
234

```

```

235         # Print Summary Log
236         summary_lines = [
237             "",
238             "=" * 60,
239             "                GEMINI CHUNKING SUMMARY",
240             "=" * 60,
241             f" Total time: {time.time() - t_chunking_start:.1f}s",
242             f" Total chunks: {num_chunks}",
243             f" Total play segments found: {len(ai_segments)}",
244             "-" * 60,
245             f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}}",
246             "-" * 60,
247         ]
248         for i in range(num_chunks):
249             lbl = f"Chunk {i+1}/{num_chunks}"
250             m = all_metrics.get(lbl, {})
251             sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
252             upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
253             prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
254             gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
255             seg = m.get('segments_found', 0) if m else 0
256             summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
257         summary_lines.append("=" * 60)
258         logger.info("\n".join(summary_lines))
259
260         # Clean up local temp files
261         cleanup_temp_files()
262         try:
263             os.rmdir(chunks_dir)
264         except Exception:
265             pass
266
267         logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
268
269     else:
270         # --- SINGLE-FILE PROCESSING (short videos) ---
271         try:
272             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
273         except Exception:
274             file_sz = 0.0
275
276         # Get prompt from sport profile
277         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
278         ai_segments, metrics = provider.analyze_video(
279             video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
280         )
281         if metrics.get("oversize_skipped"):
282             logger.error(
283                 f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and
was "
284                 f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
285             )
286             run_signals.incr(video_id, "oversize_clips_skipped", 1)
287         summary_lines = [
288             "",
289             "=" * 50,
290             "                GEMINI SINGLE FILE SUMMARY",
291             "=" * 50,
292             f" File Size: {file_sz:.1f} MB",
293             f" Upload time: {metrics.get('upload_time', 0):.1f}s",

```

```

294         f" API Processing time: {metrics.get('process_time', 0):.1f}s",
295         f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
296         f" Play segments found: {len(ai_segments)}",
297         "=" * 50,
298     ]
299     logger.info("\n".join(summary_lines))
300     cleanup_temp_files()
301
302     if not ai_segments:
303         return []
304
305     # Populate SQLite DB with parsed play segments
306     segments_data = []
307     logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
308
309     def parse_time(val) -> float:
310         """Converts a timestamp value to total float seconds.
311         Handles float/int pass-through, plain numeric strings, and
312         colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
313         when unexpected formats are encountered."""
314         if isinstance(val, (int, float)):
315             return float(val)
316         if isinstance(val, str):
317             val = val.strip()
318             if ":" in val:
319                 # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
320
321                 # Convert it, but warn so the issue is visible in logs.
322                 parts = val.split(":")
323                 parts.reverse()
324                 total = 0.0
325                 for i, p in enumerate(parts):
326                     try:
327                         total += float(p) * (60 ** i)
328                     except ValueError:
329                         pass
330                 logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
331                 return total
332             try:
333                 return float(val)
334             except ValueError:
335                 logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
336                 return 0.0
337             logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
338             return 0.0
339
340     # 7. Parse timestamps and build candidate segment list
341     idx_cfg = self.config.get("indexing", {})
342     min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
343     max_segment_duration = 90.0 # Singles play segments almost never exceed this
344
345     candidates = []
346     for idx, segment in enumerate(ai_segments):
347         start = parse_time(segment.get("start_time", 0.0))
348         end = parse_time(segment.get("end_time", 0.0))
349         if start >= end:
350             logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
351             continue
352         candidates.append({
353             "idx": idx,

```

```

353         "start": start,
354         "end": end,
355         "raw": segment
356     })
357
358     # 8. Post-hoc validation pass
359     logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
360     validated = []
361     rejected = []
362
363     for c in candidates:
364         label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
365
366         # A: Reject segments that start beyond the video
367         if video_duration > 0 and c["start"] >= video_duration:
368             rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
369             continue
370
371         # B: Clamp end_time to video duration
372         if video_duration > 0 and c["end"] > video_duration:
373             logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
374             c["end"] = video_duration
375
376         dur = c["end"] - c["start"]
377
378         # C: Reject segments shorter than min_segment_duration
379         if dur < min_segment_duration:
380             rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
381             continue
382
383         # D: Flag suspiciously long segments
384         if dur > max_segment_duration:
385             logger.warning(
386                 f"{label}: duration ({dur:.1f}s) is unusually long
(>{max_segment_duration}s). "
387                 f"This may be a detection error. Keeping but flagging."
388             )
389
390         validated.append(c)
391
392     # E: Resolve overlapping segments (keep the longer one)
393     if len(validated) > 1:
394         # Sort by start time
395         validated.sort(key=lambda r: r["start"])
396         non_overlapping = [validated[0]]
397         for curr in validated[1:]:
398             prev = non_overlapping[-1]
399             # Check if current segment overlaps with the previous one
400             if curr["start"] < prev["end"]:
401                 prev_dur = prev["end"] - prev["start"]
402                 curr_dur = curr["end"] - curr["start"]
403                 kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
prev)
404                 kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
{kept['end']:.2f}s]"
405                 dropped_label = f"Segment #{dropped['idx']+1}
[{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
406                 rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
the longer segment"))
407                 if kept == curr:
408                     non_overlapping[-1] = curr # Replace previous with current

```



```

409         else:
410             non_overlapping.append(curr)
411         validated = non_overlapping
412
413     # Log validation summary
414     if rejected:
415         reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
416         logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
417 "\n".join(reject_lines))
418         logger.info(f"{len(validated)} play segments passed validation (out of
419 {len(ai_segments)} from Gemini).")
420
421     stitch_cfg = self.config.get("stitching", {})
422     begin_pad = stitch_cfg.get("begin_padding", 0.5)
423     end_pad = stitch_cfg.get("end_padding", 0.5)
424
425     total_highlight_dur = 0.0
426     for c in validated:
427         total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
428
429     mins = int(total_highlight_dur // 60)
430     secs = int(total_highlight_dur % 60)
431     logger.info(
432         f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
433         f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
434 {end_pad}s end padding per segment)"
435     )
436
437     # 9. Populate SQLite DB with validated play segments
438     for c in validated:
439         start = c["start"]
440         end = c["end"]
441         segment = c["raw"]
442         dur = end - start
443
444         # Extract shot_count from Gemini or compute estimate if not present
445         shot_count = segment.get("shuttle_exchange_count")
446         if shot_count is None:
447             shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
448 fallback estimate
449         else:
450             try:
451                 shot_count = int(shot_count)
452             except Exception:
453                 shot_count = max(3, int(dur / 0.8))
454
455         # Only store metrics we actually know ? no fabricated data
456         metadata = {
457             "shot_count": shot_count,
458             "shot_count_confidence": segment.get("shot_count_confidence",
459 "Unknown"),
460             "detector": model_name
461         }
462
463         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
464 metadata)
465
466         if segment_id:
467             segments_data.append({
468                 "id": segment_id,
469                 "start_time": start,
470                 "end_time": end,
471                 "duration": dur,
472                 "metadata": metadata
473             })

```

```

468
469         return segments_data
470
471     # Register the segmenter
472     segmenter_registry.register(GeminiPlaySegmenter)
473

```

54. assistant (2026-06-18T01:39:12.529Z)

I notice a subtle bug in the overlap-resolution logic (lines 392-411). When the kept segment is `prev` (because prev is longer or equal), the code does NOT update `non\_overlapping[-1]` ? it keeps `prev`. But consider three segments where seg A is long, seg B overlaps A and is dropped, then seg C overlaps B but NOT A. Because B was dropped and A remains in `non\_overlapping[-1]`, C is compared against A. That's correct behavior actually.

But there's a real concern: when `kept == curr` (curr is longer), it replaces `non\_overlapping[-1]` with curr. But the original prev's `end` might extend beyond curr's end. Consider prev=[0,10] (dur 10), curr=[5,12] (dur 7). prev is longer, kept=prev, so curr is dropped. Fine. Now prev=[0,7] (dur 7), curr=[5,20] (dur 15). curr is longer, kept=curr, non_overlapping[-1] becomes curr=[5,20]. But prev started at 0 ? we just dropped coverage [0,5]. That's the documented "keep the longer one" behavior, acceptable by design.

The `kept == curr` comparison uses dict equality ? comparing dicts by value. Two distinct segment dicts could theoretically be equal, but they have distinct `idx`, so they're never equal. Fine. This is intended behavior, not a bug.

Let me look at the remaining production module: wasb_runner and the base segmenter / trajectory_hybrid. Let me also check `calibrate\_wasb.load\_golden\_gts`.

55. user (2026-06-18T01:39:13.631Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)

```

```

35
36
37     def _fmt_ts(seconds: float) -> str:
38         seconds = max(0, int(seconds))
39         h, rem = divmod(seconds, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55         Any]]:
56             """Return (runner, detector-specific detection_params extras)."""
57             raise NotImplementedError
58
59         @staticmethod
60         def _probe_fps_width(video_path: str) -> tuple:
61             """Return (fps, frame_width) for a video, with sensible fallbacks."""
62             cap = cv2.VideoCapture(video_path)
63             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
64             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
65             cap.release()
66             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
67
68         @staticmethod
69         def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
70             """Provider-reported exchange count, else a duration-based estimate.
71
72             One canonical implementation ? the twins had drifted (wasb relied on
73             ``int(None)`` raising into the fallback; same result, by accident).
74             """
75             shot_count = segment.get("shuttle_exchange_count")
76             if shot_count is None:
77                 return max(3, int(duration / 0.8))
78             try:
79                 return int(shot_count)
80             except (ValueError, TypeError):
81                 return max(3, int(duration / 0.8))
82
83         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
84         -----
85         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
86         frame_width: float, video_path: str,
87         idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
88             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
89             BASE
90             returns exactly the 4 motion keys, so the trajectory twins persist byte-
91             identical
92             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
93             features). ``points`` are the whole-video trajectory points
94             (TrajectoryPoint)."""
95             return {
96                 "peak_velocity": cw["peak_velocity"],
97                 "mean_velocity": cw["mean_velocity"],
98                 "active_frame_count": cw["active_frame_count"],
99                 "track_id_count": cw["track_id_count"],

```

```

94         }
95
96     def _select_for_handoff(self, windows: List[Dict[str, Any]],
97                           window_stats: List[Dict[str, Any]],
98                           idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
103         offline
104         re-scoring."""
105         return list(range(len(windows)))
106
107     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
108                     player_pool: Optional[List[str]] = None,
109                     max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
110         # override-ignore: the ABC declares the Result contract (Success|Failure)
111         # but every live segmenter still returns a bare list ? the documented
112         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
113         label = self.display_label
114         # --- Sport profile + AI provider wiring (identical across segmenters) ---
115         sport_name = self.config.get("sport", "badminton")
116         try:
117             sport_profile = sport_registry.get(sport_name)
118         except KeyError as e:
119             logger.error(str(e))
120             return []
121
122         idx_cfg = self.config.get("indexing", {})
123         # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become
124         the
125         # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key
126         is
127         # needed and nothing is uploaded. Used to run the local detector standalone
128         (e.g. to
129         # measure it against the Gemini path, or to avoid the cloud entirely). With
130         # FusionSegmenter the windows are still Gate-A/B-filtered via
131         `_select_for_handoff`.
132         skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
133         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
134         "gemini"))
135         if skip_ai:
136             model_name = "local-noai"
137             logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate
138         windows will "
139             "be emitted as rallies; no %s handoff, no API key needed.",
140             label, provider_name)
141         else:
142             model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
143         "gemini-2.5-flash"))
144             api_key_env_var = f"{provider_name.upper()}_API_KEY"
145             api_key = os.environ.get(api_key_env_var)
146             if not api_key:
147                 logger.error(
148                     f"{api_key_env_var} environment variable is not set! "
149                     f"To run the {provider_name} handoff, set your API key. "
150                     f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")
151             )
152             return []
153         try:
154             provider = provider_registry.get(provider_name, api_key=api_key,
155         model_name=model_name)
156         except KeyError as e:
157             logger.error(str(e))
158             return []

```

```

150
151     video_duration = get_video_duration(video_path)
152     if video_duration <= 0:
153         logger.error("Invalid video duration.")
154         return []
155     fps, frame_width = self._probe_fps_width(video_path)
156
157     # --- Runner config + windowing thresholds ---
158     runner, runner_params = self._build_runner(idx_cfg)
159
160     velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
161     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
162     min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
163     # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See
IndexingConfig.
164     max_window_duration = idx_cfg.get("max_window_duration", 0.0)
165     window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
166     window_hard_cap = idx_cfg.get("window_hard_cap", False)
167     window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
168     inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
169     inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
170     window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
171     window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
172     handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
173     ai_max_workers = idx_cfg.get("ai_max_workers", 5)
174     log_candidates = idx_cfg.get("log_yolo_candidates", True)
175     store_candidates = idx_cfg.get("store_yolo_candidates", True)
176
177     detection_params = {
178         "detector": self.name,
179         "velocity_thresh": velocity_thresh,
180         "merge_gap": merge_gap,
181         "min_action_window_duration": min_window_duration,
182         **runner_params,
183     }
184
185     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
186     ok, msg = runner.healthcheck()
187     if not ok:
188         logger.error("%s WSL environment not ready: %s", label, msg)
189         return []
190     logger.info("%s WSL env OK: %s", label, msg)
191
192     # --- Phase 2: trajectory -> candidate rally windows ---
193     # Trajectory detectors process the whole video in one pass (they carry
194     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
195     t_start = time.time()
196     out_dir = os.path.join("output", self.family)
197     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
198     if not csv_path:
199         logger.error("%s produced no trajectory; aborting.", label)
200         return []
201
202     points = runner.parse_trajectory_csv(csv_path)
203     logger.info("Parsed %d trajectory points (%d visible).",
204                 len(points), sum(1 for p in points if p.visible))
205
206     all_candidate_windows = runner.trajectory_to_action_windows(
207         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
208         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
209         min_window_duration=min_window_duration, chunk_index=0,
210         max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
211         window_hard_cap=window_hard_cap,
212         window_inactivity_end=window_inactivity_end,
213         inactivity_rally_thresh=inactivity_rally_thresh,

```

```

214         inactivity_min_density=inactivity_min_density,
215         window_blob_fallback=window_blob_fallback,
216         window_blob_factor=window_blob_factor,
217     )
218     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
219                 label, time.time() - t_start, len(all_candidate_windows))
220
221     if log_candidates:
222         for cw in all_candidate_windows:
223             logger.info(f"[{label} rally]
224             {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
225             f"({cw['end'] - cw['start']:.1f}s) |
frames={cw['active_frame_count']} "
226             f"peak_v={cw['peak_velocity']:.3f}
mean_v={cw['mean_velocity']:.3f}")
227
228     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
fusion
229     # segmenter adds net_crossings + person-cue features. Computed once, reused for
both
230     # persistence and the handoff gate below.
231     window_stats = [
232         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
idx_cfg)
233         for cw in all_candidate_windows
234     ]
235
236     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
237     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
238     if store_candidates:
239         for i, cw in enumerate(all_candidate_windows):
240             candidate_ids[i] = self.db.add_candidate_segment(
241                 video_id, detector=self.name,
242                 start_time=cw["start"], end_time=cw["end"],
243                 chunk_index=cw["chunk_index"], confidence=min(1.0,
cw["peak_velocity"]),
244                 stats=window_stats[i], detection_params=detection_params,
245             )
246
247     if not all_candidate_windows:
248         return []
249
250     # --- Phase 3: AI provider handoff over padded candidate clips ---
251     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
252     # candidates above stay the full superset ? only the served play_segments are
gated.
253     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
254
255     ai_segments: List[dict] = []
256     if skip_ai:
257         # Fully-local: the selected candidate windows ARE the rallies ? emit them
verbatim
258         # (no clip extraction, no upload). Boundaries are the local detector's; shot
count
259         # falls back to the duration-based estimate (no AI exchange count).
260         ai_segments = [
261             {
262                 "start_time": all_candidate_windows[wi]["start"],
263                 "end_time": all_candidate_windows[wi]["end"],
264                 "shuttle_exchange_count": None,
265                 "shot_count_confidence": "LocalNoAI",
266                 "_candidate_id": candidate_ids[wi],

```

```

267         for wi in handoff_indices
268     ]
269     logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as
rallies "
270                 "(no AI handoff).", len(ai_segments))
271 else:
272     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
273     os.makedirs(handoff_dir, exist_ok=True)
274     logger.info("Phase 3: %s validation on %d candidate windows...",
275                 provider_name, len(handoff_indices))
276
277     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
278         w_start = max(0, window["start"] - handoff_padding)
279         w_end = min(video_duration, window["end"] + handoff_padding)
280         w_dur = w_end - w_start
281         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
282         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
283             return []
284         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
285         segments, _ = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur,
286                                             label=f"Handoff {wi+1}")
287         valid = []
288         for seg in segments:
289             try:
290                 seg["start_time"] = float(seg["start_time"]) + w_start
291                 seg["end_time"] = float(seg["end_time"]) + w_start
292                 seg["_candidate_id"] = candidate_ids[wi]
293                 valid.append(seg)
294             except (ValueError, TypeError, KeyError) as e:
295                 logger.warning("Dropping segment with unparseable timestamps: %s
? %s", seg, e)
296         try:
297             os.remove(w_path)
298         except OSError:
299             pass
300         return valid
301
302     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
303         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
304                     for wi in handoff_indices]
305         for future in futures:
306             ai_segments.extend(future.result())
307         try:
308             os.rmdir(handoff_dir)
309         except OSError:
310             pass
311         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
312
313     # --- Phase 4: DB population + candidate linking ---
314     segments_data = []
315     for segment in ai_segments:
316         try:
317             start = float(segment["start_time"])
318             end = float(segment["end_time"])
319             dur = end - start
320             metadata = {
321                 "shot_count": self._shot_count_from(segment, dur),
322                 "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
323                 "detector": f"{self.family}-hybrid-{model_name}",

```

```

324         }
325         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
326         if segment_id:
327             candidate_id = segment.get("_candidate_id")
328             if candidate_id is not None:
329                 self.db.link_candidate_to_segment(candidate_id, segment_id)
330             segments_data.append({
331                 "id": segment_id, "start_time": start, "end_time": end,
332                 "duration": dur, "metadata": metadata,
333             })
334         except Exception as e:
335             logger.warning("Skipping invalid segment struct: %s", e)
336
337     return segments_data
338

```

56. assistant (2026-06-18T01:39:18.437Z)

This is solid. I want to look at the `fusion_hybrid` (which overrides `_select_for_handoff`` and `_enrich_candidate_stats``, where Gate-A logic lives) and `fusion_features`` / `person_detector`` - the more complex production overrides. Let me also check `calibrate_wasb.load_golden_gts``.

57. user (2026-06-18T01:39:19.270Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
experiment
20    harness can re-score any variant.
21
22    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26    experiment harness (EXPERIMENT_HARNESS.md), never silently.
27    """
28    from typing import Any, Dict, List, Optional, Tuple
29
30    from backend.pipeline.segmenters.base import segmenter_registry

```



```

31     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
41 B)."""
42         family = "fusion"
43         display_label = "Fusion"
44
45         def __init__(self, db: Any, config: Any):
46             super().__init__(db, config)
47             # Built lazily only if the person cue is enabled (no detector model loaded
48 otherwise).
49             self._person: Optional[Any] = None
50             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
51 estimate it
52             # over the whole trajectory once per candidate window (it depends only on
53 `points`).
54             self._axis_cache_key: Optional[int] = None
55             self._axis_cache: Optional[tuple] = None
56
57         @property
58         def name(self) -> str:
59             return "fusion_hybrid"
60
61         @property
62         def description(self) -> str:
63             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
64 rallies, "
65 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
66 and the "
67 "AI provider sets semantic boundaries.")
68
69         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
70 Any]]:
71             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
72             if substrate == "tracknet":
73                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
74                 return TrackNetRunner(cfg), {
75                     "weights_path": cfg.weights_path,
76                     "queue_length": cfg.queue_length,
77                     "fusion_substrate": "tracknet",
78                 }
79             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
80             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
81 "wasb"}
82
83         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
84 frame_width: float, video_path: str,
85 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
86             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
87 video_path, idx_cfg)
88             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
89 persisted
90             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
91 estimated
92             # over the whole trajectory by rally_gate (camera-agnostic); reused across
93 windows.
94             gated = rally_gate.gate_windows(points, [[cw["start"], cw["end"]]], fps,

```

```

84                                     min_crossings=0,
estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92         # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93         # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94         # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95         # for shuttle-seeded windows + padding, not whole video) is already in place via the
lazy
96         # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100        def _axis_estimate(self, points: List[Any]) -> tuple:
101            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102            computed once per video, not once per candidate window (it depends only on
points)."""
103            key = id(points)
104            if self._axis_cache_key != key or self._axis_cache is None:
105                self._axis_cache_key = key
106                self._axis_cache = rally_gate.estimate_play_axis(points)
107            est = self._axis_cache
108            assert est is not None
109            return est
110
111        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112                             frame_width: float, video_path: str,
113                             fcfg: Dict[str, Any]) -> Dict[str, float]:
114            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115            frame range and compute the scale/translation-invariant person features. Lazily
builds
116            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118            from backend.pipeline.detectors.person_detector import PersonDetector
119            if self._person is None:
120                self._person = PersonDetector(self.config)
121
122            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
123            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
124            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
125            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127            fw = det["frame_width"] or frame_width
128            fh = det["frame_height"]
129            # If we can't trust the frame dims, every normalised feature would silently
collapse
130            # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
131            if fw <= 0 or fh <= 0:

```

```

132         return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
133
134     poly = fcfg.get("court_polygon")
135     polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
136     net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
137     shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
138     return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
139                                     court_polygon=polygon, net=net)
140
141     def _select_for_handoff(self, windows: List[Dict[str, Any]],
142                           window_stats: List[Dict[str, Any]],
143                           idx_cfg: Dict[str, Any]) -> List[int]:
144         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
145         (0) keep every window ? identical to the substrate. Persisted candidates are
146         untouched (full superset).
147
148         NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If
the
149         thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed
trajectory
150         where no segment clears ``min_crossings``), the gate falls back to the full
ungated
151         set rather than emit a video with zero rallies. This makes Gate A/B strictly
152         *pruning* ? it can shrink a candidate set but never zero it out ? so layering it
on
153         the substrate is "do no harm": the bounded-windowing / largest-internal-gap
split
154         below it always wins on an otherwise-empty video. Observed on real footage
155         (mahadevpura-1): a single mega-window split into pieces, none clearing the
crossings
156         gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
157         fcfg = idx_cfg.get("fusion", {})
158         min_crossings = int(fcfg.get("min_crossings", 0))
159         min_players = int(fcfg.get("min_players", 0))
160         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
161
162         keep: List[int] = []
163         for i, st in enumerate(window_stats):
164             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
165                 continue
166             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
167                 continue
168             keep.append(i)
169
170         if not keep and window_stats:
171             # Gating emptied a non-empty video ? fall back to the ungated set.
172             return list(range(len(window_stats)))
173         return keep
174
175
176     segmenter_registry.register(FusionSegmenter)
177

```

58. assistant (2026-06-18T01:39:23.472Z)

This is carefully constructed. Let me look at `fusion\_features.py` (compute_person_features ? geometry with court polygon, the most complex untested-on-real-data code), and check `calibrate\_wasb.load\_golden\_gts`. These are good under-examined candidates.

59. user (2026-06-18T01:39:24.373Z)

```

1     """Pure per-window person-cue features for multi-signal rally fusion
(MULTI_SIGNAL_FUSION_PLAN §3.2/§4).

```

```

2
3 NO I/O, no torch, no cv2 ? mirrors `rally_gate.py`'s discipline so the whole feature
4 math is unit-testable in a GPU-free dev container. The detection (person boxes,
5 shuttle trajectory) is produced elsewhere (`PersonDetector`, the WASB runner); this
6 module only turns those signals into a handful of **scale/translation-invariant**
7 features (counts / ratios / zone-membership ? never raw pixel coordinates, which would
8 memorise this court/camera and fail to generalise off a small labelled set).
9
10 Conventions:
11 - A person box is `(x1, y1, x2, y2)` in **pixels**; `(track_id, box)` when motion
needs it.
12 - **per_frame** is `{frame_idx: [(track_id, box), ...]}` keyed by the ORIGINAL video
frame
13 index (so it aligns directly with the shuttle trajectory's **frame** ? no clip re-
timing).
14 - A shuttle point has **frame**, **visible**, **x**, **y** (pixels) ? the WASB
trajectory shape.
15 - **court_polygon** is normalised 0-1 **[(x, y), ...]** or **None** (None ? no mask,
every
16 detection counts ? the player-count-only mode).
17 - **net** is **('axis 'x'|'y', position 0-1 normalised)** or **None** (None ? two-sided =
0.0).
18
19 Every function degrades gracefully (returns 0.0 / empty) on missing inputs; outputs are
20 floats in [0, 1] except **mean_player_motion** (a non-negative normalised speed).
21 """
22 from __future__ import annotations
23
24 from typing import Dict, List, Optional, Sequence, Tuple
25
26 BBox = Tuple[float, float, float, float]
27 Net = Tuple[str, float]
28
29
30 def point_in_polygon(point: Tuple[float, float], polygon: Sequence[Tuple[float, float]])
-> bool:
31     """Ray-casting point-in-polygon. `point` and `polygon` share a coordinate space
32     (we use normalised 0-1). Empty/degenerate polygon (<3 vertices) ? False."""
33     if polygon is None or len(polygon) < 3:
34         return False
35     x, y = point
36     inside = False
37     n = len(polygon)
38     j = n - 1
39     for i in range(n):
40         xi, yi = polygon[i]
41         xj, yj = polygon[j]
42         # Does the horizontal ray at y cross edge (i, j)?
43         if (yi > y) != (yj > y):
44             x_cross = (xj - xi) * (y - yi) / (yj - yi) + xi if yj != yi else xi
45             if x < x_cross:
46                 inside = not inside
47         j = i
48     return inside
49
50
51 def _foot_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
52     """Normalised foot point (bottom-centre) of a person box ? where the player stands,
53     the right anchor for court membership. Returns (0,0) if frame dims are unusable."""
54     if frame_width <= 0 or frame_height <= 0:
55         return (0.0, 0.0)
56     x1, _y1, x2, y2 = box
57     return ((x1 + x2) / 2.0 / frame_width, y2 / frame_height)
58

```

```

59
60 def _center_norm(box: BBox, frame_width: float, frame_height: float) -> Tuple[float,
float]:
61     if frame_width <= 0 or frame_height <= 0:
62         return (0.0, 0.0)
63     x1, y1, x2, y2 = box
64     return ((x1 + x2) / 2.0 / frame_width, (y1 + y2) / 2.0 / frame_height)
65
66
67 def person_in_court(box: BBox, frame_width: float, frame_height: float,
68                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> bool:
69     """Is this person on the court? Foot-point-in-polygon when a polygon is supplied;
70     True (count everyone) when it is None. Unusable frame dims (w/h <= 0) make the foot
71     point meaningless, so a masked check returns False rather than testing a bogus
72     (0,0)."""
73     if court_polygon is None:
74         return True
75     if frame_width <= 0 or frame_height <= 0:
76         return False
77     return point_in_polygon(_foot_norm(box, frame_width, frame_height), court_polygon)
78
79 def in_court_counts(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
80                    frame_height: float,
81                    court_polygon: Optional[Sequence[Tuple[float, float]]]) ->
List[int]:
82     """Per-frame count of in-court persons (one entry per sampled frame)."""
83     return [
84         sum(1 for _tid, box in dets if person_in_court(box, frame_width, frame_height,
85 court_polygon))
86         for _frame_idx, dets in sorted(per_frame.items())
87     ]
88
89 def _median(vals: Sequence[float]) -> float:
90     if not vals:
91         return 0.0
92     s = sorted(vals)
93     n = len(s)
94     return float(s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2]))
95
96
97 def players_in_court(counts: Sequence[int]) -> float:
98     """Median per-frame in-court count (?2 singles / 4 doubles; 0-1 = dead time)."""
99     return _median(list(counts))
100
101
102 def in_court_ratio(counts: Sequence[int], min_players: int = 2) -> float:
103     """Fraction of sampled frames with >= `min_players` in court (track stability vs
104 flicker)."""
105     if not counts:
106         return 0.0
107     return sum(1 for c in counts if c >= min_players) / len(counts)
108
109
110 def mean_player_motion(per_frame: Dict[int, List[Tuple[int, BBox]]],
111                       frame_width: float, frame_height: float) -> float:
112     """Mean per-track centre displacement between consecutive sampled frames, with x
113     normalised by frame width and y by frame height (per-axis; uniform-scale-invariant,
114     not aspect-ratio-isotropic). 0.0 if <2 frames or no shared tracks."""
115     if frame_width <= 0 or len(per_frame) < 2:
116         return 0.0
117     frames = sorted(per_frame.keys())
118     steps: List[float] = []
119     for a, b in zip(frames, frames[1:]):

```

```

119         prev = {tid: box for tid, box in per_frame[a]}
120         for tid, box in per_frame[b]:
121             if tid in prev:
122                 cx0, cy0 = _center_norm(prev[tid], frame_width, frame_height)
123                 cx1, cy1 = _center_norm(box, frame_width, frame_height)
124                 steps.append(((cx1 - cx0) ** 2 + (cy1 - cy0) ** 2) ** 0.5)
125     if not steps:
126         return 0.0
127     return sum(steps) / len(steps)
128
129
130 def two_sided_motion(per_frame: Dict[int, List[Tuple[int, BBox]]], frame_width: float,
131                     frame_height: float, net: Optional[Net],
132                     court_polygon: Optional[Sequence[Tuple[float, float]]]) -> float:
133     """Fraction of sampled frames with >=1 in-court person on BOTH net halves.
134
135     A real rally is two-sided; a single player feeding/holding the shuttle is one-sided.
136     The net splits the play axis at `net=(axis, position)` in normalised coords. None ?
137     0.0.
138     """
139     if net is None or not per_frame:
140         return 0.0
141     axis, pos = net
142     both = 0
143     for _frame_idx, dets in per_frame.items():
144         near, far = False, False
145         for _tid, box in dets:
146             if not person_in_court(box, frame_width, frame_height, court_polygon):
147                 continue
148             fx, fy = _foot_norm(box, frame_width, frame_height)
149             coord = fx if axis == "x" else fy
150             if coord < pos:
151                 near = True
152             else:
153                 far = True
154             if near and far:
155                 both += 1
156     return both / len(per_frame)
157
158 def _shuttle_in_box(sx: float, sy: float, box: BBox, frame_width: float, frame_height:
float,
159                   pad_frac: float) -> bool:
160     """Is the (pixel) shuttle point inside a person box, expanded by `pad_frac` of frame
161     width/height (the 'adjacent / in-hand' tolerance)?"""
162     x1, y1, x2, y2 = box
163     px = pad_frac * frame_width
164     py = pad_frac * frame_height
165     return (x1 - px) <= sx <= (x2 + px) and (y1 - py) <= sy <= (y2 + py)
166
167
168 def shuttle_player_colocation(per_frame: Dict[int, List[Tuple[int, BBox]]],
shuttle_points,
169                             frame_width: float, frame_height: float,
170                             pad_frac: float = 0.03) -> float:
171     """Fraction of (visible shuttle frames that were also person-sampled) in which the
172     shuttle sits inside/adjacent to a person box. **held/in-hand ? high; in-flight ?
173     ~0**
174     ? the direct held-shuttle discriminator. 0.0 if no aligned visible shuttle frames.
175     """
176     if not per_frame or frame_width <= 0 or frame_height <= 0:
177         return 0.0
178     aligned = 0
179     colocated = 0
180     for p in shuttle_points or []:

```

```

180         if not getattr(p, "visible", False):
181             continue
182         dets = per_frame.get(int(p.frame))
183         if dets is None:             # shuttle frame wasn't person-sampled ? can't judge
184             continue
185         aligned += 1
186         if any(_shuttle_in_box(p.x, p.y, box, frame_width, frame_height, pad_frac)
187             for _tid, box in dets):
188             colocated += 1
189     return colocated / aligned if aligned else 0.0
190
191
192     # The ~5 person features persisted into candidate `stats` (order = the learned-fusion
193     # feature columns appended after the shuttle features in distill_local.FEATURE_NAMES).
194     PERSON_FEATURE_NAMES = [
195         "players_in_court", "two_sided_motion", "mean_player_motion",
196         "in_court_ratio", "shuttle_player_colocation",
197     ]
198
199
200     def compute_person_features(per_frame: Dict[int, List[Tuple[int, BBox]]],
201 shuttle_points,
202                                     frame_width: float, frame_height: float,
203                                     court_polygon: Optional[Sequence[Tuple[float, float]]] =
204 None,
205                                     net: Optional[Net] = None,
206                                     min_players: int = 2) -> Dict[str, float]:
207     """Aggregate the per-window person features from per-frame detections + the in-
208 window
209 shuttle trajectory. Pure; returns exactly `PERSON_FEATURE_NAMES`, all floats."""
210     counts = in_court_counts(per_frame, frame_width, frame_height, court_polygon)
211     return {
212         "players_in_court": players_in_court(counts),
213         "two_sided_motion": two_sided_motion(per_frame, frame_width, frame_height, net,
214 court_polygon),
215         "mean_player_motion": mean_player_motion(per_frame, frame_width, frame_height),
216         "in_court_ratio": in_court_ratio(counts, min_players),
217         "shuttle_player_colocation": shuttle_player_colocation(
218             per_frame, shuttle_points, frame_width, frame_height),
219     }

```

60. assistant (2026-06-18T01:39:28.596Z)

These are clean. I've now examined the bulk of the production runtime path and the eval surface. Let me check a few remaining error-path/serialization modules I haven't opened: `calibrate\_wasb.py` (load_golden_gts), `harness.py` (get_predictions), and the config loader. Let me also check `run\_signals.py` (in-process signal store used by reporting).

61. user (2026-06-18T01:39:30.013Z)

```

1     """In-process signal channel: pipeline internals -> the per-video report.
2
3     Some report-worthy facts (e.g. the provider refusing an oversize chunk) surface
4     deep inside a segmenter's worker threads, with no return path to the harvest ?
5     segmenters return only their segment list. Layers record those facts here keyed
6     by video_id; `emit_indexer_report` pops them when the run's report is built.
7
8     Process-local and best-effort BY DESIGN: recording must never break a run, the
9     store is consumed (popped) once per report, and signals for a video are simply
10    dropped if no report is emitted. No obsreport dependency ? importable everywhere.
11    """
12    from __future__ import annotations
13
14    import threading

```

```

15     from typing import Any, Dict
16
17     _lock = threading.Lock()
18     _signals: Dict[str, Dict[str, Any]] = {}
19
20
21     def incr(video_id: str, key: str, by: int = 1) -> None:
22         """Add `by` to the counter `key` for this video's in-flight run."""
23         if not video_id or by == 0:
24             return
25         with _lock:
26             sig = _signals.setdefault(video_id, {})
27             sig[key] = int(sig.get(key, 0)) + by
28
29
30     def pop(video_id: str) -> Dict[str, Any]:
31         """Consume (remove and return) all signals recorded for this video's run."""
32         with _lock:
33             return _signals.pop(video_id, {})
34

```

62. user (2026-06-18T01:39:30.348Z)

```

1     """Score the pipeline's detected rallies against ground-truth annotations.
2
3     Reads predictions straight from the SQLite DB (no pipeline re-run, no API cost),
4     so scoring is fast, deterministic, and free. Two prediction "stages" can be
5     scored:
6
7     - ``candidates`` ? raw detector windows (``candidate_segments`` table). What
8       the shuttle/person detector found, before AI confirmation.
9     - ``final`` ? confirmed play segments (``play_segments`` table). After the
10       AI handoff trimmed/validated boundaries.
11
12     Comparing the two tells you whether the weak link is *detection* (candidates
13     already miss rallies) or *segmentation* (candidates catch them but final is
14     wrong) ? see docs/EVALUATION.md.
15     """
16
17     import logging
18     from dataclasses import dataclass, field
19     from typing import Any, List, Dict, Optional
20
21     from backend.infrastructure.database import Database
22     from backend.eval import metrics
23     from backend.eval.metrics import Interval, Score, MatchResult
24
25     logger = logging.getLogger(__name__)
26
27     STAGES = ("candidates", "final")
28
29
30     @dataclass
31     class StageReport:
32         stage: str
33         pred_intervals: List[Interval]
34         score: Score
35         match_result: MatchResult
36         pr_curve: List[Score] = field(default_factory=list)
37
38
39     @dataclass
40     class EvalReport:
41         video_id: str
42         iou_threshold: float

```



```

43     gt_intervals: List[Interval]
44     stages: Dict[str, StageReport] = field(default_factory=dict)
45
46
47 def _rows_to_intervals(rows: List[dict]) -> List[Interval]:
48     """Pull (start_time, end_time) out of DB segment rows, sorted by start."""
49     ivs = [(float(r["start_time"]), float(r["end_time"])) for r in rows]
50     ivs.sort(key=lambda iv: iv[0])
51     return ivs
52
53
54 def get_predictions(db: Database, video_id: str, stage: str,
55                    detector: Optional[str] = None) -> List[Interval]:
56     """Fetch predicted rally intervals for one stage from the DB."""
57     if stage == "candidates":
58         rows = db.query_candidate_segments(video_id, detector=detector)
59     elif stage == "final":
60         rows = db.query_play_segments(video_id, {})
61     else:
62         raise ValueError(f"unknown stage {stage!r}; expected one of {STAGES}")
63     return _rows_to_intervals(rows)
64
65
66 def evaluate(db: Database, video_id: str, gt_intervals: List[Interval],
67             stages: List[str], iou_threshold: float = 0.5,
68             detector: Optional[str] = None,
69             with_pr_curve: bool = False) -> EvalReport:
70     """Score the requested stages for one video against ground truth."""
71     report = EvalReport(video_id=video_id, iou_threshold=iou_threshold,
72                        gt_intervals=gt_intervals)
73     for stage in stages:
74         preds = get_predictions(db, video_id, stage, detector=detector)
75         mr = metrics.match_intervals(preds, gt_intervals, iou_threshold)
76         s = metrics.score(preds, gt_intervals, iou_threshold, match_result=mr)
77         curve = metrics.pr_curve(preds, gt_intervals) if with_pr_curve else []
78         report.stages[stage] = StageReport(stage=stage, pred_intervals=preds,
79                                           score=s, match_result=mr, pr_curve=curve)
80     return report
81
82
83 # ----- reporting
84
85 def _fmt_ts(seconds: float) -> str:
86     seconds = max(0, int(seconds))
87     h, rem = divmod(seconds, 3600)
88     m, s = divmod(rem, 60)
89     return f"{h:02d}:{m:02d}:{s:02d}"
90
91
92 def report_to_dict(report: EvalReport) -> dict:
93     """JSON-serialisable view of an EvalReport."""
94     out: Dict[str, Any] = {
95         "video_id": report.video_id,
96         "iou_threshold": report.iou_threshold,
97         "num_ground_truth": len(report.gt_intervals),
98         "stages": {},
99     }
100     for stage, sr in report.stages.items():
101         out["stages"][stage] = {
102             "score": sr.score.as_dict(),
103             "pr_curve": [s.as_dict() for s in sr.pr_curve],
104         }
105     return out
106
107

```

```

108     def format_report_text(report: EvalReport, show_failures: bool = False) -> str:
109         """Human-readable console report for one video."""
110         lines = [
111             "=" * 64,
112             f" Rally evaluation ? video '{report.video_id}'",
113             f" Ground-truth rallies: {len(report.gt_intervals)} | IoU threshold:
{report.iou_threshold}",
114             "=" * 64,
115             f" {'Stage':<12} {'Pred':>5} {'TP':>4} {'FP':>4} {'FN':>4} "
116             f"{'Prec':>6} {'Rec':>6} {'F1':>6} {'mIoU':>6} {'dStart':>7} {'dEnd':>7}",
117             "-" * 64,
118         ]
119         for stage in report.stages:
120             s = report.stages[stage].score
121             lines.append(
122                 f" {stage:<12} {s.num_pred:>5} {s.true_positives:>4} {s.false_positives:>4}
"
123                 f"{s.false_negatives:>4} {s.precision:>6.2f} {s.recall:>6.2f} {s.f1:>6.2f} "
124                 f"{s.mean_iou:>6.2f} {s.mean_start_error:>6.1f}s {s.mean_end_error:>6.1f}s"
125             )
126             lines.append("=" * 64)
127
128         if show_failures:
129             for stage, sr in report.stages.items():
130                 misses = [report.gt_intervals[i] for i in
sr.match_result.unmatched_gt_indices]
131                 fps = [sr.pred_intervals[i] for i in sr.match_result.unmatched_pred_indices]
132                 lines.append(f"\n [{stage}] missed rallies (false negatives):
{len(misses)}")
133                 for iv in misses:
134                     lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
135                 lines.append(f" [{stage}] spurious detections (false positives):
{len(fps)}")
136                 for iv in fps:
137                     lines.append(f"   - {_fmt_ts(iv[0])}?{_fmt_ts(iv[1])}")
138
139         return "\n".join(lines)
140

```

63. assistant (2026-06-18T01:39:33.406Z)

These are clean. Let me look at the `calibrate_wasb` `load_golden_gts` (used by `distill_local`) and the config loader/models, which are externally-fed validation. Let me also check the `wasb_runner` CSV parse and the `person_detector`.

64. user (2026-06-18T01:39:35.354Z)

```

1     """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
"+X%".
2
3     Wires the already-tested pieces together:
4         WASB trajectory CSV (Frame,Visibility,X,Y)
5         -> TrackNetRunner.trajectory_to_action_windows(cfg) [predict_fn]
6         -> backend.eval.calibration (improvement_report / cross_validate)
7
8     Run (after a full-video WASB pass exists):
9         python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
10             --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
11
12     Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
13     same GT is an **optimistic** upper bound. The trustworthy number is the
14     cross-validated held-out **recall** (and, once a 2nd labelled video exists,
15     `leave_one_video_out` for precision/F1 generalization).
16     """
17     from __future__ import annotations

```

```

18
19 import argparse
20 from typing import List, Tuple, Callable
21
22 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
23 from backend.eval.calibration import (
24     select_best, improvement_report, cross_validate, evaluate,
25 )
26
27 Interval = Tuple[float, float]
28 Config = Tuple[float, float, float] # (velocity_thresh, merge_gap,
min_window_duration)
29
30 BASELINE: Config = (0.02, 2.0, 2.0) # the WasbConfig / trajectory_to_action_windows
defaults
31
32
33 def load_golden_gts(csv_path: str) -> List[Interval]:
34     """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
35
36     Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
37     start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
38     driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
39     """
40     from backend.eval.gt_loader import load_gt_intervals
41     return load_gt_intervals(csv_path, strict=False)
42
43
44 def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
Callable[[Config], List[Interval]]:
45     """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
46     points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
47
48     def predict_fn(cfg: Config) -> List[Interval]:
49         vt, mg, mwd = cfg
50         wins = TrackNetRunner.trajectory_to_action_windows(
51             points, fps=fps, frame_width=frame_width,
52             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
53         )
54         return [(w["start"], w["end"]) for w in wins]
55
56     return predict_fn
57
58
59 def default_grid() -> List[Config]:
60     return [(vt, mg, mwd)
61             for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
62             for mg in (1.0, 2.0, 3.0)
63             for mwd in (1.0, 2.0, 3.0)]
64
65
66 def run(trajectory_csv: str, labels_csv: str, fps: float, frame_width: float,
67         iou: float = 0.5) -> dict:
68     gts = load_golden_gts(labels_csv)
69     if not gts:
70         raise SystemExit(f"no usable golden rallies in {labels_csv}")
71     predict_fn = make_predict_fn(trajectory_csv, fps, frame_width)
72     grid = default_grid()
73
74     base = evaluate(predict_fn(BASELINE), gts, iou)
75     best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
76     fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou) # OPTIMISTIC
(fit on full GT)
77     cv = cross_validate(predict_fn, grid, gts, k=5) # HONEST held-

```

```

out recall
78
79     print("=== WASB rally-segmentation calibration ===")
80     print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")

```

65. user (2026-06-18T01:39:35.697Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21    TrackNetRunner.
22    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
24    wsl_tilde_quote
25
26    logger = logging.getLogger(__name__)
27
28    # Windows path to the inference wrapper we stage into WSL.
29    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
30
31    @dataclass
32    class WasbConfig:
33        """Resolved settings for a WASB WSL run (built from config.json ->
34        indexing.wasb)."""
35        distro: str = "Ubuntu"
36        repo_dir: str = "~/models/WASB-SBDT"
37        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
38        conda_env: str = "wasb"
39        weights_path: str = "~/models/WASB-
40        SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
41        sport: str = "badminton"
42        wsl_stage_dir: str = "~/clips"
43        timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
44        keep_frames: bool = False # retain staged video + frame cache after success (debug
45        only)
46        min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
47        off)
48
49        # FAST PATH (default OFF until verified): decode the staged video on the fly inside
50        # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
51        # extraction that dominates a long clip. Output is bit-identical to the PNG path
52        # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
53        it.
54        stream_video: bool = False
55
56    @classmethod
57    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
58        w = (idx_cfg or {}).get("wasb", {}) or {}

```

```

52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                  f"python -c \"import torch; print('torch', torch.__version__, "
76                  f"\"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86             return True, out
87
88         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89             src_mnt = to_wsl_mnt_path(video_win_path)
90             reason = self.wsl_source_unreadable_reason(src_mnt)
91             if reason:
92                 logger.error("Cannot stage video into WSL: %s", reason)
93                 return None
94             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96             try:
97                 res = self._wsl_bash(cmd)
98             except subprocess.TimeoutExpired:
99                 logger.error("Staging video into WSL timed out.")
100                 return None
101             if res.returncode != 0 or "OK" not in (res.stdout or ""):
102                 logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103                 return None
104             return dest
105
106         def _stage_infer_script(self) -> bool:
107             """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108             src_mnt = to_wsl_mnt_path(_INFER_WIN)
109             cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110             res = self._wsl_bash(cmd)
111             return res.returncode == 0 and "OK" in (res.stdout or "")
112

```

```

113 def run_predict(self, video_win_path: str, output_win_dir: str,
114                 video_id: Optional[str] = None) -> Optional[str]:
115     """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117     All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118     two videos that share a filename (e.g. two ``match.mp4`` from different folders)
119     can never reuse each other's staged frames, resumable cache, or CSV.
120
121     """
122     # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
123     # translation: to_wsl_mnt_path passes relative paths through unchanged, so
124     # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
125     # the CSV landed inside WSL while Windows polled the repo-relative path.
126     output_win_dir = os.path.abspath(output_win_dir)
127     os.makedirs(output_win_dir, exist_ok=True)
128     # Normalize for cross-platform basename (tests use Windows paths on Linux).
129     video_win_path = str(video_win_path).replace("\\", "/")
130     base = os.path.basename(video_win_path)
131     stem, ext = os.path.splitext(base)
132     # Unique, content-derived key: same filename + different bytes -> different key.
133     key = f"{stem}_{video_id[:12]}" if video_id else stem
134     wsl_out_dir = to_wsl_mnt_path(output_win_dir)
135     expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
136
137     if not self._stage_infer_script():
138         logger.error("Could not stage wasb_infer.py into WSL.")
139         return None
140     staged_video = self._stage_video(video_win_path, f"{key}{ext}")
141     if staged_video is None:
142         return None
143     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
144
145     # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
146     # WSL stage filesystem is already low so the user can clean up before it fills.
147     # (Streaming never materializes the PNGs, so the guard only matters off the fast
148     path.)
149     if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
150         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
151         if free is not None and free < self.cfg.min_free_gb:
152             logger.warning(
153                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
154                 extraction "
155                 "may fill the disk ? consider running tools/cleanup_caches.py.",
156                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
157
158     # Fast path: stream-decode the video in-process (no PNG extraction). Output is
159     # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
160     clip.
161     if self.cfg.stream_video:
162         frame_args = "--stream-video "
163     else:
164         frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
165     cmd = (
166         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
167         f"python wasb_infer.py "
168         f"--video {wsl_tilde_quote(staged_video)} "
169         f"{frame_args}"
170         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
171         f"--sport {shlex.quote(self.cfg.sport)} "
172         f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}")
173     )
174     logger.info("Running WASB inference on %s ...", base)
175     try:
176         res = self._wsl_bash(cmd)
177     except subprocess.TimeoutExpired:
178         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)

```

```

175         return None
176     if res.returncode != 0:
177         logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178         return None
179     if not os.path.exists(expected_csv_win):
180         logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181         return None
182     logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184     # Success ? the trajectory CSV is the durable output; the staged video copy and
185     # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186     # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187     # so the frame cache + detector cache remain for a resumable re-run.
188     # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189     if not self.cfg.keep_frames:
190         stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191         logger.info("Cache hygiene: removing staged video%s for %s "
192                     "(set indexing.wasb.keep_frames=true to retain).",
193                     "" if self.cfg.stream_video else " + frame cache", key)
194         self._wsl_rm_rf(stale)
195         return expected_csv_win
196
197     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

66. assistant (2026-06-18T01:39:39.855Z)

These are clean. Let me check the remaining eval modules I haven't seen which involve serialization/computation that might be under-examined: `experiment.py` (the CONTROL/predict path the golden fixtures depend on), `promotion.py`, and `calibration.py` (cross_validate, k-fold split logic ? easy off-by-one). Let me look at calibration.py and experiment.py.

67. user (2026-06-18T01:39:40.979Z)

```

1     """Quantify rally-segmentation improvement ? and guard against overfitting.
2
3     This is the measurement layer for per-video / per-corpus calibration. It answers
4     two questions honestly:
5
6     1. """Did precision improve, by how much?""" ? `improvement_report` scores a
7       baseline config vs a tuned config on the same ground truth and reports the
8       before/after metrics and the % delta.
9
10    2. """Is that gain real or overfit?""" ? calibration must be measured on data the
11       tuning never saw, otherwise you're just memorising. Two protocols:
12       - `cross_validate` ? k-fold over one video's rallies: pick the config on the
13       train folds, score on the held-out fold. The generalization_gap
14       (train ? test) is the overfit alarm.
15       - `leave_one_video_out` ? the protocol that matters as more videos arrive:
16       pick the config on all other videos, score on the held-out video. Reports
17       per-video held-out scores + mean±std. This is the number to trust.
18
19    Everything is segmenter-agnostic: callers pass a `predict_fn(config) ->
20    List[Interval]` (e.g. WASB trajectory ? windows at those thresholds), so the same
21    harness works for WASB, yolo_hybrid, or anything else. Pure + unit-tested; no I/O.
22    """
23    from __future__ import annotations

```

```

24
25 import statistics as _st
26 from typing import Callable, Dict, List, Sequence, Tuple, Any
27
28 from backend.eval.metrics import score, Score, Interval
29
30 Config = Any                                # opaque to this module (e.g. a thresholds
dict)
31 PredictFn = Callable[[Config], List[Interval]]
32
33
34 def pct_improvement(before: float, after: float) -> float:
35     """Percentage change from `before` to `after` (e.g. 0.6 -> 0.75 == +25.0)."""
36     if before <= 0:
37         return float("inf") if after > 0 else 0.0
38     return (after - before) / before * 100.0
39
40
41 def evaluate(preds: List[Interval], gts: List[Interval], iou_threshold: float = 0.5) ->
Score:
42     """Precision / recall / F1 / mean-IoU of predicted vs ground-truth intervals."""
43     return score(preds, gts, iou_threshold)
44
45
46 def kfold_indices(n: int, k: int) -> List[Tuple[List[int], List[int]]]:
47     """Deterministic k-fold split of indices 0..n-1 into (train_idx, test_idx)."""
48     if n <= 0:
49         return []
50     k = max(2, min(k, n))                # need >=2 folds; cap at n
51     folds = [list(range(i, n, k)) for i in range(k)]    # round-robin ? balanced,
deterministic
52     splits = []
53     for f in range(k):
54         test = folds[f]
55         train = [i for i in range(n) if i not in set(test)]
56         if test and train:
57             splits.append((train, test))
58     return splits
59
60
61 def select_best(predict_fn: PredictFn, configs: Sequence[Config], gts: List[Interval],
62                 metric: str = "f1", iou_threshold: float = 0.5) -> Tuple[Config, float]:
63     """Pick the config whose predictions maximise `metric` against `gts`."""
64     best_cfg, best_val = None, -1.0
65     for cfg in configs:
66         val = getattr(score(predict_fn(cfg), gts, iou_threshold), metric)
67         if val > best_val:
68             best_cfg, best_val = cfg, val
69     return best_cfg, best_val
70
71
72 def cross_validate(predict_fn: PredictFn, configs: Sequence[Config], gts:
List[Interval],
73                   k: int = 5, metric: str = "recall", iou_threshold: float = 0.5) ->
Dict[str, Any]:
74     """k-fold threshold calibration on ONE video's rallies (within-video overfit guard).
75
76     Per fold: choose the best config on the train rallies, score it on the held-out
77     rallies. A large `generalization_gap` (train >> test) means the thresholds are
78     overfitting that video's quirks.
79
80     Default metric is recall on purpose: predictions are global (whole-video
81     windows), so scoring them against a fold subset of GT would unfairly count the
82     other rallies' windows as false positives ? precision/F1 are not well-defined
83     per-fold. Recall ("did the tuned thresholds still detect the held-out rallies?")

```



```

84     is the honest within-video generalization signal. For precision/F1, use
85     `leave_one_video_out` (each video scored whole) or `improvement_report` on a
86     held-out video.
87     """
88     splits = kfold_indices(len(gts), k)
89     if not splits:
90         return {"error": "not enough rallies for cross-validation", "n": len(gts)}
91     train_scores, test_scores, picked = [], [], []
92     for train_idx, test_idx in splits:
93         gts_tr = [gts[i] for i in train_idx]
94         gts_te = [gts[i] for i in test_idx]
95         cfg, tr = select_best(predict_fn, configs, gts_tr, metric, iou_threshold)
96         te = getattr(score(predict_fn(cfg), gts_te, iou_threshold), metric)
97         train_scores.append(tr)
98         test_scores.append(te)
99         picked.append(cfg)
100     mean_test = _st.mean(test_scores)
101     mean_train = _st.mean(train_scores)
102     return {
103         "metric": metric,
104         "folds": len(splits),
105         "mean_test": mean_test,
106         "std_test": _st.pstdev(test_scores) if len(test_scores) > 1 else 0.0,
107         "mean_train": mean_train,
108         "generalization_gap": mean_train - mean_test,    # >~0.1 = likely overfitting
109         "picked_configs": picked,
110     }
111
112
113 def leave_one_video_out(videos: List[Dict[str, Any]], configs: Sequence[Config],
114                         metric: str = "f1", iou_threshold: float = 0.5) -> Dict[str,
115 Any]:
116     """Leave-one-video-out CV ? the honest metric as the corpus grows.
117
118     `videos`: list of {"name": str, "predict_fn": PredictFn, "gts": [Interval]}.
119     For each held-out video: pick the config that's best *pooled across the other
120     videos*, then score it on the held-out one. The mean held-out score is the
121     "precision on unseen videos" number; per-video spread shows stability.
122     """
123     if len(videos) < 2:
124         return {"error": "need >=2 videos for leave-one-video-out", "n": len(videos)}
125     per_video = []
126     for i, held in enumerate(videos):
127         others = [v for j, v in enumerate(videos) if j != i]
128
129         def pooled_score(cfg: Config, others: List[dict] = others) -> float:
130             vals = [getattr(score(v["predict_fn"](cfg), v["gts"], iou_threshold),
131 metric)
132                         for v in others]
133             return _st.mean(vals)
134
135         best_cfg, best_train = max(((c, pooled_score(c)) for c in configs), key=lambda
136 t: t[1])
137         held_score = getattr(score(held["predict_fn"](best_cfg), held["gts"],
138 iou_threshold), metric)
139         per_video.append({"video": held.get("name", f"video_{i}"),
140                         "held_out": held_score, "train_pooled": best_train, "config":
141 best_cfg})
142     held_scores = [p["held_out"] for p in per_video]
143     train_scores = [p["train_pooled"] for p in per_video]
144     return {
145         "metric": metric,
146         "mean_held_out": _st.mean(held_scores),
147         "std_held_out": _st.pstdev(held_scores) if len(held_scores) > 1 else 0.0,
148         "generalization_gap": _st.mean(train_scores) - _st.mean(held_scores),

```

```

144         "per_video": per_video,
145     }
146
147
148     def improvement_report(predict_fn: PredictFn, baseline_config: Config, tuned_config:
Config,
149                             gts: List[Interval], iou_threshold: float = 0.5) -> Dict[str,
Any]:
150         """The "+X% precision" headline: baseline vs tuned config on the same GT.
151
152         NB: for an *honest* headline, pass GT the tuning never saw (a held-out video),
153         or read the cross-validated `mean_test` from `cross_validate` instead.
154         """
155         b = score(predict_fn(baseline_config), gts, iou_threshold)
156         t = score(predict_fn(tuned_config), gts, iou_threshold)
157         return {
158             "baseline": {"precision": b.precision, "recall": b.recall, "f1": b.f1,
"mean_iou": b.mean_iou},
159             "tuned": {"precision": t.precision, "recall": t.recall, "f1": t.f1, "mean_iou":
t.mean_iou},
160             "precision_delta_pts": t.precision - b.precision,
161             "precision_delta_pct": pct_improvement(b.precision, t.precision),
162             "f1_delta_pct": pct_improvement(b.f1, t.f1),
163         }
164

```

68. assistant (2026-06-18T01:39:46.849Z)

These are clean. I've now done a thorough cross-cut of the production runtime path, the eval/metrics surface, serialization paths, error handlers, the DB migration ladder, the API boundary, and the under-examined detector/window math.

Let me look at a few final under-examined modules I haven't opened: `experiment.py` (the predict/CONTROL path that golden_fixtures relies on), the config models/loader (validation), and `gemini\_refine.merge\_overlaps` (used by distill_local). Let me check experiment.py and gemini_refine.

69. user (2026-06-18T01:39:48.537Z)

```

1     """Refine WASB rally candidates with Gemini (verify boundaries + reject false fires).
2
3     WASB (shuttle-only) over-fires on between-point service feeds. This drives the
4     EXISTING Gemini provider over the SHORT WASB candidate clips (not the whole video):
5     each candidate window is cut to a small padded clip and handed to Gemini, which ?
6     seeing the actual footage ? returns the true rally inside it (tight boundaries) or
7     nothing (it was a toss / dead time). Cheap because uploads are seconds-long clips,
8     not GB-scale chunks; high-precision because Gemini judges each candidate visually.
9
10    Reuses (no reinvented wheels): providers.GeminiProvider (via provider_registry),
11    sports.badminton prompt (sport_registry), utils.video.extract_video_chunk,
12    calibrate_wasb.make_predict_fn, export_wasb_segments.write_segments_csv.
13
14    This mirrors wasb_hybrid's Phase-3 handoff but runs standalone on a cached
15    trajectory (no WASB re-run, no DB) ? an eval/tuning driver, not the live pipeline.
16
17    Run (GEMINI_API_KEY in env):
18        python -m backend.eval.gemini_refine --video Adarsh.MP4 --trajectory adarsh_traj.csv \
19            --fps 59.94 --frame-width 1920 --out output/adarsh_gemini.csv [--limit 4]
20    """
21    from __future__ import annotations
22
23    import os
24    import os.path as osp
25    import time

```

```

26     import argparse
27     import concurrent.futures
28     from typing import List, Tuple, Optional
29
30     from backend.eval.calibrate_wasb import make_predict_fn, BASELINE
31     from backend.eval.export_wasb_segments import TUNED, write_segments_csv, seconds_to_mmss
32     from backend.utils.video import get_video_duration, extract_video_chunk
33     from backend.sports import sport_registry
34     from backend.providers import provider_registry
35
36     Interval = Tuple[float, float]
37
38
39     def merge_overlaps(intervals: List[Interval], gap_tol: float = 0.0) -> List[Interval]:
40         """Union of overlapping intervals, optionally stitching across a small gap.
41
42         gap_tol > 0 merges two intervals separated by <= gap_tol seconds ? used to heal
43         shuttle-invisibility splits WITHIN one rally, while keeping genuine between-rally
44         gaps
45         (several seconds) as separate rallies. (Replaces the old envelope-per-candidate
46         logic, which over-merged multiple real rallies WASB had bundled into one candidate.)"""
47         merged: List[Interval] = []
48         for s, e in sorted(intervals):
49             if merged and s <= merged[-1][1] + gap_tol:
50                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
51             else:
52                 merged.append((s, e))
53         return merged
54
55     def _offset_segments(raw: list, clip_start: float, clip_end: float) -> List[Interval]:
56         """Map a Gemini clip's returned segments back to full-video seconds, clamped to
57         clip."""
58         out: List[Interval] = []
59         for sg in raw or []:
60             try:
61                 st = float(sg["start_time"]) + clip_start
62                 en = float(sg["end_time"]) + clip_start
63             except (TypeError, ValueError, KeyError):
64                 continue
65             st = max(clip_start, st)
66             en = min(clip_end, en)
67             if en > st:
68                 out.append((round(st, 3), round(en, 3)))
69         return out
70
71     def refine_window(api_key: str, model: str, sport_profile, video: str, window: Interval,
72                       pad: float, duration: float, tmpdir: str, idx: int,
73                       clip_scale_width: Optional[int] = 1280) -> Tuple[int, Interval,
74                       List[Interval]]:
75         # Own provider/client per call: the genai client isn't reliably thread-safe, and a
76         # shared one across workers throws socket errors. A fresh lazy client is cheap.
77         provider = provider_registry.get("gemini", api_key=api_key, model_name=model)
78         s, e = window
79         cs = max(0.0, s - pad)
80         ce = min(duration, e + pad) if duration > 0 else e + pad
81         clip = osp.join(tmpdir, f"cand_{idx:04d}.mp4")
82         # Downscale the uploaded clip (Gemini analyses at low res) -> small, fast, under the
83         # 500MB upload cap even for 4K/5K sources.
84         if not extract_video_chunk(video, cs, ce - cs, clip, reencode=True,
85                                   scale_width=clip_scale_width):
86             return idx, window, []

```

```

85         try:
86             prompt = sport_profile.get_prompt(chunk_duration=ce - cs)
87             raw, _ = provider.analyze_video(clip, prompt, chunk_duration=ce - cs,
label=f"cand{idx}")
88             mapped = _offset_segments(raw, cs, ce)
89         finally:
90             try:
91                 os.remove(clip)
92             except OSError:
93                 pass
94         # Keep Gemini's segments separate ? a WASB candidate can bundle several real rallies
95         # (merge_gap bridged the between-point gaps), and collapsing them to one envelope
swept
96         # dead time into a "rally" (precision loss). Tiny invisibility splits are healed
later
97         # by the gap-tolerant global merge in run().
98         return idx, window, mapped
99
100
101     def run(video: str, trajectory: str, fps: float, frame_width: float,
102           cfg=BASELINE, pad: float = 3.0, model: str = "gemini-flash-latest",
103           max_workers: int = 4, out_csv: str = "output/gemini_refined.csv",
104           min_dur: float = 2.0, limit: int = 0, clip_scale_width: Optional[int] = 1280) ->
dict:
105         api_key = os.environ.get("GEMINI_API_KEY")
106         if not api_key:
107             raise SystemExit("GEMINI_API_KEY not set in env")
108         sport_profile = sport_registry.get("badminton")
109         duration = get_video_duration(video)
110
111         candidates = make_predict_fn(trajectory, fps, frame_width)(cfg)
112         if limit:
113             candidates = candidates[:limit]
114         tmpdir = osp.join("output", "gemini_refine_clips")
115         os.makedirs(tmpdir, exist_ok=True)
116
117         print(f"[gemini_refine] {len(candidates)} WASB candidates (cfg {cfg}) -> Gemini
{model}, "
118               f"pad {pad}s, {max_workers} workers", flush=True)
119         t0 = time.time()
120         per_candidate = []
121         with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
122             futs = [ex.submit(refine_window, api_key, model, sport_profile, video, w, pad,
duration,
123                             tmpdir, i, clip_scale_width)
124                     for i, w in enumerate(candidates)]
125             for fut in concurrent.futures.as_completed(futs):
126                 idx, window, mapped = fut.result()
127                 per_candidate.append((idx, window, mapped))
128                 tag = "REJECT (no rally)" if not mapped else \
129                     " ".join(f"{seconds_to_mmss(a)}-{seconds_to_mmss(b)}" for a, b in
mapped)
130                 print(f"  cand {idx:>2}
{seconds_to_mmss(window[0])}-{seconds_to_mmss(window[1])} -> {tag}", flush=True)
131
132             raw_intervals = [iv for _, _, m in per_candidate for iv in m]
133             refined = [(s, e) for s, e in merge_overlaps(raw_intervals, gap_tol=1.0) if e - s >=
min_dur]
134             write_segments_csv(out_csv, refined)
135             try:
136                 os.rmdir(tmpdir)
137             except OSError:
138                 pass
139
140             kept_cands = sum(1 for _, _, m in per_candidate if m)

```

```

141     play = sum(e - s for s, e in refined)
142     print(f"\n[gemini_refine] {kept_cands}/{len(candidates)} candidates had a rally; "
143           f"{len(refined)} merged rallies, {play:.1f}s play -> {out_csv} "
144           f"({time.time() - t0:.0f}s)", flush=True)
145     return {"refined": refined, "num_candidates": len(candidates),
146            "candidates_with_rally": kept_cands, "out_csv": out_csv}
147
148
149     def full_video_rallies(video: str, model: str = "gemini-flash-latest",
150                           out_csv: str = "output/gemini_fullvideo.csv", clip_scale_width:
Optional[int] = 1280,
151                           min_dur: float = 2.0, chunk_sec: float = 120.0, overlap: float =
10.0,
152                           max_workers: int = 3) -> List[Interval]:
153         """Full-context pass: hand Gemini the WHOLE clip (downscaled, chunked to stay under
the
154         upload cap), so it finds every rally itself ? not bounded by WASB's candidate
recall.
155         Independent of the trajectory; complements refine()."""
156         api_key = os.environ.get("GEMINI_API_KEY")
157         if not api_key:
158             raise SystemExit("GEMINI_API_KEY not set in env")
159         sport_profile = sport_registry.get("badminton")
160         duration = get_video_duration(video)
161         starts, t = [], 0.0
162         while t < duration:
163             starts.append(t)
164             t += max(1.0, chunk_sec - overlap)
165         tmpdir = osp.join("output", "gemini_full_clips")
166         os.makedirs(tmpdir, exist_ok=True)
167
168         def do_chunk(ci: int, cstart: float) -> List[Interval]:
169             cend = min(cstart + chunk_sec, duration)
170             clip = osp.join(tmpdir, f"chunk_{ci:03d}.mp4")
171             if not extract_video_chunk(video, cstart, cend - cstart, clip, reencode=True,
scale_width=clip_scale_width):
172                 return []
173             try:
174                 provider = provider_registry.get("gemini", api_key=api_key,
model_name=model)
175                 prompt = sport_profile.get_prompt(chunk_duration=cend - cstart)
176                 raw, _ = provider.analyze_video(clip, prompt, chunk_duration=cend - cstart,
label=f"chunk{ci}")
177                 return _offset_segments(raw, cstart, cend)
178             finally:
179                 try:
180                     os.remove(clip)
181                 except OSError:
182                     pass
183
184         print(f"[gemini_full] {len(starts)} chunk(s) of {video} (<= {chunk_sec}s, width
{clip_scale_width}) "
185               f"-> Gemini {model}", flush=True)
186         with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
187             chunks = [f.result() for f in [ex.submit(do_chunk, i, s) for i, s in
enumerate(starts)]]
188         rallies = [(s, e) for s, e in merge_overlaps([iv for c in chunks for iv in c],
gap_tol=1.0) if e - s >= min_dur]
189         write_segments_csv(out_csv, rallies)
190         try:
191             os.rmdir(tmpdir)
192         except OSError:
193             pass
194         play = sum(e - s for s, e in rallies)
195         print(f"[gemini_full] {len(rallies)} rallies, {play:.1f}s play -> {out_csv}",

```

```

flush=True)
196         return rallies
197
198
199     def main() -> None:
200         ap = argparse.ArgumentParser(description="Refine WASB candidates with Gemini.")
201         ap.add_argument("--video", required=True)
202         ap.add_argument("--trajectory", default=None, help="WASB trajectory CSV (required
unless --full-video)")
203         ap.add_argument("--fps", type=float, default=59.94)
204         ap.add_argument("--frame-width", type=float, default=1920.0)
205         ap.add_argument("--config", choices=["baseline", "tuned"], default="baseline",
206                         help="WASB candidate config to feed Gemini (baseline = whole
rallies, good recall)")
207         ap.add_argument("--pad", type=float, default=3.0)
208         ap.add_argument("--model", default="gemini-flash-latest")
209         ap.add_argument("--max-workers", type=int, default=4)
210         ap.add_argument("--min-dur", type=float, default=2.0)
211         ap.add_argument("--out", default="output/gemini_refined.csv")
212         ap.add_argument("--limit", type=int, default=0, help="only the first N candidates
(smoke test)")
213         ap.add_argument("--clip-scale-width", type=int, default=1280,
214                         help="downscale uploaded clips to this width (0=keep source res)")
215         ap.add_argument("--full-video", action="store_true",
216                         help="full-context pass: send the whole clip (chunked/downscaled) to
Gemini ")
217         ap.add_argument("--no-refine", action="store_true", help="instead of refining WASB candidates (needs no --trajectory)")
218         args = ap.parse_args()
219         scale = args.clip_scale_width or None
220         if args.full_video:
221             full_video_rallies(args.video, model=args.model, out_csv=args.out,
222                               clip_scale_width=scale, min_dur=args.min_dur,
max_workers=args.max_workers)
223             return
224         if not args.trajectory:
225             raise SystemExit("--trajectory is required unless --full-video")
226         cfg = BASELINE if args.config == "baseline" else TUNED
227         run(args.video, args.trajectory, args.fps, args.frame_width, cfg=cfg, pad=args.pad,
228             model=args.model, max_workers=args.max_workers, out_csv=args.out,
229             min_dur=args.min_dur, limit=args.limit, clip_scale_width=scale)
230
231
232 if __name__ == "__main__":
233     main()
234

```

70. user (2026-06-18T01:39:48.665Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`      ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`      ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`    ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine` ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`    ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`   ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17    (per-frame signals ? run once on the GPU/WSL box, persisted into

```

```

18     `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those
19     signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20     given persisted candidate features it produces `List[Interval]` deterministically,
21     with no GPU and zero production risk. So most experiments are pure offline
22     re-scoring of stored data and never touch the served pipeline.
23
24     Three modes (`EXPERIMENT_HARNESS.md` §5):
25     - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26       aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27       held-out loop, but variant-parameterised.
28     - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29       variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30       human spot-checks (and what two highlight reels would show side by side).
31     - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32       (exit 0/1/3); **rollback = flip the variant/config, never `git revert`.**
33
34     Pure + offline + unit-tested. The only DB-touching helper is
35     `load_video_candidates`, which reads persisted candidates via
36     `Database.query_candidate_segments`.
37     """
38     from __future__ import annotations
39
40     import json
41     import logging
42     import os
43     from dataclasses import dataclass, field
44     from datetime import datetime, timezone
45     from hashlib import sha1
46     from typing import Any, Callable, Dict, List, Optional, Sequence
47
48     from backend.eval.calibration import pct_improvement
49     from backend.eval.classifier import LogisticRegression
50     from backend.eval.gemini_refine import merge_overlaps
51     from backend.eval.metrics import Interval, match_intervals, score
52     from backend.eval.regression import gt_hash
53     from backend.eval.training import label_candidates
54     from backend.eval import eval_stats as _stats # C1: CIs + paired sign test
55     from backend.eval import segmentation_metrics as _segm # C4: F1@k + segment-count
56     ratio
57     from backend.eval.score_calibration import fit_isotonic, fit_platt # C2: calibrate cue
58     scores
59
60     logger = logging.getLogger(__name__)
61
62     # Where a comparison run appends one reproducible record. Tracked location (NOT under
63     # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
64     # regression.DEFAULT_BASELINE_PATH.
65     DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
66     _METRICS = ("precision", "recall", "f1", "mean_iou")
67
68     class ExperimentError(ValueError):
69         """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70         score an unlabeled video with no pinned model, or a gate referencing an absent
71         feature)."""
72
73     # ----- Variant
74
75     @dataclass
76     class Variant:
77         """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
78
79         A variant turns one video's persisted candidate windows + features into rally

```

```

81 intervals. Every knob defaults to the **control** behaviour (no gate, no learned
82 filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83 to control ? the core no-regression guarantee.
84
85 Fields:
86 - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87   for the leaderboard and for selecting the served path (the existing
88   `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89   New cues are recorded here default-OFF.
90 - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91   ``net_crossings`` feature is  $\geq$  this. 0 = off. This is the eval-only gate from
92   `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93   windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94 - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95   whose persisted ``players_in_court`` is  $\geq$  this. 0 = off (no-op until the person
96   cue persists that feature).
97 - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98   `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99   required for `compare_unlabeled` on a learned variant.
100 - ``feature_names`` ? the persisted features the learned filter consumes. When set
101   WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102   (honest) ? the recipe, not a pinned artifact.
103 - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104   overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105 """
106
107 name: str
108 segmenter: str = "wasb_hybrid"
109 config_overrides: Dict[str, Any] = field(default_factory=dict)
110 cue_flags: Dict[str, bool] = field(default_factory=dict)
111 min_crossings: int = 0
112 min_players: int = 0
113 classifier_model: Optional[str] = None
114 feature_names: Optional[List[str]] = None
115 threshold: float = 0.5
116 merge_gap: float = 1.0
117 # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118 # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119 # failure where a fixed 0.5 zeroed out a small-candidate video.
120 tune_threshold: bool = False # pick the F1-best threshold on the train fold
121 calibrate: Optional[str] = None # None | "platt" | "isotonic" ? calibrate
proba first
122
123 def is_learned(self) -> bool:
124     """True if this variant applies a learned classifier (pinned model or
recipe)."""
125     return self.classifier_model is not None or bool(self.feature_names)
126
127 def to_dict(self) -> dict:
128     return {
129         "name": self.name,
130         "segmenter": self.segmenter,
131         "config_overrides": self.config_overrides,
132         "cue_flags": self.cue_flags,
133         "min_crossings": self.min_crossings,
134         "min_players": self.min_players,
135         "classifier_model": self.classifier_model,
136         "feature_names": self.feature_names,
137         "threshold": self.threshold,
138         "merge_gap": self.merge_gap,
139         "tune_threshold": self.tune_threshold,
140         "calibrate": self.calibrate,

```



```

141         }
142
143     @classmethod
144     def from_dict(cls, d: dict) -> "Variant":
145         known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146         return cls(**{k: v for k, v in d.items() if k in known})
147
148     def spec_hash(self) -> str:
149         """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150         and makes a result reproducible. The pinned **control** variant always hashes
151         the
152         same, so a regression is always traceable to a recipe change, never to drift."""
153         blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
154         return sha1(blob.encode()).hexdigest()[:12]
155
156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     `training.extract_yolo_features`, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196     cols = [_feature_column(vc, name) for name in feature_names]
197     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202     players)."""
203     n = len(vc.intervals)

```

```

203     mask = [True] * n
204     if variant.min_crossings > 0:
205         col = _feature_column(vc, "net_crossings")
206         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
207     if variant.min_players > 0:
208         col = _feature_column(vc, "players_in_court")
209         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
210     return mask
211
212
213     def predict(variant: Variant, vc: VideoCandidates,
214                 model: Optional[LogisticRegression] = None,
215                 threshold: Optional[float] = None,
216                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217         """Variant prediction: gate by persisted features, optionally filter by a learned
218         model, then merge. Pure and deterministic.
219
220         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225         them.
226
227         """
228         mask = _gate_mask(variant, vc)
229         if variant.feature_names:
230             if model is None:
231                 if variant.classifier_model is None:
232                     raise ExperimentError(
233                         f"variant {variant.name!r} is learned but has no model to predict "
234                         f"with (pass a fitted model or set classifier_model)")
235                 model = LogisticRegression.load(variant.classifier_model)
236                 proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
237 variant.feature_names))]
238                 if calibrator is not None:
239                     proba = [calibrator(p) for p in proba]
240                 thr = variant.threshold if threshold is None else threshold
241                 mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
242                 kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
243                 return merge_overlaps(kept, gap_tol=variant.merge_gap)
244
245     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
246                             train_videos: Sequence[VideoCandidates], iou: float
247                             ) -> "tuple[Optional[Callable[[float], float]],
248 Optional[float]]":
249         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
250         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
251         (use the variant default). Honest: never looks at the held-out video.
252
253         """
254         calibrator: Optional[Callable[[float], float]] = None
255         if variant.calibrate:
256             raw: List[float] = []
257             y: List[int] = []
258             for vc in train_videos:
259                 raw.extend(float(p) for p in
260                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
261 [])))
262                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
263             if variant.calibrate == "platt":
264                 calibrator = fit_platt(raw, y)
265             elif variant.calibrate == "isotonic":
266                 calibrator = fit_isotonic(raw, y)
267             else:
268                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use

```

```

'platt'/'isotonic'))
264     threshold: Optional[float] = None
265     if variant.tune_threshold:
266         best_t, best_f1 = variant.threshold, -1.0
267         for k in range(1, 20):
268             t = round(0.05 * k, 2)
269             f1s = [score(predict(variant, vc, model=model, threshold=t,
270                                 vc.gts or [], iou).f1 for vc in train_videos]
271             mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272             if mean_f1 > best_f1:
273                 best_f1, best_t = mean_f1, t
274             threshold = best_t
275     return calibrator, threshold
276
277
278     # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
283         the
284         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
285         X: List[List[float]] = []
286         y: List[int] = []
287         for vc in videos:
288             if vc.gts is None:
289                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
290             X.extend(_feature_matrix(vc, variant.feature_names or []))
291             y.extend(label_candidates(vc.intervals, vc.gts, iou))
292         if not X:
293             raise ExperimentError("cannot train a learned variant on zero candidates")
294         return LogisticRegression().fit(X, y, variant.feature_names)
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
303         training,
304         so no leakage; per-video scoring is already honest.
305         - **learned, pinned model** (`classifier_model` set): score every video with the
306         SHIPPED model. ? optimistic if those videos were in its training set ? use this
307         to
308         report "how the shipped artifact does here", not for the promotion decision.
309         - **learned recipe** (`feature_names`, no pinned model) + `honest=True`: LOVO
310         ?
311         train on the OTHER videos, predict the held-out one. The number to trust.
312         - learned recipe + `honest=False`: train on all, predict each (overfit; sanity
313         only).
314
315         """
316         for vc in videos:
317             if vc.gts is None:
318                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
319
320
321         per_video: List[Dict[str, Any]] = []
322         predictions: Dict[str, List[Interval]] = {}
323
324         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None

```

```

319         pinned_model = (LogisticRegression.load(variant.classifier_model)
320                         if variant.classifier_model is not None else None)
321     all_model = None
322     if recipe_lovo and not honest:
323         all_model = _train(variant, videos, iou)
324
325     for i, vc in enumerate(videos):
326         cal: Optional[Callable[[float], float]] = None
327         thr: Optional[float] = None
328         if recipe_lovo and honest:
329             others = [v for j, v in enumerate(videos) if j != i]
330             if not others:
331                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332             model: Optional[LogisticRegression] = _train(variant, others, iou)
333             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                 assert model is not None # _train never returns
None
335                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                     for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:

```

```

374         f1k[ov].append(got[ov])
375         ratios.append(_segm.segment_count_ratio(preds, gts))
376
377     def _mean(xs: List[float]) -> float:
378         return sum(xs) / len(xs) if xs else 0.0
379     out: Dict[str, Any] = {"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380     out["segment_count_ratio"] = _mean(ratios)
381     return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                             "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404                iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (EXPERIMENT_HARNESS.md §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}

```

```

425
426 by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427 per_video_delta = [
428     {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429     for p in eval_b["per_video"]]
430 ]
431
432 # One fingerprint over the whole label set ? detects "the deltas were measured
against
433 # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434 all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436 result: Dict[str, Any] = {
437     "iou": iou,
438     "label_set_hash": gt_hash(all_gts),
439     "num_videos": len(videos),
440     "variant_a": eval_a,
441     "variant_b": eval_b,
442     "delta": delta,
443     "delta_pct": delta_pct,
444     "per_video_delta": per_video_delta,
445 }
446 if honest_stats:
447     result["honest"] = honest_summary(videos, eval_a, eval_b)
448 return result
449
450
451 # ----- SxS on unlabeled video
452
453
454 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455     agree_iou: float = 0.5) -> Dict[str, Any]:
456     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458     With no labels there is no lift to measure ? instead this surfaces where the two
459     variants **agree vs diverge**, which is exactly what a human reviews (and what two
460     highlight reels would show side by side):
461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPS:
465       the human / a later golden label adjudicates).
466
467     Both variants must be predictable without labels: a learned variant therefore needs
468     a
469     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470     ``metrics.match_intervals`` ? no new matching logic.
471     """
472     preds_a = predict(variant_a, video)
473     preds_b = predict(variant_b, video)
474     mr = match_intervals(preds_a, preds_b, agree_iou)
475
476     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
477               for m in mr.matches]
478     agree_iou = [m.iou for m in mr.matches]
479     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
480     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
481
482     def _dur(ivs: List[Interval]) -> float:
483         return sum(e - s for s, e in ivs)
484
485     return {
486         "video": video.name,
487         "agree_iou": agree_iou,

```

```

487         "variant_a": variant_a.name,
488         "variant_b": variant_b.name,
489         "num_a": len(preds_a),
490         "num_b": len(preds_b),
491         "num_agreed": len(agreed),
492         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
493         "duration_a": _dur(preds_a),
494         "duration_b": _dur(preds_b),
495         "agreed": agreed,
496         "a_only": a_only,
497         "b_only": b_only,
498     }
499
500
501     # ----- leaderboard / DB
502
503
504     def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                          timestamp: Optional[str] = None) -> Dict[str, Any]:
506         """Append a compact, reproducible record of a `compare()` result to the JSON
507         leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508         `regression_baseline.json`; deliberately the size of a baseline file, not a service
509         (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510         """
511         row: Dict[str, Any] = {
512             "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513             "iou": result.get("iou"),
514             "label_set_hash": result.get("label_set_hash"),
515             "num_videos": result.get("num_videos"),
516             "variant_a": {"name": result["variant_a"]["variant"],
517                          "spec_hash": result["variant_a"]["spec_hash"],
518                          "aggregate": result["variant_a"]["aggregate"]},
519             "variant_b": {"name": result["variant_b"]["variant"],
520                          "spec_hash": result["variant_b"]["spec_hash"],
521                          "aggregate": result["variant_b"]["aggregate"]},
522             "delta": result.get("delta"),
523         }
524         # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525         the
526         # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527         honest = result.get("honest") or {}
528         if honest.get("paired_delta_f1"):
529             row["honest_delta_f1"] = honest["paired_delta_f1"]
530         entries: List[Dict[str, Any]] = []
531         if os.path.isfile(path):
532             with open(path, encoding="utf-8") as f:
533                 entries = json.load(f)
534         entries.append(row)
535         os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
536         with open(path, "w", encoding="utf-8") as f:
537             json.dump(entries, f, indent=2)
538         logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
539                    row["variant_a"]["name"], row["variant_b"]["name"],
540                    (result.get("delta") or {}).get("f1", 0.0), path)
541         return row
542
543     def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                             gts: Optional[List[Interval]] = None,
545                             feature_names: Sequence[str] = ()) -> VideoCandidates:
546         """Bridge persisted detection ? `VideoCandidates` via
547         `Database.query_candidate_segments`.
548         ``feature_names`` selects which persisted features to lift out of each row's
549         ``stats``

```

```

549     JSON (``"duration"`` is derived from the window bounds). This is the only DB-
touching
550     function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551     but works against any `Database` (including an in-memory one in tests).
552     """
553     rows = db.query_candidate_segments(video_id, detector=detector)
554     intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555     features: Dict[str, List[float]] = {}
556     for name in feature_names:
557         if name == "duration":
558             features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559         else:
560             features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561     return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

71. assistant (2026-06-18T01:39:56.396Z)

I noticed something in `full\_video\_rallies` (gemini_refine.py, lines 161-164). The chunk-start loop:

```

```python
starts, t = [], 0.0
while t < duration:
 starts.append(t)
 t += max(1.0, chunk_sec - overlap)
...

```

If `duration <= 0` (get\_video\_duration returns 0.0 on probe failure), the loop doesn't execute, starts is empty ? fine, no crash. This is a CLI eval driver anyway.

I've now examined a comprehensive cross-cut. Let me do a final check on two modules I haven't seen that handle serialized output / validation that are commonly under-examined: the config loader/models (Pydantic validation), and the `score\_calibration.py` (fit\_platt/fit\_isotonic ? numeric edge cases). Let me check score\_calibration which has math edge cases.

## 72. user (2026-06-18T01:39:57.772Z)

```

1 """Per-cue score calibration (course-correction C2) ? make ``confidence`` mean
something.
2
3 A CNN softmax, an audio onset score, and an integer net-crossing count live on wildly
4 different scales. Fed raw into the learned arbiter (the ADR-004 logistic regression),
the
5 most over-confident cue dominates and starves complementary cues. Calibrating each
6 producer's score to a probability *before* it becomes a feature column makes the
7 ``(value, confidence)`` semantics real and the arbiter's weights stable. See
8 ``docs/ANALYZERS_AND_RECONCILERS.md`` §13/C2.
9
10 Two calibrators, both standard:
11 - **Platt scaling** ? fit ``p = sigmoid(a*s + b)`` (parametric; good for monotone,
12 roughly-sigmoidal cues; uses Platt's target smoothing to avoid 0/1 overfitting).
13 - **Isotonic regression** (pool-adjacent-violators) ? non-parametric monotone fit; more
14 flexible, needs a bit more data.
15
16 Plus ``brier_score`` and ``expected_calibration_error`` to *measure* calibration. All
17 **additive**, pure (stdlib only), deterministic ? fits run in the numpy-free lanes too.
18 Calibrators are tiny dataclasses (their params are JSON-friendly), so a fitted
calibrator
19 can be pinned alongside a variant the way the classifier is.
20 """
21 from __future__ import annotations

```



```

22
23 import bisect
24 import math
25 from dataclasses import dataclass
26 from typing import List, Sequence, Tuple
27
28 _EPS = 1e-12
29
30
31 def _sigmoid(z: float) -> float:
32 if z >= 0:
33 return 1.0 / (1.0 + math.exp(-z))
34 e = math.exp(z)
35 return e / (1.0 + e)
36
37
38 def _clip01(p: float) -> float:
39 return min(1.0 - _EPS, max(_EPS, p))
40
41
42 @dataclass(frozen=True)
43 class PlattCalibrator:
44 """`p = sigmoid(a*score + b)` . Call it on a raw cue score to get a probability."""
45 a: float
46 b: float
47
48 def __call__(self, s: float) -> float:
49 return _clip01(_sigmoid(self.a * s + self.b))
50
51
52 @dataclass(frozen=True)
53 class IsotonicCalibrator:
54 """Piecewise-constant monotone map from raw score to probability (PAVA fit).
55
56 `xs` are sorted training scores, `ys` the non-decreasing fitted probabilities;
57 a query returns the fitted value of the largest `x <= score` (clipped at the
ends).
58 """
59 xs: List[float]
60 ys: List[float]
61
62 def __call__(self, s: float) -> float:
63 if not self.xs:
64 return 0.5
65 idx = bisect.bisect_right(self.xs, s) - 1
66 if idx < 0:
67 idx = 0
68 return self.ys[idx]
69
70
71 def fit_platt(scores: Sequence[float], labels: Sequence[int],
72 iters: int = 200, lr: float = 0.5) -> PlattCalibrator:
73 """Fit Platt scaling by gradient descent on (smoothed) log-loss.
74
75 Scores are standardised internally for a stable fit, then folded back so the
returned
76 `a`/`b` apply to the **raw** score. Platt target smoothing
77 (`t+ = (N+ + 1)/(N+ + 2)`, `t- = 1/(N- + 2)`) keeps the fit off the 0/1 rails.
78 Degenerate inputs (empty, zero-variance, single class) fall back to a sane constant
map.
79 """
80 n = len(scores)
81 if n == 0 or n != len(labels):
82 return PlattCalibrator(a=0.0, b=0.0) # ? p = 0.5 everywhere
83 mean = sum(scores) / n

```

[illegible]

```

147 """Expected calibration error: |accuracy ? confidence| per equal-width bin,
weighted."""
148 n = len(probs)
149 if n == 0 or n != len(labels):
150 return 0.0
151 bins = max(1, n_bins)
152 bin_sum_p = [0.0] * bins
153 bin_sum_y = [0.0] * bins
154 bin_cnt = [0] * bins
155 for p, y in zip(probs, labels):
156 b = min(bins - 1, int(p * bins))
157 bin_sum_p[b] += p
158 bin_sum_y[b] += y
159 bin_cnt[b] += 1
160 ece = 0.0
161 for i in range(bins):
162 if bin_cnt[i]:
163 conf = bin_sum_p[i] / bin_cnt[i]
164 acc = bin_sum_y[i] / bin_cnt[i]
165 ece += (bin_cnt[i] / n) * abs(acc - conf)
166 return ece
167
168
169 def calibration_report(probs: Sequence[float], labels: Sequence[int],
170 n_bins: int = 10) -> Tuple[float, float]:
171 """Convenience: ``(brier, ece)`` for a set of probabilities vs labels."""
172 return brier_score(probs, labels), expected_calibration_error(probs, labels, n_bins)
173

```

### 73. assistant (2026-06-18T01:40:04.569Z)

I notice a potential issue in `\_pava` (line 122). The `fitted.extend([value] \* int(round(weight)))` reconstructs the fitted array by replicating each block value `weight` times. Since weights are accumulated as sums of 1.0, `int(round(weight))` is exact for integer counts. But in `expected\_calibration\_error`, line 156: `b = min(bins - 1, int(p \* bins))`. If `p` is exactly 1.0 (a label can be passed in as `probs`), `int(1.0 \* 10) = 10`, clamped to `bins-1=9`. Good. If `p` could be negative... probs come from calibrators clipped to `[\_EPS, 1-\_EPS]`, so non-negative. Fine.

These are all clean. Let me do a final review of the config loader/models (Pydantic validation is externally-fed) and the `training.py` label\_candidates (the IoU labeling that the whole experiment harness depends on).

### 74. user (2026-06-18T01:40:05.988Z)

```

1 """Build a training set for the offline, API-free learned rally segmenter.
2
3 Phase 4 of the collector->indexer bridge (see docs/ROADMAP.md, ADR-004). The idea:
4 a detector (YOLO player-motion today, optionally WASB shuttle-motion later)
5 produces candidate rally windows with features; the collector's ground-truth
6 rally intervals tell us which windows are real. A lightweight classifier learns
7 that mapping, replacing the paid Gemini refinement with a local model.
8
9 This module is the *data layer* ? pure and unit-tested:
10
11 - `label_candidates` ? generic: label windows positive/negative by whether they
12 match a ground-truth rally (same IoU matching the evaluator uses, so training
13 labels are consistent with how we score).
14 - `extract_yolo_features` ? YOLO-specific feature vector from a candidate row's
15 persisted `stats`. Pluggable: a WASB substrate supplies its own feature_fn.
16 - `build_training_set` ? glue: rows + GT -> (X, y, intervals).
17
18 No model training or I/O here; that lives in the trainer/segmenter once the
19 substrate is chosen from the Phase 3 candidate-recall result.
20 """

```

```

21
22 from typing import Callable, Dict, List, Tuple
23
24 from backend.eval.metrics import Interval, match_intervals
25
26 # YOLO Stage-1 persists these per candidate window in `candidate_segments.stats`
27 # (see HybridYoloSegmenter). Duration is derived from the window bounds.
28 YOLO_FEATURE_NAMES = [
29 "duration", "peak_velocity", "mean_velocity", "active_frame_count",
30 "track_id_count",
31]
32
33 def extract_yolo_features(row: Dict) -> List[float]:
34 """Feature vector for one YOLO candidate row (dict from query_candidate_segments).
35
36 `stats` is already JSON-deserialized by the DB layer. Missing fields default
37 to 0.0 so a sparse/old row still yields a well-formed vector.
38 """
39 stats = row.get("stats") or {}
40 duration = float(row["end_time"]) - float(row["start_time"])
41 return [
42 duration,
43 float(stats.get("peak_velocity", 0.0)),
44 float(stats.get("mean_velocity", 0.0)),
45 float(stats.get("active_frame_count", 0)),
46 float(stats.get("track_id_count", 0)),
47]
48
49
50 def label_candidates(candidate_intervals: List[Interval], gt_intervals: List[Interval],
51 iou_threshold: float = 0.5) -> List[int]:
52 """Label each candidate window 1 (real rally) or 0 (spurious).
53
54 A window is positive iff it is matched one-to-one to a ground-truth rally at
55 `iou_threshold` ? the *same* matching the evaluator uses for true positives,
56 so the classifier is trained on exactly what we score. Substrate-agnostic:
57 works for YOLO or WASB-derived windows alike.
58 """
59 mr = match_intervals(candidate_intervals, gt_intervals, iou_threshold)
60 positive = {m.pred_index for m in mr.matches}
61 return [1 if i in positive else 0 for i in range(len(candidate_intervals))]
62
63
64 def build_training_set(
65 candidate_rows: List[Dict],
66 gt_intervals: List[Interval],
67 iou_threshold: float = 0.5,
68 feature_fn: Callable[[Dict], List[float]] = extract_yolo_features,
69) -> Tuple[List[List[float]], List[int], List[Interval]]:
70 """Turn one video's candidate rows + ground truth into (X, y, intervals).
71
72 `feature_fn` is pluggable so a WASB substrate can supply shuttle-motion
73 features without changing the labeling logic. Returns the feature matrix `X`,
74 binary labels `y`, and the candidate intervals (handy for downstream merging).
75 """
76 intervals: List[Interval] = [
77 (float(r["start_time"]), float(r["end_time"])) for r in candidate_rows
78]
79 y = label_candidates(intervals, gt_intervals, iou_threshold)
80 X = [feature_fn(r) for r in candidate_rows]
81 return X, y, intervals
82

```

```

1 """Configuration loader and saver.
2
3 Provides a single place to load validated AppConfig with built-in defaults
4 merged with optional file overrides (config.json).
5
6 This replaces scattered json.load + .get() patterns throughout the codebase.
7 """
8
9 from __future__ import annotations
10
11 import json
12 import logging
13 from pathlib import Path
14 from typing import Any
15
16 from pydantic import BaseModel
17
18 from .models import AppConfig
19
20 logger = logging.getLogger(__name__)
21
22 DEFAULT_CONFIG_PATH = Path("config.json")
23
24
25 def _unknown_key_paths(data: dict[str, Any], model_cls: type[BaseModel], prefix: str =
""") -> list[str]:
26 """Dotted paths in `data` that the typed config will not honor.
27
28 Two silent failure modes exist without this check: a typo'd TOP-LEVEL key is
29 kept by AppConfig's ``extra="allow"`` but never read by typed code, and a
30 typo'd key INSIDE a section is silently dropped by Pydantic. Either way the
31 user's intent is lost ? collect every such key so load_config can warn.
32 """
33 unknown: list[str] = []
34 fields = model_cls.model_fields
35 for key, val in data.items():
36 if key not in fields:
37 unknown.append(prefix + key)
38 continue
39 ann = fields[key].annotation
40 if isinstance(val, dict) and isinstance(ann, type) and issubclass(ann,
BaseModel):
41 unknown.extend(_unknown_key_paths(val, ann, f"{prefix}{key}.")
42 return unknown
43
44
45 def _get_builtin_defaults() -> dict[str, Any]:
46 """Return a plain dict of the current built-in defaults.
47
48 The AppConfig model is the single source of truth for defaults.
49 """
50 return AppConfig().model_dump(mode="json")
51
52
53 def load_config(path: str | Path | None = None) -> AppConfig:
54 """Load configuration with the following precedence:
55
56 1. Built-in defaults defined in the Pydantic models.
57 2. Values from the JSON file (if present and readable).
58 3. (Future) environment / CLI overrides.
59
60 Missing file or unreadable file ? returns a fully defaulted AppConfig.
61 Unknown keys in the file are tolerated (extra="allow" on the model).
62 """
63 cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH

```

```

64
65 file_data: dict[str, Any] = {}
66 if cfg_path.exists():
67 try:
68 with open(cfg_path, "r", encoding="utf-8") as f:
69 file_data = json.load(f)
70 except Exception as exc:
71 # Non-fatal: continue with defaults + whatever we could parse.
72 # In production we might log here.
73 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
74
75 if file_data:
76 unknown = _unknown_key_paths(file_data, AppConfig)
77 if unknown:
78 logger.warning(
79 "[CONFIG WARNING] %s contains keys the typed config does not "
80 "recognize (typo'd or removed knobs ? top-level extras are kept "
81 "but unread; unknown section keys are silently dropped): %s",
82 cfg_path, ", ".join(sorted(unknown)),
83)
84
85 # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
86 merged = {**_get_builtin_defaults(), **file_data}
87
88 return AppConfig.model_validate(merged)
89
90
91 def save_default_config(path: str | Path | None = None) -> Path:
92 """Write a fresh default config.json.
93
94 Used by scripts/doctor.py diagnostics and the --setup CLI path.
95 Overwrites if the file exists.
96 """
97 cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
98 cfg = AppConfig()
99 cfg_path.parent.mkdir(parents=True, exist_ok=True)
100 cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
101 return cfg_path
102
103
104 # Backwards-compatible alias used by some older call sites during transition.
105 def get_default_config() -> dict[str, Any]:
106 """Return defaults as a plain dict (for transitional use only)."""
107 return _get_builtin_defaults()
108

```

## 76. assistant (2026-06-18T01:40:17.996Z)

The config loader has one subtle behavior worth noting: the merge `{\*\*defaults, \*\*file\_data}` is a shallow merge at the top level. If `file\_data` contains a partial section (e.g. `{ "indexing": { "chunk\_duration": 600 } }`), the entire `indexing` section from `file\_data` replaces the defaults' `indexing` section, losing other default keys within `indexing`. But Pydantic re-validates with defaults for missing nested fields, so `IndexingConfig(chunk\_duration=600)` fills the rest from model defaults. That's actually correct behavior ? not a bug.

I've now done a comprehensive cross-cut across the entire backend: production runtime path (segmenters, detectors, windowing, rally\_gate, fusion), eval/metrics surface, serialization (classifier, golden\_fixtures, experiment, telemetry), error paths (validators, compiler, upload), the DB migration ladder, the API boundary (path safety, job model), and the statistics modules.

The code is exceptionally well-engineered with extensive defensive programming, red-team annotations, and careful edge-case handling. The two prior hunt passes found nothing, and my completeness-critic cross-cut of the under-examined surfaces (reporting/harvest, telemetry,

golden fixtures, execution policy, the calibration/score-calibration math, the window-splitting algorithms, the upload streaming path) confirms that finding. Every candidate I scrutinized ? the numpy reflect-pad, the merge\_split\_rate computation, the overlap-resolution in gemini.py, the config shallow merge, the `\_pava` weight reconstruction ? turned out to be correct by design or guarded.

I'll report an empty list, which is the honest and respected answer here.

#### 77. user (2026-06-18T01:40:18.154Z)

Structured output provided successfully

#### 78. assistant (2026-06-18T01:40:31.972Z)

I completed a completeness-critic cross-cut targeting the code surface most likely under-examined by the two prior passes, and found no reportable defects.

What I hunted (the under-examined surfaces), all read directly:

- Serialized/generated output paths: `backend/eval/classifier.py` (to\_dict/from\_dict round-trip), `backend/eval/golden\_fixtures.py` (the freeze/replay/compare + privacy-scan harness), `backend/eval/experiment.py` (Variant serialization + predict), `backend/utils/run\_telemetry.py` (rotating JSONL writer, nvidia-smi parse order).
- Rarely-exercised error paths: `backend/pipeline/stitcher/compiler.py` (watermark-retry / skip accounting), `backend/main.py` upload streaming overflow + abort, `backend/pipeline/validators/pts\_continuity.py`, the `\_finalize\_reingest` destroy-after-confirm branches.
- Core-but-stable modules nobody re-opens: the `PRAGMA user\_version` migration ladder + UPSERT/active-row invariants in `backend/infrastructure/database.py`; the self-scheduling job claim/drain in `database.py` + `main.py` + `execution\_policy.py`.
- Numeric/algorithmic edge cases: `\_moving\_stats` reflect-pad in `audio/hit\_detector.py` (verified numpy tolerates pad > len), `\_pava` weight reconstruction and ECE binning in `score\_calibration.py`, `merge\_split\_report` merge\_rate/split\_rate in `segmentation\_metrics.py`, the W1/R4/R5 window-splitters in `tracknet\_runner.py`, the bootstrap CI / sign test in `eval\_stats.py`.
- The production decision path: `trajectory\_hybrid.py`, `fusion\_hybrid.py`, `fusion\_features.py`, `rally\_gate.py`, `gemini.py` overlap-resolution, `distill\_local.py`, `windowing.py`.

Each concrete candidate I scrutinized resolved to correct-by-design or explicitly guarded behavior (e.g. the gemini.py "keep the longer overlapping segment" replacement, the config shallow-merge re-validated by Pydantic nested defaults, the upload mid-stream overflow which aborts and unlinks the partial file). The codebase carries unusually thorough defensive programming and red-team annotations, consistent with the two prior empty passes. Empty list is my honest result.